Beyond Isotropy in JEPAs: Hamiltonian Geometry and Symplectic Prediction

Robert Jenkinson Alvarez [1]

Abstract

JEPAs often regularize embeddings toward an isotropic Gaussian, implicitly baking Euclidean symmetry into the representation. We show that when downstream tasks and token geometries carry structure, the canonical target is instead a Hamiltonian Gaussian $\mathcal{N}(0, \alpha H^{-1})$ for a symmetric positive definite (SPD) operator H, and enforcing Euclidean isotropy can be provably suboptimal via a closed-form price of isotropy bound. A minimax and maximum-entropy analysis under a Hamiltonian energy budget singles out $\mathcal{N}(0, \alpha H^{-1})$ as the canonical structured target. We introduce *HamJEPA*, which encodes views into a phase-space state (q, p) and predicts via a symplectic integrator; Liouville volume preservation holds for the predictor map, and encoder collapse is prevented using a log-determinant volume floor and fixed-unit scale budgets. This motivates a practical route beyond Euclidean-isotropic latent regularization. HamJEPA improves frozen-feature performance over SIGReg: on CIFAR-100 it increases linear-probe top-1 by $+3.75$ points (30 epochs) and $\approx +10.6$ points (80 epochs), and kNN@20 top-1 by $+4.89$ points (30 epochs) and $\approx +6.5$ points (80 epochs); on ImageNet-100 (45 epochs) it increases linear-probe top-1 by $+6.54$ points and kNN best-over-k top-1 by $+4.26$ points (best over $q/p/(q,p)$ readouts).

1. Introduction

Joint Embedding Predictive Architectures (JEPAs) learn representations by predicting one embedding from another. For a fixed predictive loss, downstream performance depends on embedding geometry: scale, anisotropy, and collapse. We study objectives of the form $\min_\theta \; \mathcal{L}_{\text{JEPA}}(\theta) + \lambda \, \mathcal{L}_{\text{reg}}(Z_\theta)$, where $\mathcal{L}_{\text{reg}}$ regularizes the embedding distribution induced by the encoder.

A standard target is an *isotropic Gaussian* embedding distribution. LeJEPA motivates this choice under a "no downstream structure" viewpoint and enforces it with Sketched Isotropic Gaussian Regularization (SIGReg) (Balestriero & LeCun, 2025). Isotropy is a symmetry assumption: it treats all embedding directions as exchangeable. Throughout, "Euclidean isotropy" refers to second-order geometry ($\Sigma \propto I$) in the ambient coordinates; it does *not* mean that representations literally lie on an ℓ_2 sphere. In many settings (patch-token grids, sequences, graphs), the index set carries known structure that induces preferred correlations and invariances; enforcing Euclidean isotropy can misalign the representation with this geometry.

This paper makes two complementary contributions that go beyond Euclidean isotropy as a latent design principle. First, we develop a *theory of Hamiltonian embedding priors and the price of isotropy*. Fixing a structured symmetric positive definite (SPD) operator $H \succ 0$ and an H-energy budget $\mathrm{tr}(H\Sigma) = c$, we study structured downstream linear task families of the form $\{w : w^\top H^{-1} w \le 1\}$. We show that the unique minimax-optimal covariance is $\Sigma^\star = (c/d)H^{-1}$, and that among all distributions satisfying the same budget the unique maximum-entropy law is the Hamiltonian Gaussian $Z \sim \mathcal{N}(0, (c/d)H^{-1})$. Conversely, enforcing Euclidean isotropy $\Sigma \propto I$ under the same Hamiltonian budget can incur a worst-case multiplicative gap $\rho(H) = d\,\lambda_{\max}(H)/\mathrm{tr}(H)$ when $H \not\propto I$. Directly enforcing a fixed structured marginal can be brittle under mis-specified H and non-Gaussian encoder dynamics. This motivates a complementary approach that imposes Hamiltonian structure on the predictive map rather than on the encoder marginal.

Second, we move beyond prescribing isotropic-Gaussian marginals and instead constrain *how views relate* in representation space. We introduce **HamJEPA**, which represents each view as a phase-space state $s = (q, p)$ and predicts view-to-view transitions using a symplectic (volume-preserving) map obtained from Hamiltonian dynamics with a scalar *energy* $\mathcal{H}_\phi(q, p)$ (distinct from the SPD operator H above) and a symplectic integrator. This imposes a strong structural prior on the predictor (symplecticity, invertibility, and stable multi-step evolution) without enforcing an isotropic marginal for the encoder. Symplectic volume preservation constrains the predictor but does not by itself prevent encoder collapse, hence HamJEPA pairs the symplectic predictor with an *encoder-side* non-

[1]Independent. Correspondence to: Robert Jenkinson Alvarez <robert.j.jenkinson.alvarez@bath.edu>.

Preprint. March 3, 2026

degeneracy bridge that avoids Gaussian isotropy: a fixed-unit norm/energy budget to control scale and a projected log-determinant volume floor to prevent rank collapse. A useful bridge between these viewpoints is that the oracle Gaussian $\mathcal{N}(0, \frac{c}{d}H^{-1})$ is exactly the q-marginal of a quadratic Hamiltonian Gibbs law on phase space; HamJEPA can therefore be read as a practical unknown-H Hamiltonian relaxation of the oracle geometry from Section 3.

Code and models: GitHub: HamJEPA-HamSIGReg.

1.1. Positioning relative to prior regularizers and dynamics priors

Euclidean JEPA regularizers. Within JEPA-style self-supervised learning (Assran et al., 2023), LeJEPA proposes regularizing Euclidean embeddings toward an isotropic Gaussian and implements this through SIGReg, a sliced characteristic-function (CF) matching objective (Balestriero & LeCun, 2025). KerJEPA broadens this discrepancy family by replacing the specific sliced CF construction with kernel-based discrepancies, but remains within the same Euclidean marginal-regularization viewpoint (Zimmermann et al., 2025). HamJEPA departs from this line: rather than prescribing a Euclidean-isotropic target marginal, we ask when Euclidean isotropy is theoretically misaligned with structured task geometry and instead place the structure in the predictive map.

Moment-based and coding-rate anti-collapse objectives. In the broader SSL literature, several anti-collapse objectives act mainly through low-order statistics. Barlow Twins reduces redundancy through cross-correlation shaping, while VICReg combines invariance with explicit variance and covariance regularization (Zbontar et al., 2021; Bardes et al., 2021). A neighboring family uses coding-rate or log-determinant criteria on covariance- or Gram-type objects to encourage spectral spread or representational volume, as in MCR^2 and, more recently, coding-rate-regularized self-distillation methods such as SimDINO (Yu et al., 2020; Wu et al., 2025). HamJEPA's projected log-determinant, participation-ratio, and top-eigenvalue controls belong to this broad anti-collapse family; we do not claim these penalties are novel in isolation.

Dynamics priors. On the dynamics side, Hamiltonian Neural Networks, Hamiltonian Generative Networks, and Symplectic ODE-Net impose energy-based or symplectic structure on learned dynamics, typically in settings with temporal trajectories or control signals (Greydanus et al., 2019; Toth et al., 2019; Zhong et al., 2019). HamJEPA borrows this structural idea but uses it in a different role: the Hamiltonian module is not a physics model of observed time-series data, but a symplectic predictor prior inside a JEPA trained from paired augmentations of the same sample.

What HamJEPA does and does not constrain. HamJEPA should therefore not be read as an all-moments marginal-matching objective. In contrast to SIGReg, whose CF matching is an all-moments constraint in principle when the target CF is matched exactly, the encoder-side HamJEPA bridge is primarily first/second-order: mean control, fixed second-moment budgets, variance floors, and projected covariance-shape constraints. The higher-order structure comes from the nonlinear symplectic rollout between views, i.e. from constraining cross-view transitions rather than explicitly matching a target marginal distribution.

Scope of novelty. The contribution of HamJEPA is thus the combination of (i) a theory showing when Euclidean isotropy is misaligned with structured task geometry, and (ii) a practical JEPA training mechanism that injects Hamiltonian structure into the predictor while maintaining non-collapse without enforcing Euclidean-isotropic marginals.

2. Background and Setup

Data, views, and randomness. Let $x \sim \mathcal{D}$ be a data sample and let $\tau \sim \mathcal{A}$ be a random augmentation. A *view* is $v := \tau(x)$. We sample paired views of the same sample:

$$v_a = \tau_a(x), \qquad v_b = \tau_b(x). \tag{1}$$

Encoders and representations. We use two representation parameterizations.

(A) Euclidean embedding. An encoder $f_\theta : \mathcal{X} \to \mathbb{R}^d$ maps a view to an embedding random variable

$$Z \;:=\; f_\theta(V) \in \mathbb{R}^d, \tag{2}$$

where V denotes a generic random view. Assume finite second moments. Let $\mu_\theta := \mathbb{E}[Z]$ and $\Sigma_\theta := \mathrm{Cov}(Z) \succeq 0$. We write centered embeddings as $\widetilde{Z} := Z - \mu_\theta$ so $\mathbb{E}[\widetilde{Z}] = 0$ and $\mathrm{Cov}(\widetilde{Z}) = \Sigma_\theta$.

(B) Phase-space state (HamJEPA). A phase-space encoder $E_\theta : \mathcal{X} \to \mathbb{R}^{2d_0}$ outputs a state

$$S := E_\theta(V) = (Q, P), \qquad Q, P \in \mathbb{R}^{d_0}. \tag{3}$$

We interpret Q as a *content* coordinate and P as an auxiliary *momentum* coordinate used to model structured view-to-view transitions. We write projections $\pi_q(Q, P) = Q$ and $\pi_p(Q, P) = P$.

Batch (row) convention. For a minibatch of size n, we stack embeddings/states as row vectors: $\mathbf{Z} \in \mathbb{R}^{n\times d}$ has rows $z_i^\top$, and $\mathbf{S} \in \mathbb{R}^{n\times 2d_0}$ has rows $s_i^\top = (q_i^\top, p_i^\top)$. We center batches by subtracting the row-mean, e.g. $\widetilde{\mathbf{Z}} = \mathbf{Z} - \frac{1}{n}\mathbf{1}\mathbf{1}^\top\mathbf{Z}$.

JEPA predictive loss (abstract). We treat the predictive component as a loss $\mathcal{L}_{\text{JEPA}}$ computed from paired views (1) (and possibly a predictor network). We use $\text{sg}(\cdot)$ to denote stop-gradient on targets. Our focus is how additional structure shapes representation geometry and prevents collapse.

Baseline Gaussian regularization (SIGReg). LeJEPA employs Sketched Isotropic Gaussian Regularization (SIGReg) (Balestriero & LeCun, 2025), which encourages embeddings to match an isotropic Gaussian in a slicing/sketching sense. We treat SIGReg as a black box and use it only as a baseline reference point; our main method does not rely on imposing isotropic Gaussian marginals.

Structured geometry via an SPD operator H (for theory). We will formalize "structured geometry" on $\mathbb{R}^d$ using a symmetric positive definite operator $H \in \mathbb{R}^{d\times d}$, written $H \succ 0$. It defines a quadratic energy

$$E_H(z) := \tfrac{1}{2} z^\top H z, \tag{4}$$

and an associated Hamiltonian (precision-form) Gaussian family

$$\mathcal{N}\big(0, \alpha H^{-1}\big), \qquad \alpha > 0. \tag{5}$$

Equivalently, in H-isotropic coordinates $z' := H^{1/2} z$ we have $z' \sim \mathcal{N}(0, \alpha I)$. Under the row convention, applying the same per-sample transform to a batch corresponds to

$$\mathbf{Z}' := \mathbf{Z} H^{1/2}. \tag{6}$$

Section 3 uses H to derive a canonical covariance and a closed-form price-of-isotropy bound.

Structured tasks and a Hamiltonian energy budget (for theory). We model downstream linear tasks by $y = w^\top \widetilde{Z}$ and constrain task complexity by the H^{-1}-unit ball

$$\mathcal{W}_H := \{w \in \mathbb{R}^d : w^\top H^{-1} w \le 1\}. \tag{7}$$

We pair this with a representation-energy budget in H-geometry:

$$\mathbb{E}\Big[\widetilde{Z}^\top H \widetilde{Z}\Big] = \text{tr}(H\Sigma_\theta) = c, \qquad c > 0. \tag{8}$$

Given $\Sigma \succeq 0$, we will study the worst-case task variance

$$V(\Sigma) := \sup_{w \in \mathcal{W}_H} w^\top \Sigma w \;=\; \lambda_{\max}\big(H^{1/2}\Sigma H^{1/2}\big), \tag{9}$$

which is the central object in our minimax characterization and the resulting price-of-isotropy theorem.

Two Hamiltonians (to avoid notation collisions). Section 3 uses an SPD operator $H \succ 0$ to specify a *geometry* on embedding space. HamJEPA uses a *scalar energy* $\mathcal{H}_\phi(q,p)$ generating a symplectic map on phase space. We reserve H for the SPD operator and $\mathcal{H}_\phi$ for the scalar energy.

Why symplectic dynamics, and why leapfrog? Once H is unknown, the Hamiltonian bias is most naturally placed in the *predictive map* rather than in a fixed encoder marginal. A generic predictor can arbitrarily contract local volume and distort the geometry that Section 3 is meant to preserve. By contrast, a symplectic predictor preserves phase-space volume and admits a meaningful inverse-time interpretation. For the separable Hamiltonian form used below, leapfrog is the standard explicit reversible symplectic discretization, so it is the minimal geometry-preserving rollout compatible with the phase-space model.

Hamiltonian dynamics and symplectic maps. HamJEPA uses a different notion of "Hamiltonian": a *scalar energy function* $\mathcal{H}_\phi : \mathbb{R}^{2d_0} \to \mathbb{R}$ that generates dynamics on phase space (distinct from the SPD operator H above). Let

$$J := \begin{pmatrix} 0 & I \\ -I & 0 \end{pmatrix}, \tag{10}$$

and consider Hamilton's ODE $\dot{s} = J\nabla_s \mathcal{H}_\phi(s)$ with flow map $\Phi_{\phi,t}$. In practice we use a discrete-time symplectic integrator (leapfrog/Verlet) with step size Δt and K steps, yielding a predictor map $\hat{s} = \Phi_\phi^{(K)}(s)$. For separable energies $\mathcal{H}_\phi(q,p) = T(p) + V_\phi(q)$, leapfrog defines a symplectic (volume-preserving) map:

$$(D\Phi_\phi^{(K)}(s))^\top J (D\Phi_\phi^{(K)}(s)) = J \Rightarrow \det(D\Phi_\phi^{(K)}(s)) = 1. \tag{11}$$

Non-invertible views and encoder non-degeneracy (preview). Augmentations in (1) are generally non-invertible in pixel space. Accordingly, HamJEPA treats Q as the content coordinate while allowing P to serve as an auxiliary state for dynamics. Symplecticity constrains the predictor map (11) but does not by itself prevent encoder collapse; HamJEPA therefore adds encoder-side non-degeneracy constraints (fixed-unit scale budgets and volume/effective-rank floors), defined in Section 4.3.

3. Hamiltonian Embedding Priors and the Price of Isotropy

This section is pure theory: given a structured notion of task complexity and an energy budget, what covariance/distribution is canonical, and what is the cost of forcing Euclidean isotropy? Throughout, fix an SPD operator $H \succ 0$ and let d denote the embedding dimension. We work with the centered embedding $\widetilde{Z} := Z - \mathbb{E}[Z]$ and write $\Sigma := \text{Cov}(\widetilde{Z})$.

3.1. Setup: structured tasks and a Hamiltonian budget

We model downstream *linear* tasks by $y = w^\top \widetilde{Z}$ and constrain task complexity by the H^{-1}-unit ball

$$\mathcal{W}_H := \{w \in \mathbb{R}^d : \; w^\top H^{-1} w \le 1\}. \tag{12}$$

We pair this with a representation energy budget in the H-geometry:

$$\mathbb{E}[\widetilde{Z}^\top H \widetilde{Z}] = \mathrm{tr}(H\Sigma) = c, \qquad c > 0. \tag{13}$$

Given $\Sigma \succeq 0$, define the worst-case task variance functional

$$V(\Sigma) := \sup_{w \in \mathcal{W}_H} w^\top \Sigma w. \tag{14}$$

The next lemma gives a convenient spectral form.

Lemma 3.1 (Spectral form of $V(\Sigma)$)**.** *For any $\Sigma \succeq 0$,*

$$V(\Sigma) = \lambda_{\max}\big(H^{1/2}\Sigma H^{1/2}\big). \tag{15}$$

Moreover, if $u_{\max}$ is a unit top eigenvector of $H^{1/2}\Sigma H^{1/2}$, then $w_{\max} := H^{1/2} u_{\max}$ attains the supremum in (14).

Proof. See Appendix A.1.1. □

3.2. Minimax covariance under a Hamiltonian budget

We ask for the covariance that minimizes the worst-case task variance (14) under the H-budget (13).

Theorem 3.2 (Minimax covariance under a Hamiltonian budget)**.** *Fix $H \succ 0$ and $c > 0$. Consider*

$$\min_{\Sigma \succeq 0:\ \mathrm{tr}(H\Sigma) = c} V(\Sigma). \tag{16}$$

The unique minimizer is

$$\Sigma^\star = \frac{c}{d} H^{-1}, \tag{17}$$

and the optimal value is

$$\min_{\Sigma \succeq 0:\ \mathrm{tr}(H\Sigma) = c} V(\Sigma) = \frac{c}{d}. \tag{18}$$

Equivalently, $H^{1/2}\Sigma^\star H^{1/2} = \frac{c}{d} I$.

Proof. See Appendix A.1.2. □

3.3. Maximum entropy: the canonical distribution

The minimax result selects a canonical covariance. A complementary principle selects a canonical *distribution*.

Theorem 3.3 (Maximum entropy under a Hamiltonian energy constraint)**.** *Let Z be an $\mathbb{R}^d$-valued random vector with a density, $\mathbb{E}[Z] = 0$, and finite second moments. Among all such distributions satisfying*

$$\mathbb{E}[Z^\top H Z] = c, \tag{19}$$

the unique maximizer of differential entropy is the Gaussian

$$Z \sim \mathcal{N}\Big(0, \frac{c}{d} H^{-1}\Big). \tag{20}$$

Proof. See Appendix A.1.3. □

3.4. Price of Euclidean isotropy

Many practical objectives encourage Euclidean isotropy, i.e., $\Sigma \propto I$ in the original coordinates. This subsection quantifies the worst-case gap incurred by imposing Euclidean isotropy under the same Hamiltonian budget.

Under the restriction $\Sigma = \sigma^2 I$, the constraint $\mathrm{tr}(H\Sigma) = c$ forces the unique feasible Euclidean-isotropic covariance

$$\Sigma_{\mathrm{iso}} = \frac{c}{\mathrm{tr}(H)} I. \tag{21}$$

Theorem 3.4 (Price of Euclidean isotropy)**.** *Fix $H \succ 0$ and $c > 0$. Let $\Sigma^\star$ be as in Theorem 3.2 and let Σ_{iso} be as in (21). Then*

$$V(\Sigma_{\mathrm{iso}}) = \frac{c}{\mathrm{tr}(H)} \lambda_{\max}(H), \tag{22}$$

$$\frac{V(\Sigma_{\mathrm{iso}})}{V(\Sigma^\star)} = \rho(H) := \frac{d\,\lambda_{\max}(H)}{\mathrm{tr}(H)}. \tag{23}$$

Moreover $1 \le \rho(H) \le d$, with $\rho(H) = 1$ if and only if $H \propto I$.

Proof. See Appendix A.1.4. □

Proposition 3.5 (Metric isotropy equivalence)**.** *Let $H \succ 0$ and let $Z \in \mathbb{R}^d$ be a random vector with finite second moments and covariance $\Sigma := \mathrm{Cov}(Z)$. Define the H-whitened representation $Z' := H^{1/2} Z$. Then*

$$\mathrm{Cov}(Z') \;=\; H^{1/2}\Sigma H^{1/2}.$$

In particular, for any $\alpha > 0$,

$$\Sigma = \alpha H^{-1} \quad \Longleftrightarrow \quad \mathrm{Cov}(Z') = \alpha I.$$

Proof. See See Appendix A.1.5. □

3.5. A quadratic Hamiltonian lift of the oracle structured Gaussian

The structured Gaussian selected by Theorems 3.2 and 3.3 admits a natural lift to a Gibbs law on phase space.

Proposition 3.6 (Quadratic Hamiltonian lift of the oracle covariance)**.** *Fix $H \succ 0$ and $c > 0$, and define the quadratic Hamiltonian*

$$\mathcal{H}_H(q,p) := \tfrac{1}{2} q^\top H q + \tfrac{1}{2}\|p\|_2^2, \qquad q, p \in \mathbb{R}^d. \tag{24}$$

Consider the Gibbs density on phase space

$$\pi_H(q,p) \propto \exp\Big(-\frac{d}{c}\mathcal{H}_H(q,p)\Big). \tag{25}$$

Then q and p are independent under π_H, with

$$q \sim \mathcal{N}\Big(0, \frac{c}{d} H^{-1}\Big), \qquad p \sim \mathcal{N}\Big(0, \frac{c}{d} I\Big). \tag{26}$$

Moreover,

$$\mathbb{E}_{\pi_H}[q^\top H q] = c, \qquad \mathbb{E}_{\pi_H}[\|p\|_2^2] = c, \qquad \mathbb{E}_{\pi_H}[\mathcal{H}_H(q,p)] = c. \tag{27}$$

In particular, the q-marginal of π_H is exactly the minimax-optimal and maximum-entropy law from Theorems 3.2 and 3.3.

Proof. See Appendix A.1.6. □

Proposition 3.7 (The oracle family spans the SPD cone)**.** *Fix $c > 0$. As H ranges over SPD operators, the family*

$$\Sigma^\star(H) = \frac{c}{d} H^{-1} \tag{28}$$

ranges over the entire SPD cone. Equivalently, for every $\Sigma \succ 0$, choosing

$$H = \frac{c}{d} \Sigma^{-1} \tag{29}$$

yields $\Sigma^\star(H) = \Sigma$.

From the oracle covariance to a practical unknown-H relaxation. Proposition 3.6 shows that when H is known, the oracle covariance $\Sigma^\star = \frac{c}{d} H^{-1}$ is exactly the q-marginal of a quadratic Hamiltonian Gibbs model on phase space. In that quadratic oracle case, the induced dynamics are

$$\dot{q} = p, \qquad \dot{p} = -Hq, \tag{30}$$

so a Hamiltonian predictor is not an unrelated construction, but a natural dynamical realization of the same structured geometry.

This does *not* mean that the practical training mechanism is uniquely determined by Section 3. Rather, the theory should be read as an oracle statement: when H is known, the correct geometry is Hamiltonian-anisotropic rather than Euclidean-isotropic. When H is unknown, Proposition 3.7 shows that the oracle family $\{\Sigma^\star(H) : H \succ 0\}$ spans the entire SPD cone, so the theory no longer singles out one fixed covariance target.

4. HamJEPA: Symplectic Predictive Structure Without Isotropic Marginals

Section 3 identifies the oracle structured geometry when H is known. Proposition 3.6 shows that the oracle Gaussian $\mathcal{N}(0, \frac{c}{d} H^{-1})$ is exactly the q-marginal of a quadratic Hamiltonian Gibbs law on phase space, while Proposition 3.7 shows that, as H varies, the oracle covariance family spans the entire SPD cone. HamJEPA is not claimed to be the unique consequence of that oracle analysis. HamJEPA instead injects Hamiltonian structure into the *predictive map between views* by restricting the predictor to a symplectic (volume-preserving) family, while preventing encoder collapse via explicit non-degeneracy constraints that do *not* impose isotropic Gaussian marginals. A visual overview of HamJEPA is provided in Appendix D.

Row/column convention. A single state $s = (q,p) \in \mathbb{R}^{2d_0}$ is treated as a *column* vector. A minibatch is stacked row-wise as $\mathbf{S} \in \mathbb{R}^{B \times 2d_0}$ (each row is $s_i^\top$). All maps below act row-wise on $\mathbf{S}$.

4.1. Phase-space encoder

Given paired views v_a, v_b as defined in Section 2, the encoder $E_\theta : \mathcal{X} \to \mathbb{R}^{2d_0}$ outputs phase-space states $s_a = E_\theta(v_a)$ and $s_b = E_\theta(v_b)$ with $s = (q,p) \in \mathbb{R}^{2d_0}$.

$$s = E_\theta(v) = (q,p), \qquad q,p \in \mathbb{R}^{d_0}. \tag{31}$$

We interpret q as a *content* coordinate used for prediction and evaluation, and p as an auxiliary *momentum* coordinate enabling structured transitions. We write $\pi_q(q,p) = q$ and $\pi_p(q,p) = p$.

Observable coordinate vs. conjugate latent. Section 3 is formulated in terms of the representation used by downstream tasks; in HamJEPA, that conceptual role is played by q. The auxiliary coordinate p is the conjugate latent that makes Hamiltonian transport possible. For non-invertible augmentations, this makes q-matching the cleanest observable formulation, since the target need not contain a uniquely recoverable dynamical state. In practice, however, the exact matching choice is a stability knob rather than a theorem: on CIFAR-100 we primarily use q-matching, while on ImageNet-100 we found full-state (q,p)-matching more stable and therefore use it there.

4.2. Symplectic predictor via a separable Hamiltonian and leapfrog

Canonical symplectic structure. Let

$$J := \begin{pmatrix} 0 & I \\ -I & 0 \end{pmatrix} \in \mathbb{R}^{2d_0 \times 2d_0} \tag{32}$$

denote the canonical symplectic matrix.

Hamilton's equations and flow map. Given a differentiable scalar energy $\mathcal{H}_\phi : \mathbb{R}^{2d_0} \to \mathbb{R}$, Hamilton's ODE on phase space is

$$\dot{s}(t) \;=\; J \nabla_s \mathcal{H}_\phi\big(s(t)\big), \qquad s(0) = s_0, \tag{33}$$

where J is the canonical symplectic matrix (32). We denote its time-t flow map by $\Phi_{\phi,t}$, i.e. $s(t) = \Phi_{\phi,t}(s_0)$.

Separable energy (the theory-tight choice). To obtain a discrete predictor map with rigorous symplectic guarantees under an explicit integrator, we use a separable Hamiltonian energy

$$\mathcal{H}_\phi(q,p) = T(p) + V_\phi(q), \qquad T(p) = \tfrac{1}{2}\|p\|_2^2, \tag{34}$$

where $V_\phi : \mathbb{R}^{d_0} \to \mathbb{R}$ is a learnable potential. Fixing $T(p) = \frac{1}{2}\|p\|^2$ is stabilizing and prevents scale-control constraints from being satisfied by rescaling a learnable kinetic metric.

Leapfrog (velocity Verlet) predictor map. Given step size $\Delta t > 0$ and $K \in \mathbb{N}$ steps, define the leapfrog update

$$\begin{aligned} p_{k+\frac{1}{2}} &= p_k - \tfrac{\Delta t}{2}\,\nabla_q V_\phi(q_k), \\ q_{k+1} &= q_k + \Delta t\, p_{k+\frac{1}{2}}, \qquad\qquad k = 0, \dots, K-1. \\ p_{k+1} &= p_{k+\frac{1}{2}} - \tfrac{\Delta t}{2}\,\nabla_q V_\phi(q_{k+1}), \end{aligned} \tag{35}$$

This defines a one-step map $\Psi_{\phi,\Delta t} : \mathbb{R}^{2d_0} \to \mathbb{R}^{2d_0}$ and a K-step predictor

$$\Phi_\phi^{(K)} := \Psi_{\phi,\Delta t}^{K}. \tag{36}$$

Given $s_a = (q_a, p_a)$, we predict $\hat{s}_b = (\hat{q}_b, \hat{p}_b)$ by

$$\hat{s}_b = \Phi_\phi^{(K)}(s_a). \tag{37}$$

4.3. Objective: content prediction plus non-isotropic anti-collapse

The projected log-determinant and participation-ratio penalties should be understood as an encoder-side anti-collapse bridge, not as the primary novelty of the method; similar covariance-volume objectives appear elsewhere, whereas our contribution is to couple such non-collapse controls with a phase-space JEPA and a symplectic predictive prior.

Let $s_a = (q_a, p_a) = E_\theta(v_a)$, $s_b = (q_b, p_b) = E_\theta(v_b)$, and $\hat{s}_b = (\hat{q}_b, \hat{p}_b) = \Phi_\phi^{(K)}(s_a)$. HamJEPA optimizes

$$\begin{aligned} \min_{\theta,\phi}\ & \mathcal{L}_{\text{pred}} + \lambda_{\text{bi}}\mathcal{L}_{\text{bi}} + \lambda_{\text{stab}}\mathcal{L}_{\text{stab}} + \lambda_{\text{budget}}\mathcal{L}_{\text{budget}} \\ & + \lambda_{\text{vol}}\mathcal{L}_{\text{vol}} + \lambda_{\text{pr}}\mathcal{L}_{\text{pr}} + \lambda_{\text{wd}}\mathcal{R}_{\text{wd}}(\theta,\phi). \end{aligned} \tag{38}$$

All expectations are implemented as minibatch averages; $\mathrm{sg}(\cdot)$ denotes stop-gradient.

(1) Content prediction. We match only the content coordinate:

$$\mathcal{L}_{\text{pred}} := \mathbb{E}\Big[\big\|\hat{q}_b - \mathrm{sg}(q_b)\big\|_2^2\Big]. \tag{39}$$

(2) Bidirectional prediction (optional but often beneficial). Define the backward prediction by running the *same* integrator with step size $-\Delta t$:

$$\hat{s}_a = \Phi_\phi^{(K)}(s_b;\, -\Delta t).$$

The bidirectional loss is

$$\begin{aligned} \mathcal{L}_{\text{bi}} := \mathbb{E}\Big[& \|\pi_q(\Phi_\phi^{(K)}(s_a;\, +\Delta t)) - \mathrm{sg}(q_b)\|_2^2 \\ & + \|\pi_q(\Phi_\phi^{(K)}(s_b;\, -\Delta t)) - \mathrm{sg}(q_a)\|_2^2\Big]. \end{aligned} \tag{40}$$

Time reversibility of leapfrog (Corollary 4.5) makes this an "exact inverse in-model" constraint rather than an independent predictor.

(3) Predictor stability (optional). A lightweight stabilizer is an energy drift penalty:

$$\mathcal{L}_{\text{stab}} := \mathbb{E}\Big[\big(\mathcal{H}_\phi(\hat{s}_b) - \mathrm{sg}(\mathcal{H}_\phi(s_a))\big)^2\Big]. \tag{41}$$

We keep λ_{stab} small; the primary structural constraint is symplecticity, not exact energy conservation.

(4) Fixed-units scale budget (anti-gauge). Symplecticity constrains the predictor map but does not prevent the encoder from collapsing all views to a point. We therefore enforce scale budgets in fixed units (no Gaussianity, no isotropy):

$$\mathcal{L}_{\text{budget}} := \Big(\frac{\mathbb{E}\|q\|_2^2}{d_0} - \alpha_q\Big)^2 + \Big(\frac{\mathbb{E}\|p\|_2^2}{d_0} - \alpha_p\Big)^2, \alpha_q, \alpha_p > 0. \tag{42}$$

(5) Projected log-det volume floor. A scale budget alone can be satisfied by concentrating variance in a low-dimensional subspace. To prevent rank collapse without forcing isotropy, we impose a projected log-det floor on the content batch. Let $Q \in \mathbb{R}^{B\times d_0}$ contain centered content codes: $Q_i = q_i - \frac{1}{B}\sum_{j=1}^{B} q_j$. Sample a random projection $R \in \mathbb{R}^{d_0\times k}$ with orthonormalized columns ($k \ll d_0$) and form $Y = QR \in \mathbb{R}^{B\times k}$. Define the regularized projected covariance

$$\Sigma_Y := \frac{1}{B-1} Y^\top Y + \varepsilon I_k, \qquad \varepsilon > 0, \tag{43}$$

and per-dimension log-volume

$$\ell_{\text{vol}} := \frac{1}{k}\log\det(\Sigma_Y). \tag{44}$$

We use the hinge penalty

$$\mathcal{L}_{\text{vol}} := \big[\max(0,\ \tau - \ell_{\text{vol}})\big]^2, \qquad \tau \in \mathbb{R}. \tag{45}$$

(6) Participation ratio floor (prevents "one-spike" solutions). Log-det alone can be satisfied with one very large eigenvalue and many tiny ones when scale is not tightly controlled, or when minibatch effects dominate. To explicitly rule out the "one-spike" regime, we add a participation ratio (PR) floor on Σ_Y:

$$\mathrm{PR}(\Sigma_Y) := \frac{\big(\mathrm{tr}(\Sigma_Y)\big)^2}{\mathrm{tr}(\Sigma_Y^2)} = \frac{\big(\sum_{i=1}^k \lambda_i\big)^2}{\sum_{i=1}^k \lambda_i^2} \in [1, k], \tag{46}$$

where $\lambda_1, \dots, \lambda_k$ are the eigenvalues of Σ_Y. We penalize violations of a target effective rank $r_0 \in (1, k]$:

$$\mathcal{L}_{\text{pr}} := \big[\max(0,\ r_0 - \mathrm{PR}(\Sigma_Y))\big]^2. \tag{47}$$

The combination (42)–(47) prevents collapse while still permitting anisotropy (unequal eigenvalues).

Moment perspective. HamJEPA does not attempt to match a full target marginal distribution. In contrast to SIGReg, whose characteristic-function matching is an all-moments objective in principle, the encoder-side HamJEPA bridge acts mainly at first and second order: mean control, fixed second-moment budgets, variance floors, and projected covariance-spectrum constraints (log-det, participation ratio, and top-eigenvalue fraction). The higher-order structure in HamJEPA enters through the nonlinear predictive dynamics induced by the symplectic rollout (through constraints on cross-view transitions rather than through explicit all-moments matching of the encoder marginal).

Moreover, *HamJEPA* does not constrain representations to lie on a unit sphere, nor does it enforce an isotropic Gaussian marginal; the scale budget controls expected energy, while allowing anisotropy and sample-dependent norms.

Curriculum on horizon. We often use a horizon curriculum: start with $K = 1$ and increase K during training while keeping Δt fixed. This strengthens supervision via multi-step prediction while remaining stable due to the structural constraints below.

4.4. Theory: symplecticity, volume preservation, and anti-collapse

This section records the structural guarantees enjoyed by HamJEPA when using the separable Hamiltonian (34) and the leapfrog integrator (35). All proofs are in Appendix A.2.

Row/column convention (linear transforms). The paper uses column vectors for single states/embeddings and row-stacked matrices for minibatches. The following lemma formalizes the equivalence.

Lemma 4.1 (Row/column convention for left-multiplication by $H^{1/2}$)**.** *Let $H \in \mathbb{R}^{d\times d}$ and let $z \in \mathbb{R}^d$ be a column vector. Define $z' := H^{1/2}z$. Let $\mathbf{Z} \in \mathbb{R}^{n\times d}$ be a row-stacked minibatch with rows $\mathbf{Z}_{i,:} = z_i^\top$. Define $\mathbf{Z}' := \mathbf{Z}H^{1/2}$. Then for every i,*

$$(\mathbf{Z}')^\top_{i,:} \;=\; H^{1/2}z_i.$$

Proof. See Appendix A.2.1. □

Symplecticity of the continuous flow. We first state the standard symplecticity property of Hamiltonian flows.

Theorem 4.2 (Symplecticity of Hamiltonian flows)**.** *Assume $\mathcal{H}_\phi \in C^2(\mathbb{R}^{2d_0})$ and let $\Phi_{\phi,t}$ denote the flow map of Hamilton's ODE (33). Then $\Phi_{\phi,t}$ is symplectic: for all s and t,*

$$\big(D\Phi_{\phi,t}(s)\big)^\top J\,\big(D\Phi_{\phi,t}(s)\big) \;=\; J. \tag{48}$$

Consequently, $\det(D\Phi_{\phi,t}(s)) = 1$ for all s, t, and $\Phi_{\phi,t}$ preserves phase-space volume (Liouville).

Proof. See Appendix A.2.2. □

Corollary 4.3 (Volume-preserving maps preserve differential entropy)**.** *Let $F : \mathbb{R}^m \to \mathbb{R}^m$ be a C^1 diffeomorphism and let X admit a density. Then*

$$h(F(X)) \;=\; h(X) + \mathbb{E}\big[\log|\det(DF(X))|\big].$$

In particular, if $|\det(DF(x))| = 1$ for all x (e.g., $F = \Phi_{\phi,t}$ from Theorem 4.2), then $h(F(X)) = h(X)$.

Proof. See Appendix A.2.3. □

Symplecticity of the discrete predictor. HamJEPA uses the leapfrog/Verlet map (35) as a discrete predictor. For separable Hamiltonians this map is symplectic.

Theorem 4.4 (Leapfrog is symplectic and volume-preserving for separable Hamiltonians)**.** *Let $\mathcal{H}_\phi(q,p) = T(p) + V_\phi(q)$ with $T(p) = \frac{1}{2}\|p\|_2^2$ and $V_\phi \in C^2$. Then the one-step leapfrog map $\Psi_{\phi,\Delta t}$ defined by (35) is symplectic. Therefore, for any $K \geq 1$, the K-step predictor $\Phi_\phi^{(K)} = \Psi_{\phi,\Delta t}^K$ is symplectic and satisfies*

$$\det\big(D\Phi_\phi^{(K)}(s)\big) = 1 \quad \textit{for all } s. \tag{49}$$

Proof. See Appendix A.2.4. □

Corollary 4.5 (Invertibility and time reversibility)**.** *Under the assumptions of Theorem 4.4, the leapfrog map is bijective and satisfies*

$$\Psi_{\phi,\Delta t}^{-1} = \Psi_{\phi,-\Delta t}. \tag{50}$$

In particular, the K-step predictor admits an exact inverse by running the same integrator with step size $-\Delta t$: $(\Phi_\phi^{(K)}(\cdot;\,+\Delta t))^{-1} = \Phi_\phi^{(K)}(\cdot;\,-\Delta t)$.

Proof. See Appendix A.2.5. □

Proposition 4.6 (Approximation of separable Hamiltonian flows by HamJEPA predictors)**.** *Fix a horizon $T > 0$ and a compact set $K \subset \mathbb{R}^{2d_0}$. Let $K_q := \{q \in \mathbb{R}^{d_0} : \exists p \text{ with } (q,p) \in K\}$.*

Let $V : \mathbb{R}^{d_0} \to \mathbb{R}$ be C^3 on an open neighborhood of a compact set $\Omega_q \supseteq K_q$, and assume ∇V is Lipschitz on Ω_q. Let Φ_t^V denote the time-t flow map of Hamilton's ODE

$$\dot q = p, \qquad \dot p = -\nabla V(q)$$

(i.e., the separable Hamiltonian $H(q,p) = \frac{1}{2}\|p\|_2^2 + V(q)$).

For a learnable potential $V_\phi \in C^3$ with ∇V_ϕ Lipschitz on Ω_q, let $\Psi_{\phi,\Delta t}$ be one leapfrog step (35) and let $\Phi_\phi^{(K)} := \Psi_{\phi,\Delta t}^K$ with $K\Delta t = T$ (36). Assume that for all initial states in K, both the exact trajectory under V and the exact trajectory under V_ϕ remain inside $\Omega := \Omega_q \times \Omega_p$ for all $t \in [0,T]$.

Then there exists a constant $C = C(T, \Omega, V_\phi) > 0$ such that

$$\sup_{(q,p)\in K} \left\|\Phi_\phi^{(K)}(q,p) - \Phi_T^V(q,p)\right\|_2 \le$$

$$C\Big(\underbrace{\sup_{q\in\Omega_q} \|\nabla V_\phi(q) - \nabla V(q)\|_2}_{\text{modeling error}} + \underbrace{\Delta t^2}_{\text{integrator error}}\Big).$$

In particular, if the potential class can approximate ∇V uniformly on Ω_q (and admits bounded derivatives on Ω_q), then for every $\varepsilon > 0$ there exist ϕ and $\Delta t > 0$ (hence $K = T/\Delta t$) such that

$$\sup_{(q,p)\in K} \left\|\Phi_\phi^{(K)}(q,p) - \Phi_T^V(q,p)\right\|_2 \le \varepsilon.$$

Proof. See Appendix A.2.6. □

Lemma 4.7 (Reciprocal singular-value pairing)**.** *Let $A \in \mathbb{R}^{2d_0\times 2d_0}$ be symplectic: $A^\top J A = J$. Then the singular values of A occur in reciprocal pairs: if σ is a singular value, so is $1/\sigma$ (with the same multiplicity). Equivalently, A cannot be contractive in all directions; any local contraction must be compensated by expansion.*

Proof. See Appendix A.2.7. □

Long-horizon stability (shadow Hamiltonians). A core practical advantage of symplectic integrators is the existence of a nearby modified energy that explains stable multi-step rollouts. We record a standard backward-error-analysis statement.

Theorem 4.8 (Shadow Hamiltonian / backward error analysis (standard))**.** *Assume V_ϕ is sufficiently smooth (e.g., analytic with bounded derivatives on the region visited by the numerical trajectory). Then for any integer $N \ge 1$ there exists a modified Hamiltonian $\widetilde{\mathcal{H}}_{\phi,\Delta t}^{[N]}$ such that:*

1. ***(Closeness)*** *$\widetilde{\mathcal{H}}_{\phi,\Delta t}^{[N]}(s) = \mathcal{H}_\phi(s) + \mathcal{O}(\Delta t^2)$ uniformly on the region of interest.*
2. ***(Modified-flow representation)*** *One leapfrog step equals the time-Δt flow of $\widetilde{\mathcal{H}}_{\phi,\Delta t}^{[N]}$ up to local error $\mathcal{O}(\Delta t^{N+1})$.*
3. ***(Near-conservation)*** *Along the leapfrog iterates (s_k), the modified energy $\widetilde{\mathcal{H}}_{\phi,\Delta t}^{[N]}(s_k)$ is nearly conserved, and the original energy error $\mathcal{H}_\phi(s_k) - \mathcal{H}_\phi(s_0)$ remains bounded over long horizons when Δt is small.*

Proof (sketch via BCH/backward error analysis). See Appendix A.2.8. □

Anti-collapse: what each ingredient does (and does not do). HamJEPA uses an *encoder-side* non-degeneracy bridge that avoids isotropic Gaussianity. It is useful to separate (i) scale control, (ii) rank/volume control, and (iii) spike control.

Lemma 4.9 (Scale budgets do not prevent rank collapse)**.** *Fix $c > 0$ and consider PSD matrices $\Sigma \succeq 0$ with $\mathrm{tr}(\Sigma) = c$. There exist such Σ with rank 1 (hence complete collapse except for one direction), e.g. $\Sigma = \mathrm{diag}(c, 0, \dots, 0)$.*

Proof. See Appendix A.2.9. □

Lemma 4.10 (Log-det alone does not prevent "one-spike" spectra)**.** *Fix $k \ge 2$ and $\tau \in \mathbb{R}$. There exist positive definite $\Sigma \in \mathbb{R}^{k\times k}$ with $\frac{1}{k}\log\det(\Sigma) = \tau$ but with $\mathrm{PR}(\Sigma)$ arbitrarily close to 1 (spike regime).*

Proof. Appendix A.2.10. □

Lemma 4.11 (Participation ratio alone does not fix scale)**.** *For any PSD $\Sigma \succeq 0$ and any scalar $a > 0$, $\mathrm{PR}(a\Sigma) = \mathrm{PR}(\Sigma)$. Thus a PR constraint alone does not prevent collapse by uniformly shrinking all eigenvalues.*

Proof. See Appendix A.2.11. □

Lemma 4.12 (Projected log-det detects full-space rank collapse)**.** *Let $Q \in \mathbb{R}^{d_0}$ be a zero-mean random vector with covariance $\Sigma_Q \succeq 0$ and let $R \in \mathbb{R}^{d_0\times k}$ have full column rank k (e.g., orthonormal columns as in Sec. 4.3). Define the population projected covariance*

$$\Sigma_{Y,0} := R^\top \Sigma_Q R \in \mathbb{R}^{k\times k}.$$

Then

$$\mathrm{rank}(\Sigma_{Y,0}) \le \mathrm{rank}(\Sigma_Q).$$

In particular, if $\mathrm{rank}(\Sigma_Q) < k$ then $\Sigma_{Y,0}$ is singular and therefore $\det(\Sigma_{Y,0}) = 0$ and $\log\det(\Sigma_{Y,0}) = -\infty$.

Moreover, if one uses the ridge-regularized version from (43),

$$\Sigma_Y := \Sigma_{Y,0} + \varepsilon I_k, \qquad \varepsilon > 0,$$

and if $r := \mathrm{rank}(\Sigma_{Y,0})$, then Σ_Y has exactly $k - r$ eigenvalues equal to ε and hence

$$\log\det(\Sigma_Y) = (k-r)\log\varepsilon + \sum_{i=1}^{r}\log(\lambda_i + \varepsilon),$$

where $\lambda_1, \dots, \lambda_r > 0$ are the nonzero eigenvalues of $\Sigma_{Y,0}$.

Proof. See Appendix A.2.12. □

Projected log-det floor. Recall the projected covariance Σ_Y from (43). The following proposition quantifies how a log-det floor plus a trace/scale control lower-bounds the smallest eigenvalue.

Proposition 4.13 (Projected log-det floor prevents rank collapse)**.** *Let $\Sigma_Y \in \mathbb{R}^{k\times k}$ be SPD with eigenvalues $\lambda_1 \geq \cdots \geq \lambda_k > 0$. Suppose $\log\det(\Sigma_Y) \geq k\tau$ and $\mathrm{tr}(\Sigma_Y) \leq m$ for some $m > 0$. Then*

$$\lambda_{\min}(\Sigma_Y) \;\geq\; \exp(k\tau)\,\frac{(k-1)^{k-1}}{m^{k-1}}. \tag{51}$$

In particular, the representation cannot collapse onto a subspace of dimension $< k$ in the projected sense.

Proof. See Appendix A.2.13. □

Joint spectral non-degeneracy (budget + log-det + PR). The next theorem formalizes the "LogDet + PR + budget" bridge: it rules out both needle collapse and the one-spike "L-shaped" failure mode.

Theorem 4.14 (Spectral non-degeneracy via joint constraints)**.** *Let $\Sigma \in \mathbb{R}^{k\times k}$ be SPD with eigenvalues $\lambda_1 \geq \cdots \geq \lambda_k > 0$. Assume the three constraints*

$$\mathrm{tr}(\Sigma) = c, \qquad \mathrm{PR}(\Sigma) \geq r_0, \qquad \frac{1}{k}\log\det(\Sigma) \geq \tau, \tag{52}$$

for some $c > 0$, $r_0 \in (1, k]$, and $\tau \in \mathbb{R}$. Then

$$\lambda_{\max}(\Sigma) \leq \frac{c}{\sqrt{r_0}}; \quad \lambda_{\min}(\Sigma) \geq \exp(k\tau)\left(\frac{\sqrt{r_0}}{c}\right)^{k-1}. \tag{53}$$

Consequently, Σ cannot realize the spike regime $\lambda_1 \gg \lambda_2, \ldots, \lambda_k$ and cannot approach rank collapse $\lambda_{\min} \to 0$ under the stated floors.

Proof. See Appendix A.2.14. □

Corollary 4.15 (Bounded conditioning under joint non-degeneracy constraints)**.** *Under the assumptions of Theorem 4.14, the condition number satisfies*

$$\kappa(\Sigma) := \frac{\lambda_{\max}(\Sigma)}{\lambda_{\min}(\Sigma)} \;\leq\; \exp(-k\tau)\left(\frac{c}{\sqrt{r_0}}\right)^{k}.$$

Proof. See Immediate by dividing the eigenvalue bounds in Theorem 4.14. □

Why symplecticity is still meaningful despite encoder regularizers. Symplecticity constrains the predictor map, while the budget/volume/PR constraints constrain the encoder outputs. The next proposition states the correct conditional claim: if the encoder occupies nontrivial volume locally, then the symplectic predictor cannot destroy it.

Proposition 4.16 (Symplectic predictors cannot induce collapse)**.** *Fix ϕ and consider the predictor map $\Phi_\phi^{(K)}$ from (36). If the encoder outputs occupy a set $\mathcal{U} \subset \mathbb{R}^{2d_0}$ with positive Lebesgue volume, then $\Phi_\phi^{(K)}(\mathcal{U})$ has the same volume as $\mathcal{U}$. In particular, $\Phi_\phi^{(K)}$ cannot map a positive-volume set to a set of zero volume; any global representational collapse must arise from E_θ, not from the symplectic predictor.*

Proof. See Appendix A.2.15. □

Takeaway. Theorems 4.2 and 4.4 ensure the predictor map is symplectic and volume-preserving (hence non-contracting in a precise geometric sense), with exact time reversibility (Corollary 4.5) and reciprocal singular-value pairing (Lemma 4.7). Since symplecticity alone does not prevent encoder collapse, HamJEPA uses a fixed-units scale budget plus projected log-det and PR floors; Theorem 4.14 formalizes why the *combination* rules out both rank collapse and one-spike pathologies without enforcing isotropic Gaussian marginals.

4.5. Algorithm

Algorithm 1 HamJEPA training step (one minibatch)

1: Sample $x_1, \ldots, x_B \sim \mathcal{D}$ and augmentations $\tau_a, \tau_b \sim \mathcal{A}$.
2: Form views $v_a = \tau_a(x)$, $v_b = \tau_b(x)$.
3: Encode $s_a = (q_a, p_a) = E_\theta(v_a)$ and $s_b = (q_b, p_b) = E_\theta(v_b)$.
4: Predict $\hat{s}_b = \Phi_\phi^{(K)}(s_a)$ using (35).
5: Compute $\mathcal{L}_{\text{pred}}$ (and optionally $\mathcal{L}_{\text{bi}}$) from (39)–(40).
6: Compute $\mathcal{L}_{\text{budget}}$ from (42).
7: Form Σ_Y from (43) and compute $\mathcal{L}_{\text{vol}}$, $\mathcal{L}_{\text{pr}}$ from (45), (47).
8: Optionally compute $\mathcal{L}_{\text{stab}}$ from (41).
9: Take a gradient step on (38).

5. Results

Experimental setup. We evaluate representation quality under the downstream protocols on CIFAR-100 and ImageNet-100. Unless stated otherwise, all methods share the same JEPA backbone, augmentations, optimizer, batch size, and training schedule; full details are deferred to Appendix E. We use two global views, and no local views; for CIFAR-100 we use batch size 512 and for ImageNet-100 we use batch size 256. *Design choice (shared across datasets).* To isolate the effect of the dynamical prior (rather than head architecture), we operate in a *headless token regime*: the encoder runs in token mode using ResNet stage-3 (`layer3`) features, and the projection head is fixed to the identity map for *all* methods. This preserves the coordinate semantics required by HamJEPA's phase-space interpretation and avoids conflating gains with heavy projection heads, but it also yields intentionally conservative linear separability compared to projector-based SSL recipes.

Representations and evaluation convention. Let $z = f_\theta(v) \in \mathbb{R}^d$ denote the embedding of a view v. All *downstream* evaluation results in this section use the learned representation directly (no post-hoc transform), unless explicitly stated. For HamJEPA, the encoder outputs $s = (q, p) \in \mathbb{R}^{2d_0}$ via a fixed channel-wise split of the token representation; we report results using q unless otherwise stated. Appendix C reports additional readouts (including p and (q, p)).

Methods compared. We report: (1) **SIGReg**: isotropic Gaussian regularization applied to standard JEPA embeddings under the same headless token encoder used throughout, and (2) **HamJEPA**: phase-space (q, p) with a symplectic predictor (leapfrog integration of a learned separable Hamiltonian) and an encoder-side anti-collapse bridge (scale budget + projected log-det floor + PR floor), applied per-coordinate to stabilize both q and p. For HamJEPA we additionally report evaluation using p and (q, p) to quantify how information is distributed across the phase-space coordinates. We use SIGReg as the primary isotropic-Gaussian baseline because it is the closest comparator to our theoretical framing; a VICReg-style first/second-order baseline under the same tokenized headless encoder is a natural additional control, but is not included in the present version.

Metrics. We report geometry-sensitive, nonparametric evaluation via frozen-feature kNN (cosine similarity) and linear probing as a secondary metric. Unless stated otherwise, kNN uses $k = 20$ and reports top-1 accuracy.

5.1. Main results on CIFAR-100

Table 1 reports CIFAR-100 results after $T_{\text{CIFAR}} = 30$ epochs, mean±std over 3 seeds. Full learning curves (per-epoch kNN and linear probe) and additional ablations are in Appendix C.

Method	kNN@20 (%)	Linear probe (%)
SIGReg	26.56 ± 0.18	30.43 ± 0.30
HamJEPA (q-eval)	31.45 ± 0.27	33.95 ± 0.24
HamJEPA (p-eval)	29.71 ± 0.25	33.07 ± 0.36
HamJEPA ((q, p)-eval)	30.88 ± 0.39	34.18 ± 0.17

Table 1. CIFAR-100 downstream performance after $T_{\text{CIFAR}} = 30$ epochs of pretraining. Mean±std over 3 seeds.

Discussion. HamJEPA improves over SIGReg across both nonparametric and linear evaluations after the same short pretraining budget. Using the primary q readout, HamJEPA gains $+4.89$ points in kNN@20 (31.45 vs. 26.56) and $+3.52$ points in linear probing (33.95 vs. 30.43). The momentum readout p is also predictive (29.71 kNN@20; 33.07 linear), but consistently behind q, supporting the intended design: q serves as the content coordinate optimized for view prediction and downstream use, while p acts as an auxiliary state enabling structured transitions. Concatenating (q, p) yields the strongest linear probe in Table 1, suggesting p carries complementary linearly-decodable signal even when q already dominates nearest-neighbor performance.

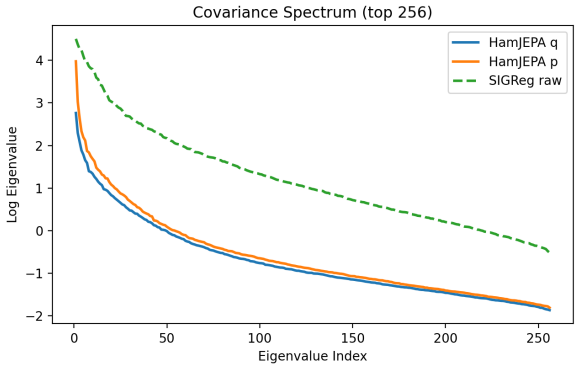


Figure 1. CIFAR-100 (representative run): covariance eigenspectra (top 256) for SIGReg (raw) and HamJEPA readouts (q and p). Spectra are shown on a log scale. HamJEPA-p is more top-heavy than HamJEPA-q, consistent with p serving as an auxiliary momentum coordinate.

5.2. Main results on ImageNet-100

Table 2 reports ImageNet-100 results after $T_{\text{IN}} = 45$ epochs of pretraining (single seed). Full learning curves (per-epoch kNN and linear probe) and additional ablations are in Appendix C.

Method	kNN@20 (%)	Linear probe (%)
SIGReg (q-eval)	20.10	24.40
SIGReg (p-eval)	18.78	23.86
SIGReg ((q, p)-eval)	20.94	25.54
HamJEPA (q-eval)	24.92	31.92
HamJEPA (p-eval)	24.72	32.08
HamJEPA ((q, p)-eval)	24.64	32.04

Table 2. ImageNet-100 downstream performance after $T_{\text{IN}} = 45$ epochs of pretraining (single seed). For HamJEPA, q and p correspond to the learned phase-space coordinates (content/position vs. auxiliary/momentum). For SIGReg, we report q/p using the same split interface (two 1024-d blocks whose concatenation yields a 2048-d feature); these should be interpreted as representation-block / size ablations rather than literal position–momentum coordinates.

Discussion. On ImageNet-100, HamJEPA improves over SIGReg across both nonparametric kNN and linear probing under the same pretraining budget ($T_{\text{IN}} = 45$ epochs, 1 seed). Using the primary q readout, HamJEPA gains $+4.82$ points in kNN@20 (24.92 vs. 20.10) and $+7.52$ points in linear probing (31.92 vs. 24.40). These gains are not explained by feature dimensionality: even when SIGReg is evaluated with the larger (q, p) representation (2048-d), HamJEPA-

q (1024-d) remains ahead by $+3.98$ points in kNN@20 (24.92 vs. 20.94) and $+6.38$ points in linear probing (31.92 vs. 25.54).

A notable configuration difference on ImageNet-100 is that we *match the full phase space* (q, p) during HamJEPA pretraining (rather than matching q alone as in CIFAR-100) because the q-only regime showed intermittent training-instability signals (notably spikes in the learned potential-variance statistic V_{var} and borderline $q_{\mathrm{pr}}/p_{\mathrm{pr}}$ behavior), while (q, p) remained better controlled and thus provided a fairer, more reliable head-to-head evaluation. Consistent with this, the auxiliary coordinate p is highly predictive and slightly exceeds q on the linear probe (32.08 vs. 31.92), indicating that the transition state retains linearly-decodable semantic information when it is directly constrained by the matching objective. However, concatenating (q, p) does not improve cosine kNN (24.64 vs. 24.92) and only marginally changes linear probing (32.04 vs. 31.92), suggesting that nearest-neighbor geometry is still primarily governed by the content coordinate, while any complementary signal in p is small (or redundant with q) at this training budget.

For SIGReg, the (q, p) concatenation *does* improve over either block alone (20.94/25.54 vs. 20.10/24.40 for q and 18.78/23.86 for p), which is consistent with p acting as a complementary feature block that is not strong in isolation but becomes useful when a learned readout can exploit the larger representation.

5.3. Is it just a larger representation?

A potential confound is representation dimension: improvements could arise simply from evaluating a higher-dimensional embedding. Here both methods admit a direct size/control comparison because we can evaluate q (1024-d), p (1024-d), or the concatenation (q, p) (2048-d).

On ImageNet-100, SIGReg benefits modestly from increasing dimension: moving from q to (q, p) improves by $+0.84$ kNN@20 points (20.10 $\rightarrow$ 20.94) and $+1.14$ linear-probe points (24.40 $\rightarrow$ 25.54). In contrast, HamJEPA shows essentially no gain from concatenation (24.92 $\rightarrow$ 24.64 in kNN@20; 31.92 $\rightarrow$ 32.04 in linear probing). Crucially, HamJEPA-q (1024-d) already outperforms SIGReg-(q, p) (2048-d) by large margins (24.92 vs. 20.94 in kNN@20; 31.92 vs. 25.54 in linear), indicating that HamJEPA's improvements are not a trivial consequence of evaluating a larger vector, but reflect the structure induced by the predictive dynamics and the matching objective. On CIFAR-100 (Table 1), q alone achieves the best kNN result, while (q, p) only modestly changes performance. This suggests the gain is not only a consequence of increasing ambient dimension, but is tied to the predictive structure and the way information is organized across phase space.

Both methods have the same wall-clock time ($t_{\mathrm{wall}}^{\mathrm{SIGREG}} = t_{\mathrm{wall}}^{\mathrm{HAMJEPA}}$) because they use the same backbone, data pipeline, batch size, optimizer/schedule, and number of training steps on the same hardware, with only an objective-level change whose compute is effectively negligible relative to the shared forward-backward pass.

5.4. Representation variants: q vs. p vs. (q, p)

Table 1 summarizes evaluation under different readouts of the phase-space state. On CIFAR-100, the ordering $q \gtrsim (q, p) \gg p$ for kNN (Table 1) matches the modeling intent: q is the supervised content coordinate, while p is optimized indirectly through its role in symplectic prediction.

Table 2 also provides an internal ablation over readouts. For HamJEPA, q is the primary content coordinate intended for view prediction and downstream use, while p is an auxiliary state enabling structured transitions. Under the ImageNet-100 setup where we match (q, p) during pretraining, p becomes nearly as strong as q for kNN and slightly stronger for linear probing (24.72 vs. 24.92 in kNN@20; 32.08 vs. 31.92 in linear), consistent with p being directly pressured to track semantically relevant state. The concatenation (q, p) yields comparable linear performance (32.04) but does not improve kNN (24.64), suggesting that any additional information carried by p is either redundant with q for nearest-neighbor geometry or requires a learned readout to exploit (and even then only weakly at this budget).

For SIGReg, the interpretation of q/p differs: SIGReg does not parameterize a Hamiltonian phase space, so our reported q and p should be viewed as two 1024-d feature blocks exposed by the same evaluation interface (with (q, p) giving the full 2048-d embedding). In this setting, q is the stronger block, p is weaker alone, and (q, p) is best, which is the pattern expected when one block carries the bulk of the semantic signal while the other contributes complementary but not independently robust features.

5.5. Geometry diagnostics implied by the theory

Section 3 motivates evaluating representation geometry beyond accuracy. Here we summarize scalar diagnostics that probe *non-collapse* and *spectral shape* without requiring isotropic Gaussianity.

Spectral diagnostics (effective rank and participation ratio). We compute covariance-spectrum diagnostics on $n = 5000$ samples using the top 256 eigenvalues of the feature covariance.

On CIFAR-100 for SIGReg, the reported effective rank over the top-256 spectrum is ≈ 109.7 with participation ratio ≈ 60.7. For HamJEPA-q, the effective rank increases to ≈ 124.5 while the participation ratio remains comparable

(≈ 60.5), indicating a non-degenerate, broadly-supported spectrum despite the absence of an isotropic Gaussian marginal constraint. In contrast, the HamJEPA-p readout is substantially more spectrally concentrated (effective rank ≈ 78.9, PR ≈ 23.8), consistent with p being auxiliary and only indirectly constrained through the predictor.

On ImageNet-100, SIGReg-q exhibits a relatively concentrated spectrum (effective rank ≈ 38.33; participation ratio ≈ 24.91). HamJEPA produces substantially broader spectra: HamJEPA-q has effective rank ≈ 94.73 and PR ≈ 45.27, while HamJEPA-p is similarly broad (effective rank ≈ 98.10; PR ≈ 46.88). Even SIGReg evaluated with the full (q, p) representation increases these quantities (effective rank ≈ 51.16; PR ≈ 36.17), but remains well below HamJEPA at the same pretraining budget.

These diagnostics align with the anti-collapse design in Section 4.4: the combination of a fixed-units budget, a projected log-det floor, and a PR floor is sufficient to prevent rank collapse of the content representation while still allowing anisotropy. Full spectra and additional diagnostics (including their evolution over training) are reported in Appendix C.

6. Conclusion

We studied how geometric assumptions in JEPA regularization affect representation quality. First, we provided a theory-first characterization of *canonical* embedding geometry under structured task families and a Hamiltonian energy budget. Second, motivated by the fact that this theory does not uniquely determine a stable training mechanism in practice, we introduced *HamJEPA*, which imposes Hamiltonian structure on the *predictive map* via a symplectic (volume-preserving) phase-space predictor and prevents encoder collapse with non-isotropic non-degeneracy constraints (scale budgets and spectral floors). On CIFAR-100 and ImageNet-100, HamJEPA improves over the SIGReg baseline under both kNN and linear probing, and its geometry diagnostics align with the intended separation between content (q) and momentum (p). We emphasize that the structural advantage of HamJEPA stems from the *geometry* of the objective, not the scale of the data.

References

Amari, S. Differential-geometrical methods in statistics, 1985.

Assran, M., Duval, Q., Misra, I., Bojanowski, P., Vincent, P., Rabbat, M., LeCun, Y., and Ballas, N. Self-supervised learning from images with a joint-embedding predictive architecture, 2023.

Balestriero, R. and LeCun, Y. Lejepa: Provable and scalable self-supervised learning without the heuristics, 2025. URL https://arxiv.org/abs/2511.08544.

Bardes, A., Ponce, J., and LeCun, Y. Vicreg: Variance-invariance-covariance regularization for self-supervised learning, 2021.

Boltzmann, L. Weitere studien über das wärmegleichgewicht unter gasmolekülen, 1872.

Chen, T., Kornblith, S., Norouzi, M., and Hinton, G. A simple framework for contrastive learning of visual representations, 2020.

Gibbs, J. W. Elementary principles in statistical mechanics: Developed with especial reference to the rational foundation of thermodynamics, 1902.

Greydanus, S., Dzamba, M., and Yosinski, J. Hamiltonian neural networks, 2019. URL https://arxiv.org/abs/1906.01563.

Grill, J.-B., Strub, F., Altché, F., Tallec, C., Richemond, P. H., Buchatskaya, E., Doersch, C., Pires, B. A., Guo, Z. D., Azar, M. G., Piot, B., Kavukcuoglu, K., Munos, R., and Valko, M. Bootstrap your own latent: A new approach to self-supervised learning, 2020.

Hamilton, W. R. On a general method in dynamics, 1834.

Jaynes, E. T. Information theory and statistical mechanics, 1957.

Laplace, P.-S. Traité de mécanique céleste, 1799.

Liouville, J. Sur la théorie de la variation des constantes arbitraires, 1838.

Poisson, S.-D. Sur la variation des constantes arbitraires dans les questions de mécanique, 1809.

Toth, P., Rezende, D. J., Jaegle, A., Racanière, S., Botev, A., and Higgins, I. Hamiltonian generative networks, 2019.

van den Oord, A., Li, Y., and Vinyals, O. Representation learning with contrastive predictive coding, 2018.

Wang, T. and Isola, P. Understanding contrastive representation learning through alignment and uniformity on the hypersphere, 2020.

Wu, Z., Zhang, J., Pai, D., Wang, X., Singh, C., Yang, J., Gao, J., and Ma, Y. Simplifying dino via coding rate regularization, 2025. URL https://arxiv.org/abs/2502.10385.

Yu, Y., Chan, K. H. R., You, C., Song, C., and Ma, Y. Learning diverse and discriminative representations via the principle of maximal coding rate reduction. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

Zbontar, J., Jing, L., Misra, I., LeCun, Y., and Deny, S. Barlow twins: Self-supervised learning via redundancy reduction, 2021.

Zhong, Y. D., Dey, B., and Chakraborty, A. Symplectic ode-net: Learning hamiltonian dynamics with control, 2019.

Zimmermann, E., Wiltzer, H., Szeto, J., Alvarez-Melis, D., and Mackey, L. Kerjepa: Kernel discrepancies for euclidean self-supervised learning, 2025. URL https://arxiv.org/abs/2512.19605.

A. Mathematical Proofs

A.1. Proofs for Section 3

A.1.1. Proof of Lemma 3.1

Let $\Sigma \succeq 0$ and $H \succ 0$. For $w \in \mathcal{W}_H$, define $u := H^{-1/2}w$. Then

$$w^\top H^{-1} w = \|H^{-1/2}w\|_2^2 = \|u\|_2^2 \le 1,$$

so $\mathcal{W}_H = \{H^{1/2}u : \ \|u\|_2 \le 1\}$. Substituting $w = H^{1/2}u$ into the objective yields

$$w^\top \Sigma w = u^\top \big(H^{1/2}\Sigma H^{1/2}\big)u.$$

Therefore

$$V(\Sigma) = \sup_{\|u\|_2 \le 1} u^\top \big(H^{1/2}\Sigma H^{1/2}\big)u = \lambda_{\max}\big(H^{1/2}\Sigma H^{1/2}\big),$$

where the last equality is the Rayleigh quotient characterization of the top eigenvalue. If $u_{\max}$ is a unit top eigenvector of $H^{1/2}\Sigma H^{1/2}$, then the corresponding $w_{\max} := H^{1/2}u_{\max}$ satisfies $w_{\max}^\top H^{-1} w_{\max} = 1$ and attains the supremum.

A.1.2. Proof of Theorem 3.2

Let $\Sigma \succeq 0$ and define

$$A := H^{1/2}\Sigma H^{1/2} \succeq 0.$$

Then $\operatorname{tr}(H\Sigma) = \operatorname{tr}(A)$, so the constraint $\operatorname{tr}(H\Sigma) = c$ becomes $\operatorname{tr}(A) = c$. By Lemma 3.1, the objective is $V(\Sigma) = \lambda_{\max}(A)$. Thus (16) is equivalent to

$$\min_{A \succeq 0:\ \operatorname{tr}(A)=c} \lambda_{\max}(A).$$

Let the eigenvalues of A be $\lambda_1 \ge \cdots \ge \lambda_d \ge 0$. Then $\sum_{i=1}^d \lambda_i = c$, hence $\lambda_1 \ge c/d$. Therefore, for every feasible A,

$$\lambda_{\max}(A) \ge \frac{c}{d}.$$

Equality $\lambda_{\max}(A) = c/d$ holds if and only if $\lambda_1 = \cdots = \lambda_d = c/d$, i.e., $A = (c/d)I$, which is unique. Mapping back,

$$\Sigma^\star = H^{-1/2}AH^{-1/2} = \frac{c}{d}H^{-1},$$

and the optimum value is c/d.

A.1.3. Proof of Theorem 3.3

Let Z have a density, $\mathbb{E}[Z] = 0$, and satisfy $\mathbb{E}[Z^\top HZ] = c$. Define the invertible linear change of variables

$$Y := H^{1/2}Z.$$

Then $\mathbb{E}[Y] = 0$ and

$$\mathbb{E}\|Y\|_2^2 = \mathbb{E}[Z^\top HZ] = c.$$

By the change-of-variables formula for differential entropy,

$$h(Z) = h(Y) + \log|\det(H^{-1/2})| = h(Y) - \tfrac{1}{2}\log\det(H).$$

Hence maximizing $h(Z)$ under (19) is equivalent to maximizing $h(Y)$ under $\mathbb{E}\|Y\|_2^2 = c$.

Let $\Sigma_Y := \operatorname{Cov}(Y) \succeq 0$. A standard entropy bound states that for any random vector with covariance Σ_Y,

$$h(Y) \le \frac{1}{2}\log\Big((2\pi e)^d \det(\Sigma_Y)\Big),$$

with equality if and only if Y is Gaussian with covariance Σ_Y.

Next, the constraint $\mathbb{E}\|Y\|_2^2 = c$ implies $\mathrm{tr}(\Sigma_Y) = c$ (since $\mathbb{E}[Y] = 0$). For any PSD matrix with eigenvalues $\nu_1, \ldots, \nu_d \geq 0$ and fixed sum $\sum_i \nu_i = c$, the arithmetic–geometric mean inequality gives

$$\det(\Sigma_Y) = \prod_{i=1}^{d} \nu_i \leq \Big(\frac{1}{d}\sum_{i=1}^{d} \nu_i\Big)^d = \Big(\frac{c}{d}\Big)^d,$$

with equality if and only if $\nu_1 = \cdots = \nu_d = c/d$, i.e., $\Sigma_Y = (c/d)I$.

Combining the two inequalities yields

$$h(Y) \leq \frac{1}{2}\log\Big((2\pi e)^d (c/d)^d\Big),$$

with equality if and only if $Y \sim \mathcal{N}(0, (c/d)I)$. Therefore the unique maximizer is $Y \sim \mathcal{N}(0, (c/d)I)$, and hence

$$Z = H^{-1/2}Y \sim \mathcal{N}\Big(0, \frac{c}{d}H^{-1}\Big),$$

as claimed.

A.1.4. Proof of Theorem 3.4

Under Euclidean isotropy $\Sigma = \sigma^2 I$, the budget $\mathrm{tr}(H\Sigma) = c$ forces

$$\sigma^2 \mathrm{tr}(H) = c \quad \Rightarrow \quad \sigma^2 = \frac{c}{\mathrm{tr}(H)},$$

so $\Sigma_{\text{iso}} = \frac{c}{\mathrm{tr}(H)} I$.

Using Lemma 3.1,

$$V(\Sigma_{\text{iso}}) = \lambda_{\max}\big(H^{1/2}\Sigma_{\text{iso}}H^{1/2}\big) = \lambda_{\max}\Big(\frac{c}{\mathrm{tr}(H)}H\Big) = \frac{c}{\mathrm{tr}(H)}\lambda_{\max}(H),$$

which is (22). From Theorem 3.2, $V(\Sigma^\star) = c/d$, so the ratio is

$$\frac{V(\Sigma_{\text{iso}})}{V(\Sigma^\star)} = \frac{\frac{c}{\mathrm{tr}(H)}\lambda_{\max}(H)}{c/d} = \frac{d\,\lambda_{\max}(H)}{\mathrm{tr}(H)} = \rho(H).$$

Since $\lambda_{\max}(H) \geq \mathrm{tr}(H)/d$ for any SPD H, we have $\rho(H) \geq 1$. Also $\lambda_{\max}(H) \leq \mathrm{tr}(H)$ implies $\rho(H) \leq d$. Finally, $\rho(H) = 1$ holds if and only if $\lambda_{\max}(H) = \mathrm{tr}(H)/d$, which occurs if and only if all eigenvalues of H are equal, i.e. $H \propto I$.

A.1.5. Proof of Proposition 3.5

Let $\mu := \mathbb{E}[Z]$ so that $\Sigma = \mathbb{E}[(Z-\mu)(Z-\mu)^\top]$. Define $Z' := H^{1/2}Z$ and note $\mathbb{E}[Z'] = H^{1/2}\mu$. Then

$$\begin{aligned}
\mathrm{Cov}(Z') &= \mathbb{E}\big[(Z' - \mathbb{E}[Z'])(Z' - \mathbb{E}[Z'])^\top\big] \\
&= \mathbb{E}\big[H^{1/2}(Z-\mu)(Z-\mu)^\top (H^{1/2})^\top\big] \\
&= H^{1/2}\,\mathbb{E}\big[(Z-\mu)(Z-\mu)^\top\big]\,(H^{1/2})^\top \\
&= H^{1/2}\Sigma H^{1/2},
\end{aligned}$$

where we used that $H^{1/2}$ is symmetric for SPD H.

For the equivalence: if $\Sigma = \alpha H^{-1}$ then

$$\mathrm{Cov}(Z') = H^{1/2}(\alpha H^{-1})H^{1/2} = \alpha I.$$

Conversely, if $\mathrm{Cov}(Z') = \alpha I$, multiply left and right by $H^{-1/2}$ to obtain

$$\Sigma = H^{-1/2}(\alpha I)H^{-1/2} = \alpha H^{-1}.$$

□

A.1.6. Proof of Proposition 3.6

By definition,

$$\mathcal{H}_H(q,p) = \tfrac{1}{2}q^\top Hq + \tfrac{1}{2}\|p\|_2^2,$$

so

$$\pi_H(q,p) = \frac{1}{Z_H}\exp\Big(-\frac{d}{2c}q^\top Hq\Big)\exp\Big(-\frac{d}{2c}\|p\|_2^2\Big).$$

Hence the density factorizes into a q-term and a p-term, so q and p are independent.

The partition function also factorizes:

$$Z_H = \Big(\int_{\mathbb{R}^d}\exp\big(-\tfrac{d}{2c}q^\top Hq\big)\,dq\Big)\Big(\int_{\mathbb{R}^d}\exp\big(-\tfrac{d}{2c}\|p\|_2^2\big)\,dp\Big).$$

Using the standard Gaussian integral

$$\int_{\mathbb{R}^d}\exp\big(-\tfrac{1}{2}x^\top Ax\big)\,dx = (2\pi)^{d/2}\det(A)^{-1/2} \qquad (A\succ 0),$$

with $A=(d/c)H$ for the q-integral and $A=(d/c)I$ for the p-integral, we obtain

$$Z_H = \Big(2\pi\frac{c}{d}\Big)^d\det(H)^{-1/2}.$$

Therefore

$$\pi_H(q,p) = \underbrace{\Big(2\pi\frac{c}{d}\Big)^{-d/2}\det(H)^{1/2}\exp\Big(-\frac{d}{2c}q^\top Hq\Big)}_{q\text{-marginal}}\cdot\underbrace{\Big(2\pi\frac{c}{d}\Big)^{-d/2}\exp\Big(-\frac{d}{2c}\|p\|_2^2\Big)}_{p\text{-marginal}}.$$

Thus q is Gaussian with precision matrix $(d/c)H$, hence covariance $(c/d)H^{-1}$, and p is Gaussian with precision $(d/c)I$, hence covariance $(c/d)I$:

$$q\sim\mathcal{N}\Big(0,\frac{c}{d}H^{-1}\Big), \qquad p\sim\mathcal{N}\Big(0,\frac{c}{d}I\Big).$$

Next,

$$\mathbb{E}_{\pi_H}[q^\top Hq] = \mathrm{tr}\Big(H\,\mathrm{Cov}(q)\Big) = \mathrm{tr}\Big(H\,\frac{c}{d}H^{-1}\Big) = \frac{c}{d}\,\mathrm{tr}(I) = c.$$

Similarly,

$$\mathbb{E}_{\pi_H}[\|p\|_2^2] = \mathrm{tr}\big(\mathrm{Cov}(p)\big) = \mathrm{tr}\Big(\frac{c}{d}I\Big) = c.$$

Hence

$$\mathbb{E}_{\pi_H}[\mathcal{H}_H(q,p)] = \frac{1}{2}\,\mathbb{E}_{\pi_H}[q^\top Hq] + \frac{1}{2}\,\mathbb{E}_{\pi_H}[\|p\|_2^2] = \frac{1}{2}c + \frac{1}{2}c = c.$$

Finally, the q-marginal is exactly the Gaussian

$$\mathcal{N}\Big(0,\frac{c}{d}H^{-1}\Big),$$

which is the minimax-optimal covariance from Theorem 3.2 and the maximum-entropy law from Theorem 3.3. □

A.1.7. Proof of Proposition 3.7

Let $\Sigma\succ 0$ be arbitrary and define

$$H := \frac{c}{d}\Sigma^{-1}.$$

Since $\Sigma^{-1}\succ 0$, we have $H\succ 0$. Then

$$\Sigma^\star(H) = \frac{c}{d}H^{-1} = \frac{c}{d}\Big(\frac{c}{d}\Sigma^{-1}\Big)^{-1} = \frac{c}{d}\cdot\frac{d}{c}\Sigma = \Sigma.$$

Thus every $\Sigma\succ 0$ arises as $\Sigma^\star(H)$ for some $H\succ 0$, so the family $\{\Sigma^\star(H) : H\succ 0\}$ is exactly the SPD cone. □

A.2. Proofs for Section 4.4

A.2.1. Proof of Lemma 4.1

By definition, $\mathbf{Z}' = \mathbf{Z}H^{1/2}$, so its ith row is

$$(\mathbf{Z}')_{i,:} \;=\; (\mathbf{Z}_{i,:})H^{1/2} \;=\; z_i^\top H^{1/2}.$$

Taking transpose gives

$$(\mathbf{Z}')_{i,:}^\top \;=\; (z_i^\top H^{1/2})^\top \;=\; (H^{1/2})^\top z_i \;=\; H^{1/2} z_i,$$

where we used symmetry of $H^{1/2}$ (valid for SPD H).

A.2.2. Proof of Theorem 4.2

Fix $s_0 \in \mathbb{R}^{2d_0}$ and let $s(t) = \Phi_{\phi,t}(s_0)$ solve $\dot{s} = J\nabla\mathcal{H}_\phi(s)$. Let $A(t) := D\Phi_{\phi,t}(s_0) \in \mathbb{R}^{2d_0\times 2d_0}$ denote the Jacobian of the flow with respect to the initial condition. Differentiating the ODE with respect to s_0 yields the variational equation

$$\dot{A}(t) \;=\; J\,\nabla^2\mathcal{H}_\phi(s(t))\,A(t), \qquad A(0) = I. \tag{54}$$

Define $M(t) := A(t)^\top JA(t)$. Using (54) and the facts $J^\top = -J$ and $\nabla^2\mathcal{H}_\phi$ is symmetric, we compute

$$\dot{M}(t) = \dot{A}(t)^\top JA(t) + A(t)^\top J\dot{A}(t) = A(t)^\top\nabla^2\mathcal{H}_\phi(s(t))^\top J^\top JA(t) + A(t)^\top JJ\nabla^2\mathcal{H}_\phi(s(t))A(t).$$

Since $\nabla^2\mathcal{H}_\phi(s(t))^\top = \nabla^2\mathcal{H}_\phi(s(t))$ and $J^\top = -J$ and $JJ = -I$, this becomes

$$\dot{M}(t) = A(t)^\top\nabla^2\mathcal{H}_\phi(s(t))\,(-J)JA(t) + A(t)^\top(-I)\nabla^2\mathcal{H}_\phi(s(t))A(t) = A(t)^\top\nabla^2\mathcal{H}_\phi(s(t))\,A(t) - A(t)^\top\nabla^2\mathcal{H}_\phi(s(t))\,A(t) = 0.$$

Thus $M(t)$ is constant in t. At $t = 0$, $A(0) = I$ so $M(0) = J$, hence $M(t) = J$ for all t, proving (48).

For the determinant claim, take determinants of $A(t)^\top JA(t) = J$:

$$\det(A(t))^2\det(J) = \det(J) \quad\Rightarrow\quad \det(A(t))^2 = 1.$$

Since $A(0) = I$ has $\det(A(0)) = 1$ and $\det(A(t))$ varies continuously with t, we must have $\det(A(t)) = 1$ for all t.

Finally, volume preservation follows from $\det(D\Phi_{\phi,t}(s)) \equiv 1$ by the change-of-variables theorem: for any measurable set U,

$$\mathrm{Vol}(\Phi_{\phi,t}(U)) = \int_U |\det(D\Phi_{\phi,t}(s))|\,ds = \int_U 1\,ds = \mathrm{Vol}(U).$$

A.2.3. Proof of Corollary 4.3

Let $Y = F(X)$ with F a C^1 diffeomorphism. The change-of-variables formula gives $p_Y(y) = p_X(x)\,|\det(DF(x))|^{-1}$ with $x = F^{-1}(y)$. Then

$$h(Y) = -\int p_Y(y)\log p_Y(y)\,dy = -\int p_X(x)\log\big(p_X(x)\,|\det(DF(x))|^{-1}\big)\,dx = h(X) + \mathbb{E}[\log|\det(DF(X))|].$$

If $|\det(DF(\cdot))| \equiv 1$, the second term is zero, so $h(Y) = h(X)$.

A.2.4. Proof of Theorem 4.4

Write the leapfrog step as a composition of two "kicks" and one "drift". Define for $h \in \mathbb{R}$:

$$K_h(q,p) := (q,\; p - h\nabla V_\phi(q)), \qquad D_h(q,p) := (q + hp,\; p),$$

where the drift uses $\nabla T(p) = p$ for $T(p) = \frac{1}{2}\|p\|^2$. Then the one-step leapfrog map satisfies

$$\Psi_{\phi,\Delta t} \;=\; K_{\Delta t/2}\circ D_{\Delta t}\circ K_{\Delta t/2}.$$

We show that both K_h and D_h are symplectic. Their Jacobians have block forms

$$DK_h(q,p) = \begin{pmatrix} I & 0 \\ -h\nabla^2 V_\phi(q) & I \end{pmatrix} = \begin{pmatrix} I & 0 \\ B & I \end{pmatrix}, \quad B^\top = B, \qquad DD_h(q,p) = \begin{pmatrix} I & hI \\ 0 & I \end{pmatrix} = \begin{pmatrix} I & C \\ 0 & I \end{pmatrix}, \quad C^\top = C.$$

A direct calculation shows that any matrix of the form $\left(\begin{smallmatrix} I & 0 \\ B & I \end{smallmatrix}\right)$ with B symmetric satisfies $A^\top J A = J$, and similarly for $\left(\begin{smallmatrix} I & C \\ 0 & I \end{smallmatrix}\right)$ with C symmetric. Therefore DK_h and DD_h are symplectic matrices at every point, so K_h and D_h are symplectic maps.

The composition of symplectic maps is symplectic, hence $\Psi_{\phi,\Delta t}$ is symplectic, and iterating yields symplecticity of $\Phi_\phi^{(K)} = \Psi_{\phi,\Delta t}^K$. The determinant claim follows as in the proof of Theorem 4.2.

A.2.5. Proof of Corollary 4.5

Using the decomposition $\Psi_{\Delta t} = K_{\Delta t/2} \circ D_{\Delta t} \circ K_{\Delta t/2}$, we have $K_h^{-1} = K_{-h}$ and $D_h^{-1} = D_{-h}$ by inspection. Hence

$$\Psi_{\Delta t}^{-1} = K_{\Delta t/2}^{-1} \circ D_{\Delta t}^{-1} \circ K_{\Delta t/2}^{-1} = K_{-\Delta t/2} \circ D_{-\Delta t} \circ K_{-\Delta t/2} = \Psi_{-\Delta t}.$$

Taking K-fold compositions gives $(\Psi_{\Delta t}^K)^{-1} = \Psi_{-\Delta t}^K$.

A.2.6. Proof of Proposition 4.6

Write the phase-space state as $s = (q,p) \in \mathbb{R}^{2d_0}$ and define the two vector fields

$$f(s) := \begin{pmatrix} p \\ -\nabla V(q) \end{pmatrix}, \qquad f_\phi(s) := \begin{pmatrix} p \\ -\nabla V_\phi(q) \end{pmatrix}.$$

Let Φ_t^V denote the flow of $\dot{s} = f(s)$ and $\Phi_t^{V_\phi}$ the flow of $\dot{s} = f_\phi(s)$.

Throughout, work on the compact set $\Omega = \Omega_q \times \Omega_p$ from the proposition statement. Since ∇V and ∇V_ϕ are Lipschitz on Ω_q and Ω_p is compact, both f and f_ϕ are Lipschitz on Ω; let L be any Lipschitz constant of f_ϕ on Ω:

$$\|f_\phi(x) - f_\phi(y)\|_2 \le L\|x-y\|_2, \qquad \forall x, y \in \Omega.$$

Define the uniform force mismatch on Ω_q:

$$\delta := \sup_{q \in \Omega_q} \|\nabla V_\phi(q) - \nabla V(q)\|_2.$$

Step 1: Continuous dependence of flows on the force field. Fix $s_0 \in K$ and let $s(t) := \Phi_t^V(s_0)$ and $\tilde{s}(t) := \Phi_t^{V_\phi}(s_0)$. By assumption, both trajectories remain in Ω for $t \in [0,T]$. Let $\Delta(t) := \tilde{s}(t) - s(t)$. Then

$$\begin{aligned} \dot{\Delta}(t) &= f_\phi(\tilde{s}(t)) - f(s(t)) \\ &= \big(f_\phi(\tilde{s}(t)) - f_\phi(s(t))\big) + \big(f_\phi(s(t)) - f(s(t))\big). \end{aligned}$$

Taking norms and using Lipschitzness of f_ϕ on Ω plus the definition of δ gives

$$\|\dot{\Delta}(t)\|_2 \le L\|\Delta(t)\|_2 + \delta.$$

Since $\Delta(0) = 0$, Grönwall's inequality yields, for all $t \in [0,T]$,

$$\|\Delta(t)\|_2 \le \begin{cases} \dfrac{\delta}{L}\big(e^{Lt} - 1\big), & L > 0, \\ \delta t, & L = 0. \end{cases}$$

In particular, at $t = T$ we obtain a uniform (over $s_0 \in K$) modeling-error bound

$$\sup_{s_0 \in K} \|\Phi_T^{V_\phi}(s_0) - \Phi_T^V(s_0)\|_2 \le C_1\,\delta, \qquad C_1 := \begin{cases} \frac{e^{LT}-1}{L}, & L > 0, \\ T, & L = 0. \end{cases}$$

Step 2: Leapfrog global error is $O(\Delta t^2)$. Fix ϕ and consider the ODE $\dot{s} = f_\phi(s)$. Let $\Psi_{\phi,h}$ denote one leapfrog step with step size h (Eq. (29)), and define the numerical iterates

$$s_{n+1} := \Psi_{\phi,h}(s_n), \qquad s_0 \in K, \qquad t_n := nh,$$

so that $s_K = (\Psi_{\phi,h})^K(s_0) = \Phi_\phi^{(K)}(s_0)$ with $Kh = T$.

(2a) Local truncation error bound. Because $V_\phi \in C^3$ on a neighborhood of Ω_q, the vector fields

$$A(q,p) := \begin{pmatrix} p \\ 0 \end{pmatrix}, \qquad B(q,p) := \begin{pmatrix} 0 \\ -\nabla V_\phi(q) \end{pmatrix}$$

are C^2 on Ω, and their exact flows are

$$\exp(hA) : (q,p) \mapsto (q + hp, p), \qquad \exp(hB) : (q,p) \mapsto (q, p - h\nabla V_\phi(q)).$$

The leapfrog step is exactly the Strang splitting:

$$\Psi_{\phi,h} = \exp\left(\frac{h}{2}B\right)\exp(hA)\exp\left(\frac{h}{2}B\right).$$

Standard Strang splitting theory (derivable via the BCH expansion for the Lie operators L_A, L_B) implies that the local truncation error is third order: there exists a constant $C_{\mathrm{loc}} = C_{\mathrm{loc}}(\Omega, V_\phi)$ such that

$$\sup_{x\in\Omega}\left\|\Psi_{\phi,h}(x) - \Phi_h^{V_\phi}(x)\right\|_2 \le C_{\mathrm{loc}}\, h^3 \quad \text{for all sufficiently small } h.$$

(Concretely, the BCH expansion gives $\exp(\frac{h}{2}L_B)\exp(hL_A)\exp(\frac{h}{2}L_B) = \exp\big(h(L_A + L_B) + O(h^3)\big)$ as operators on smooth test functions, and boundedness of the commutators on Ω yields the uniform C_{loc}.)

(2b) Error recursion and discrete Grönwall. Define the global error at step n by

$$e_n := s_n - \Phi_{t_n}^{V_\phi}(s_0).$$

Then

$$\begin{aligned} e_{n+1} &= \Psi_{\phi,h}(s_n) - \Phi_{t_{n+1}}^{V_\phi}(s_0) \\ &= \Psi_{\phi,h}(s_n) - \Phi_h^{V_\phi}\big(\Phi_{t_n}^{V_\phi}(s_0)\big). \end{aligned}$$

Add and subtract $\Phi_h^{V_\phi}(s_n)$:

$$e_{n+1} = \underbrace{\big(\Psi_{\phi,h}(s_n) - \Phi_h^{V_\phi}(s_n)\big)}_{\text{local truncation}} + \underbrace{\big(\Phi_h^{V_\phi}(s_n) - \Phi_h^{V_\phi}(\Phi_{t_n}^{V_\phi}(s_0))\big)}_{\text{stability}}.$$

Taking norms, using the local error bound from (2a), and using Lipschitz stability of the flow $\|\Phi_h^{V_\phi}(x) - \Phi_h^{V_\phi}(y)\|_2 \le e^{Lh}\|x - y\|_2$ on Ω yields

$$\|e_{n+1}\|_2 \le C_{\mathrm{loc}} h^3 + e^{Lh}\|e_n\|_2.$$

Iterating this recursion with $e_0 = 0$ gives

$$\|e_K\|_2 \le C_{\mathrm{loc}} h^3 \sum_{j=0}^{K-1} e^{Lhj} = C_{\mathrm{loc}} h^3 \cdot \begin{cases} \dfrac{e^{LKh} - 1}{e^{Lh} - 1}, & L > 0, \\ K, & L = 0. \end{cases}$$

Since $Kh = T$ and $e^{Lh} - 1 \ge Lh$ for $Lh \ge 0$, we obtain

$$\|e_K\|_2 \le \begin{cases} \dfrac{C_{\mathrm{loc}}}{L}\big(e^{LT} - 1\big)\, h^2, & L > 0, \\ C_{\mathrm{loc}} T\, h^2, & L = 0. \end{cases}$$

Taking the supremum over $s_0 \in K$ (all iterates stay in Ω by assumption), we conclude that there exists $C_2 = C_2(T, \Omega, V_\phi)$ such that

$$\sup_{s_0\in K}\left\|\Phi_\phi^{(K)}(s_0) - \Phi_T^{V_\phi}(s_0)\right\|_2 \le C_2\, h^2.$$

Step 3: Combine modeling and integrator errors. By the triangle inequality,

$$\|\Phi_\phi^{(K)}(s_0) - \Phi_T^V(s_0)\|_2 \le \|\Phi_\phi^{(K)}(s_0) - \Phi_T^{V_\phi}(s_0)\|_2 + \|\Phi_T^{V_\phi}(s_0) - \Phi_T^V(s_0)\|_2.$$

Taking suprema over $s_0 \in K$ and applying the bounds from Steps 1–2 yields

$$\sup_{s_0 \in K} \|\Phi_\phi^{(K)}(s_0) - \Phi_T^V(s_0)\|_2 \le C_2 h^2 + C_1 \delta.$$

This is the claimed inequality with $C := \max\{C_1, C_2\}$ (or $C := C_1 + C_2$).

Finally, if the model class can make δ arbitrarily small (uniform approximation of ∇V on Ω_q), choose ϕ so that $C_1\delta \le \varepsilon/2$, then choose $h = \Delta t$ so that $C_2 h^2 \le \varepsilon/2$, and conclude

$$\sup_{s_0 \in K} \|\Phi_\phi^{(K)}(s_0) - \Phi_T^V(s_0)\|_2 \le \varepsilon.$$

□

A.2.7. Proof of Lemma 4.7

Let A be symplectic: $A^\top J A = J$. Using $J^{-1} = -J$ and $J^\top = -J$, we obtain

$$A^{-1} = -JA^\top J.$$

Then

$$(A^\top A)^{-1} = A^{-1}A^{-T} = (-JA^\top J)(-JAJ) = J^{-1}(A^\top A)J,$$

so $(A^\top A)^{-1}$ is similar to $A^\top A$ and thus has the same eigenvalues. Therefore the eigenvalues of $A^\top A$ occur in reciprocal pairs $(\mu, 1/\mu)$, and the singular values $\sigma = \sqrt{\mu}$ occur in reciprocal pairs $(\sigma, 1/\sigma)$.

A.2.8. Proof sketch of Theorem 4.8

This is a standard result from backward error analysis for symplectic integrators; we provide the BCH-based sketch specialized to leapfrog.

Let L_T and L_V denote the Lie derivatives (Poisson bracket operators) corresponding to the Hamiltonians T and V, acting on smooth observables. The exact time-h flow of T (resp. V) is $\exp(hL_T)$ (resp. $\exp(hL_V)$). The leapfrog step can be written operator-theoretically as the symmetric composition

$$\exp\Big(\tfrac{h}{2}L_V\Big)\exp(hL_T)\exp\Big(\tfrac{h}{2}L_V\Big).$$

By the Baker–Campbell–Hausdorff (BCH) formula, this equals

$$\exp\Big(h(L_T + L_V) + h^3 C_3 + h^5 C_5 + \cdots\Big),$$

where $C_3, C_5, \ldots$ are nested commutators of L_T and L_V. For Hamiltonian systems, commutators of Hamiltonian vector fields correspond to Hamiltonian vector fields of Poisson brackets; equivalently, each C_{2j+1} is itself a Lie derivative $L_{H_{2j}}$ for some scalar function H_{2j} built from iterated Poisson brackets of T and V. Thus the exponential above is the exact time-h flow of a *modified Hamiltonian*

$$\widetilde{\mathcal{H}}_{\Delta t} \;=\; \mathcal{H} + h^2 H_2 + h^4 H_4 + \cdots$$

at the level of formal series. Truncating this series at order N yields a modified Hamiltonian $\widetilde{\mathcal{H}}_{\Delta t}^{[N]}$ whose flow matches one leapfrog step up to local error $\mathcal{O}(h^{N+1})$. Under analyticity, classical results show that the truncation can be chosen so that the resulting modified energy is nearly conserved over exponentially long times in $1/h$. (See, e.g., standard references on geometric numerical integration for full details.)

A.2.9. Proof of Lemma 4.9

Take $\Sigma = \mathrm{diag}(c, 0, \ldots, 0)$. Then $\Sigma \succeq 0$, $\mathrm{tr}(\Sigma) = c$, and $\mathrm{rank}(\Sigma) = 1$.

A.2.10. Proof of Lemma 4.10

Fix $k \geq 2$ and τ. For $\varepsilon > 0$, define eigenvalues

$$\lambda_2 = \cdots = \lambda_k = \varepsilon, \qquad \lambda_1 = \exp(k\tau)\,\varepsilon^{-(k-1)}.$$

Then $\prod_{i=1}^{k} \lambda_i = \exp(k\tau)$, hence $\frac{1}{k}\log\det(\Sigma) = \tau$. But as $\varepsilon \to 0$, $\lambda_1 \to \infty$ and $\lambda_2, \ldots, \lambda_k \to 0$. The participation ratio satisfies

$$\mathrm{PR}(\Sigma) = \frac{(\sum_i \lambda_i)^2}{\sum_i \lambda_i^2} \longrightarrow \frac{\lambda_1^2}{\lambda_1^2} = 1,$$

so $\mathrm{PR}(\Sigma)$ can be made arbitrarily close to 1 while maintaining the log-det constraint.

A.2.11. Proof of Lemma 4.11

Let λ_i be the eigenvalues of Σ. Then the eigenvalues of $a\Sigma$ are $a\lambda_i$, so

$$\mathrm{PR}(a\Sigma) = \frac{(\sum_i a\lambda_i)^2}{\sum_i (a\lambda_i)^2} = \frac{a^2(\sum_i \lambda_i)^2}{a^2 \sum_i \lambda_i^2} = \mathrm{PR}(\Sigma).$$

A.2.12. Proof of Lemma 4.12

Let $\Sigma_Q \succeq 0$ and $R \in \mathbb{R}^{d_0 \times k}$ have full column rank. Write $\Sigma_Q = A^\top A$ with $A := \Sigma_Q^{1/2}$ (PSD square root). Then

$$\Sigma_{Y,0} = R^\top \Sigma_Q R = R^\top A^\top A R = (AR)^\top (AR).$$

Therefore $\Sigma_{Y,0}$ is PSD and

$$\mathrm{rank}(\Sigma_{Y,0}) = \mathrm{rank}\left((AR)^\top (AR)\right) = \mathrm{rank}(AR) \leq \mathrm{rank}(A) = \mathrm{rank}(\Sigma_Q).$$

If $\mathrm{rank}(\Sigma_Q) < k$, then $\mathrm{rank}(\Sigma_{Y,0}) < k$, so $\Sigma_{Y,0}$ is singular. Hence $\det(\Sigma_{Y,0}) = 0$ and $\log\det(\Sigma_{Y,0}) = -\infty$.

For the ridge statement, let $r := \mathrm{rank}(\Sigma_{Y,0})$ and let $\lambda_1, \ldots, \lambda_r > 0$ be its nonzero eigenvalues. Then $\Sigma_{Y,0}$ has exactly $k - r$ zero eigenvalues. Adding εI_k shifts each eigenvalue by ε, so $\Sigma_Y = \Sigma_{Y,0} + \varepsilon I_k$ has eigenvalues

$$\lambda_1 + \varepsilon,\ \ldots,\ \lambda_r + \varepsilon,\ \underbrace{\varepsilon, \ldots, \varepsilon}_{k-r \text{ times}}.$$

Thus

$$\log\det(\Sigma_Y) = \sum_{i=1}^{r} \log(\lambda_i + \varepsilon) + (k - r)\log\varepsilon,$$

which is exactly the claimed formula. □

A.2.13. Proof of Proposition 4.13

Let $\lambda_1 \geq \cdots \geq \lambda_k > 0$ be eigenvalues of Σ_Y. From $\log\det(\Sigma_Y) \geq k\tau$ we have

$$\prod_{i=1}^{k} \lambda_i \ \geq\ \exp(k\tau).$$

Let $\lambda_{\min} = \lambda_k$. Using $\sum_{i=1}^{k-1} \lambda_i \leq \mathrm{tr}(\Sigma_Y) - \lambda_k \leq m$, the AM–GM inequality yields

$$\prod_{i=1}^{k-1} \lambda_i \ \leq\ \Big(\frac{\sum_{i=1}^{k-1} \lambda_i}{k-1}\Big)^{k-1} \ \leq\ \Big(\frac{m}{k-1}\Big)^{k-1}.$$

Therefore

$$\exp(k\tau) \ \leq\ \prod_{i=1}^{k} \lambda_i = \lambda_k \prod_{i=1}^{k-1} \lambda_i \ \leq\ \lambda_k \Big(\frac{m}{k-1}\Big)^{k-1},$$

so

$$\lambda_{\min}(\Sigma_Y) = \lambda_k \ \geq\ \exp(k\tau)\,\frac{(k-1)^{k-1}}{m^{k-1}}.$$

A.2.14. Proof of Theorem 4.14

Let $\lambda_1 \geq \cdots \geq \lambda_k > 0$ be eigenvalues of Σ and $c = \sum_i \lambda_i$. The PR constraint implies

$$\mathrm{PR}(\Sigma) = \frac{(\sum_i \lambda_i)^2}{\sum_i \lambda_i^2} \geq r_0 \quad \Rightarrow \quad \sum_{i=1}^{k} \lambda_i^2 \leq \frac{c^2}{r_0}.$$

Since $\lambda_{\max}^2 = \lambda_1^2 \leq \sum_i \lambda_i^2$, we get

$$\lambda_{\max}(\Sigma) \leq \frac{c}{\sqrt{r_0}}.$$

Next, the log-det constraint implies

$$\prod_{i=1}^{k} \lambda_i = \det(\Sigma) \geq \exp(k\tau).$$

Using $\lambda_i \leq \lambda_{\max}$ for $i = 1, \ldots, k-1$ gives

$$\exp(k\tau) \leq \lambda_{\min} \lambda_{\max}^{k-1}, \quad \text{so} \quad \lambda_{\min} \geq \frac{\exp(k\tau)}{\lambda_{\max}^{k-1}} \geq \exp(k\tau) \left(\frac{\sqrt{r_0}}{c} \right)^{k-1}.$$

A.2.15. Proof of Proposition 4.16

By Theorem 4.4, $\Phi_\phi^{(K)}$ is a C^1 diffeomorphism with $|\det(D\Phi_\phi^{(K)}(s))| = 1$ for all s. For any measurable $\mathcal{U}$ with finite volume, the change-of-variables theorem gives

$$\mathrm{Vol}\big(\Phi_\phi^{(K)}(\mathcal{U})\big) = \int_{\mathcal{U}} \left| \det(D\Phi_\phi^{(K)}(s)) \right| \, ds = \int_{\mathcal{U}} 1 \, ds = \mathrm{Vol}(\mathcal{U}).$$

Hence $\mathrm{Vol}(\mathcal{U}) > 0$ implies $\mathrm{Vol}(\Phi_\phi^{(K)}(\mathcal{U})) > 0$. Therefore the predictor cannot map a positive-volume set to a zero-volume set; any collapse to a lower-volume region must already be present in the encoder outputs.

B. Supplementary theory: maximum entropy on phase space and symplectic factorization

Theorem B.1 (Maximum entropy on phase space under an expected energy constraint)**.** *Let $\mathcal{H} : \mathbb{R}^m \to \mathbb{R}$ be measurable and assume there exists $\beta > 0$ such that the partition function*

$$Z(\beta) := \int_{\mathbb{R}^m} e^{-\beta \mathcal{H}(s)} \, ds$$

is finite. Fix $c \in \mathbb{R}$ and consider the set of densities

$$\mathcal{P}_c := \left\{ p \text{ density on } \mathbb{R}^m : \int p(s)\, ds = 1, \ \int \mathcal{H}(s) p(s)\, ds = c, \ h(p) > -\infty \right\},$$

where $h(p) := -\int p \log p$ is differential entropy. If there exists $\beta^\star > 0$ such that $\mathbb{E}_{p_{\beta^\star}}[\mathcal{H}(S)] = c$ for

$$p_\beta(s) = \frac{1}{Z(\beta)} e^{-\beta \mathcal{H}(s)},$$

then the unique maximizer of $h(p)$ over $\mathcal{P}_c$ is $p^\star = p_{\beta^\star}$.

Proof. Fix $\beta > 0$ with $Z(\beta) < \infty$ and define $p_\beta(s) = Z(\beta)^{-1} e^{-\beta \mathcal{H}(s)}$. For any $p \in \mathcal{P}_c$,

$$\mathrm{KL}(p \| p_\beta) = \int p(s) \log \frac{p(s)}{p_\beta(s)} \, ds \geq 0,$$

with equality iff $p = p_\beta$ a.e. Expanding $\log p_\beta(s) = -\beta\mathcal{H}(s) - \log Z(\beta)$ gives

$$\begin{aligned}
\mathrm{KL}(p\|p_\beta) &= \int p \log p \, ds - \int p \log p_\beta \, ds \\
&= -h(p) - \int p(s)\big(-\beta\mathcal{H}(s) - \log Z(\beta)\big)\, ds \\
&= -h(p) + \beta \underbrace{\int \mathcal{H}(s)p(s)\, ds}_{=c} + \log Z(\beta) \underbrace{\int p(s)\, ds}_{=1} \\
&= -h(p) + \beta c + \log Z(\beta).
\end{aligned}$$

Thus $h(p) \le \beta c + \log Z(\beta)$ for all $p \in \mathcal{P}_c$, with equality iff $p = p_\beta$.

If there exists $\beta^\star$ such that $p_{\beta^\star} \in \mathcal{P}_c$ (i.e., it matches the energy constraint), then $p_{\beta^\star}$ attains the upper bound and is therefore a maximizer. Uniqueness follows from the equality condition for KL divergence. □

Corollary B.2 (Energy-based densities are invariant under Hamiltonian flow)**.** *Let $\mathcal{H} \in C^2(\mathbb{R}^{2d_0})$ and let Φ_t be the Hamiltonian flow of $\dot{s} = J\nabla\mathcal{H}(s)$, assumed to be a C^1 diffeomorphism for the times of interest. If S has density $p(s) = g(\mathcal{H}(s))$ for some measurable $g : \mathbb{R} \to \mathbb{R}_+$, then for all t, $\Phi_t(S) \overset{d}{=} S$. In particular, the Gibbs density $p(s) \propto e^{-\beta\mathcal{H}(s)}$ is invariant.*

Proof. Energy is conserved along Hamiltonian trajectories: if $s(t) = \Phi_t(s_0)$, then

$$\frac{d}{dt}\mathcal{H}(s(t)) = \nabla\mathcal{H}(s(t))^\top \dot{s}(t) = \nabla\mathcal{H}(s(t))^\top J\nabla\mathcal{H}(s(t)) = 0,$$

since $J^\top = -J$ implies $u^\top Ju = 0$ for all u. Thus $\mathcal{H}(\Phi_t(s)) = \mathcal{H}(s)$.

Also Hamiltonian flows are volume-preserving (Liouville): $|\det(D\Phi_t(s))| = 1$ for all s, t. Let $Y = \Phi_t(S)$. By change of variables,

$$p_Y(y) = p(\Phi_{-t}(y)) \cdot \left|\det\big(D\Phi_{-t}(y)\big)\right| = g(\mathcal{H}(\Phi_{-t}(y))) \cdot 1 = g(\mathcal{H}(y)) = p(y).$$

Hence Y and S have the same density. □

Proposition B.3 (Generic kick–drift–scaling factorization of linear symplectic maps)**.** *Let $A = \left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right) \in \mathrm{Sp}(2d_0)$ and assume the block $d \in \mathbb{R}^{d_0 \times d_0}$ is invertible. Define*

$$B := bd^{-1}, \qquad C := d^{-1}c.$$

Then B and C are symmetric and

$$A = \underbrace{\begin{pmatrix} I & B \\ 0 & I \end{pmatrix}}_{\text{upper shear (drift)}} \underbrace{\begin{pmatrix} d^{-\top} & 0 \\ 0 & d \end{pmatrix}}_{\text{symplectic scaling}} \underbrace{\begin{pmatrix} I & 0 \\ C & I \end{pmatrix}}_{\text{lower shear (kick)}}.$$

Proof. Since $A \in \mathrm{Sp}(2d_0)$, it satisfies $A^\top JA = J$, which implies the standard block identities $b^\top d = d^\top b$ and $a^\top d - c^\top b = I$. From $b^\top d = d^\top b$ and invertibility of d,

$$B^\top = (bd^{-1})^\top = d^{-\top}b^\top = bd^{-1} = B,$$

so B is symmetric.

Similarly, $AJA^\top = J$ also holds for symplectic A, and its block identities include $cd^\top = dc^\top$. Then

$$C^\top = (d^{-1}c)^\top = c^\top d^{-\top} = d^{-1}c = C,$$

so C is symmetric.

Now multiply the three proposed factors:

$$\begin{pmatrix} I & B \\ 0 & I \end{pmatrix} \begin{pmatrix} d^{-\top} & 0 \\ 0 & d \end{pmatrix} \begin{pmatrix} I & 0 \\ C & I \end{pmatrix} = \begin{pmatrix} d^{-\top} + BdC & Bd \\ dC & d \end{pmatrix}.$$

The bottom-right block matches d. The top-right block matches b because $Bd = (bd^{-1})d = b$. The bottom-left block matches c because $dC = d(d^{-1}c) = c$.

For the top-left block, use the symplectic identity $ad^\top - bc^\top = I$ (from $AJA^\top = J$) and right-multiply by $d^{-\top}$:

$$a - bc^\top d^{-\top} = d^{-\top}.$$

Using $c^\top d^{-\top} = (d^{-1}c)^\top = C^\top = C$, this becomes

$$a = d^{-\top} + bC = d^{-\top} + (Bd)C = d^{-\top} + BdC,$$

which matches the computed top-left block. Hence the factorization holds. □

C. Further results and figures

We refer to the method uniformly as *HamJEPA*; in the code/config files, the suffix `hjepa_mv` simply denotes the minimal multi-view implementation used in our experiments.

C.1. Frozen-feature diagnostic summaries

Figures 2 (CIFAR-100) and 4 (ImageNet-100) summarize the geometry and non-parametric usefulness of *frozen* encoder representations for the SIGReg baseline and HamJEPA latent states. Each method panel contains four diagnostics computed on *raw* encoder features (i.e., no whitening or other post-processing):

1. **kNN sweep:** top-1 accuracy as a function of neighborhood size k.
2. **Covariance eigenspectrum:** top-256 eigenvalues (log-scale) of the *mean-centered* empirical covariance.
3. **Random-pair cosine similarities:** histogram of cosine similarities after per-vector ℓ_2 normalization *without* mean subtraction.
4. **Feature norms:** histogram of ℓ_2 feature norms.

A key subtlety is that the covariance spectrum is computed after *mean-centering*, whereas the cosine histogram is computed on normalized features *without* centering. Consequently, strongly positive random-pair cosines can arise from a dominant *mean direction* even when the mean-centered covariance remains high-rank. We exploit this complementarity in the discussions below.

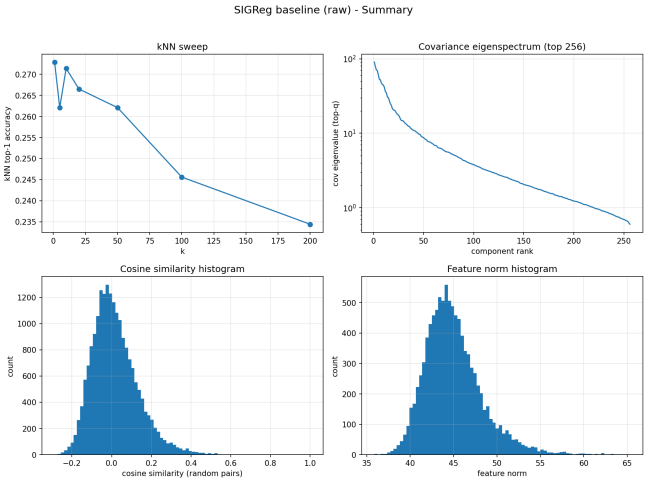
C.2. Discussion CIFAR-100

C.2.1. 30 EPOCH RESULTS

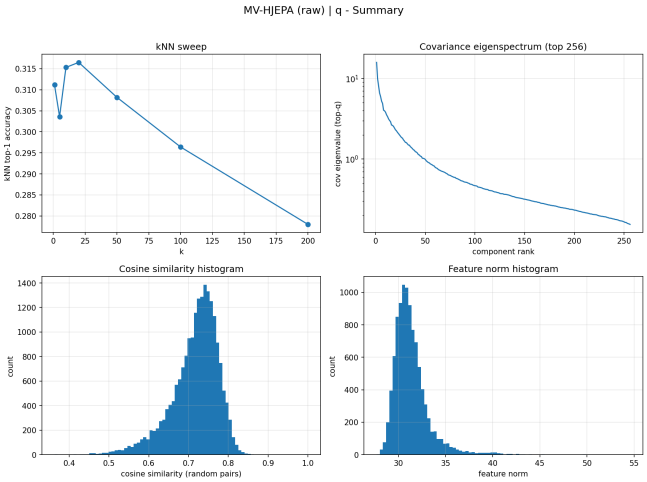
These results are for the 30 epoch pretraining.

(1) Neighborhood quality: kNN sweeps. In Fig. 2a, the SIGReg baseline peaks at small k and degrades as k grows, consistent with semantics being locally present but becoming less class-pure as neighborhoods expand. Quantitatively, SIGReg peaks at $27.46 \pm 0.31\%$ (at k=1) and falls to $23.10 \pm 0.32\%$ (at k=200).

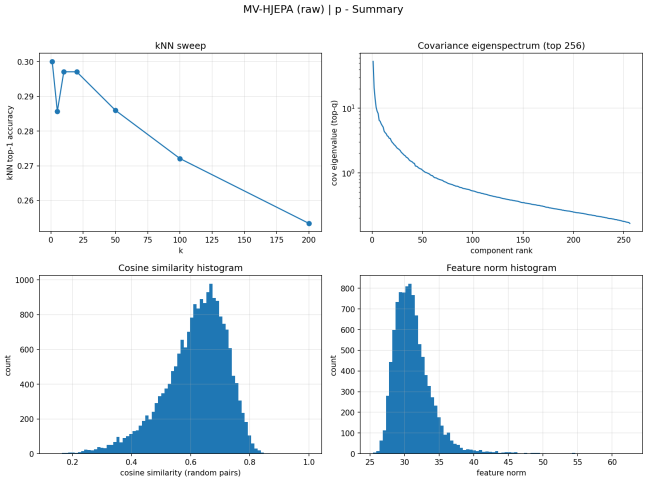
HamJEPA improves the entire sweep for all latent variants in Fig. 2. Notably, the *single* q state (1024-D) outperforms the SIGReg baseline (2048-D) across the plotted k values: $31.45 \pm 0.27\%$ at k=20 for q, versus $26.56 \pm 0.18\%$ for SIGReg at k=20 (a $+4.88 \pm 0.11$ point gain). The p and (q, p) variants achieve $29.71 \pm 0.25\%$ and $30.88 \pm 0.39\%$ at k=20, respectively. That (q, p) does not exceed q suggests that, under the default weighting, p contributes less additional *metric-consistent* neighborhood structure than q, even though it remains informative for linear decoding (see below).



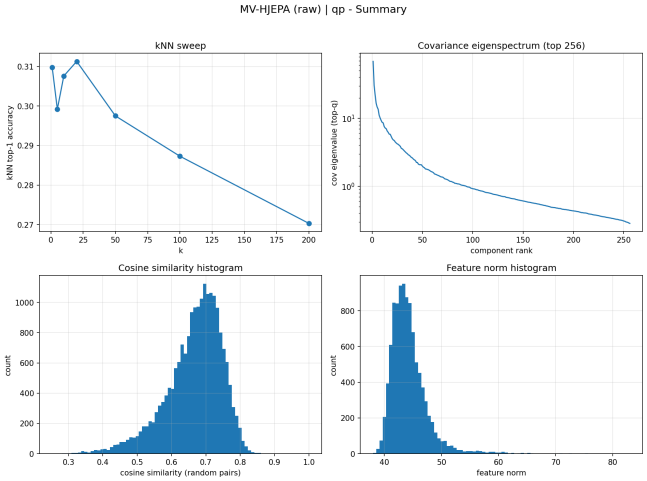
(a) SIGReg (raw).



(b) HamJEPA q (raw).



(c) HamJEPA p (raw).



(d) HamJEPA (q, p) (raw).

Figure 2. **Frozen-feature diagnostic summaries (raw) 30 epochs.** Each panel contains: (top-left) kNN sweep; (top-right) covariance eigenspectrum (top 256, log scale); (bottom-left) cosine similarity histogram (random pairs); (bottom-right) feature norm histogram.

(2) Linear probe vs. kNN: separability without neighborhood purity. While kNN distinguishes the latent variants, the linear probe largely does not: SIGReg reaches $30.43 \pm 0.30\%$ top-1, whereas HamJEPA reaches $33.95 \pm 0.24\%$, $33.07 \pm 0.36\%$ and $34.18 \pm 0.17\%$ top-1 for q, p, and (q, p) respectively. A useful interpretation is that p is *linearly* discriminative but induces less clean neighborhoods under the raw metric, so its gains show up more clearly in the probe than in kNN.

(3) Covariance eigenspectrum: rank proxies and collapse checks. The top-right panels show mean-centered covariance spectra (top 256 eigenvalues, log-scale). A convenient summary is the effective-rank proxy computed from these eigenvalues: SIGReg has effective rank $\approx 109.73 \pm 0.61$ (top-256), whereas HamJEPA-q increases to 125.58 ± 3.33. In contrast, HamJEPA-p is more concentrated (effective rank 79.46 ± 3.99), and (q, p) lies in between (93.10 ± 4.00). Two points follow: (i) HamJEPA-q does *not* achieve its downstream gain via low-rank collapse (the centered covariance is high-rank), and (ii) the q/p split is geometrically meaningful: q and p allocate variance differently even when their linear probe scores are similar.

(4) Cosine similarities: diagnosing mean-dominance. The bottom-left panels reveal the largest qualitative difference. SIGReg's random-pair cosines are centered near zero (mean 0.01699 ± 0.00089) with substantial negative mass, which is consistent with its isotropy-encouraging design.

HamJEPA's cosines shift sharply positive: mean cosine is 0.71561 ± 0.00129 for q, 0.61687 ± 0.00409 for p, and 0.66613 ± 0.00270 for (q, p). Crucially, this strong alignment coexists with a *high-rank mean-centered covariance* (especially for q). This combination is most naturally explained by a large *non-zero mean feature direction* that dominates dot products when features are not explicitly centered: the covariance spectrum removes that mean direction by construction, while the cosine histogram does not. In other words, the CIFAR-100 HamJEPA embeddings look "cone-like" in raw normalized space primarily due to mean-dominance rather than variance collapse.

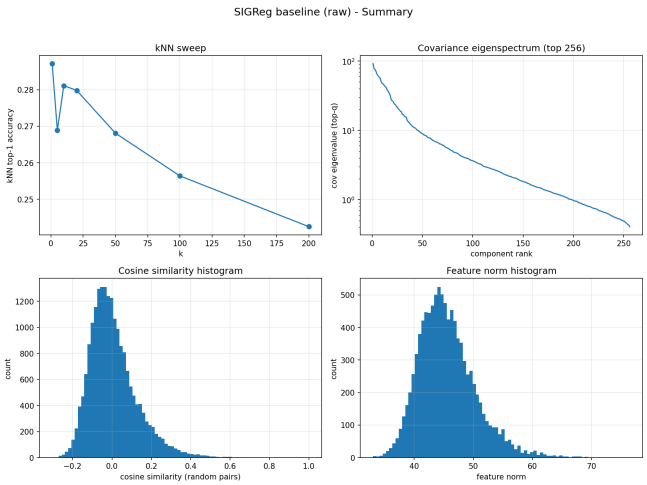
(5) Feature norms: scale control and the (q, p) energy split. SIGReg feature norms are concentrated around 44.891 ± 0.090, whereas HamJEPA's q and p have smaller typical norms around 31.358 ± 0.017 and 31.281 ± 0.029. The concatenated (q, p) norm shifts back to 44.303 ± 0.027, closely matching the Pythagorean combination $\|[q; p]\|_2 \approx \sqrt{\|q\|_2^2 + \|p\|_2^2}$, which is what one expects if q and p maintain comparable energy rather than one collapsing to near-zero. This provides a simple consistency check that the phase-space parameterization retains a balanced contribution from both coordinates.

C.2.2. 80 EPOCH RESULTS

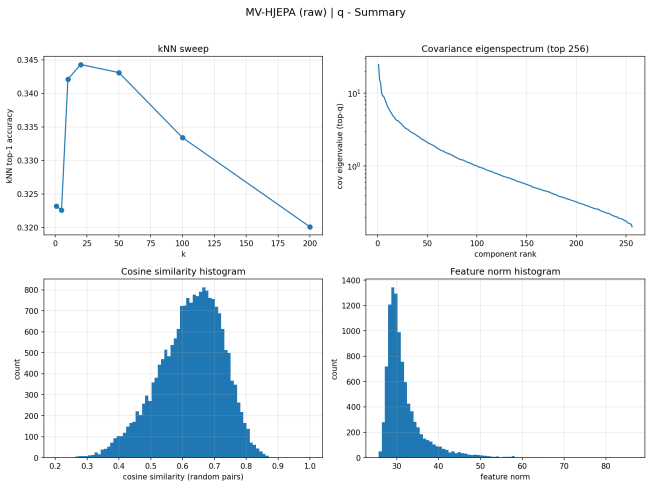
By design, the CIFAR-100 pretraining curves tend to plateau because the experiment is set up as a minimal, controlled training regime, whose goal is to isolate representational geometry effects rather than to chase absolute accuracy with an aggressively optimized recipe. CIFAR-100 is small and low-resolution, so the encoder typically learns the dominant semantic invariances early; after that point, additional epochs mainly yield diminishing returns (small refinements in neighborhood structure or feature scaling) that often do not translate into large gains in linear-probe/(k)NN top-1 under a fixed evaluation protocol. In other words, once both methods reach a "good-enough" feature space, the remaining headroom is mostly governed by recipe choices we intentionally avoid here (stronger augmentation, longer schedules, larger models, distillation/EMA, more hyperparameter search), since those would confound the attribution: we want any observed differences to come primarily from the geometry induced by the training dynamics, not from extra optimization budget.

Method	Variant	Linear probe (%)	kNN@20 (%)	kNN best (%)
SIGReg	single	33.95	27.98	28.71 @ k=1
MLP-HJEPA (non-symplectic)	q	42.26	28.10	28.10 @ k=20
MLP-HJEPA (non-symplectic)	p	41.90	28.30	28.30 @ k=20
MLP-HJEPA (non-symplectic)	(q, p)	42.77	28.15	28.15 @ k=20
HamJEPA (symplectic)	q	44.59	34.43	34.43 @ k=20
HamJEPA (symplectic)	p	44.44	32.96	32.96 @ k=20
HamJEPA (symplectic)	(q, p)	44.52	33.66	33.66 @ k=20

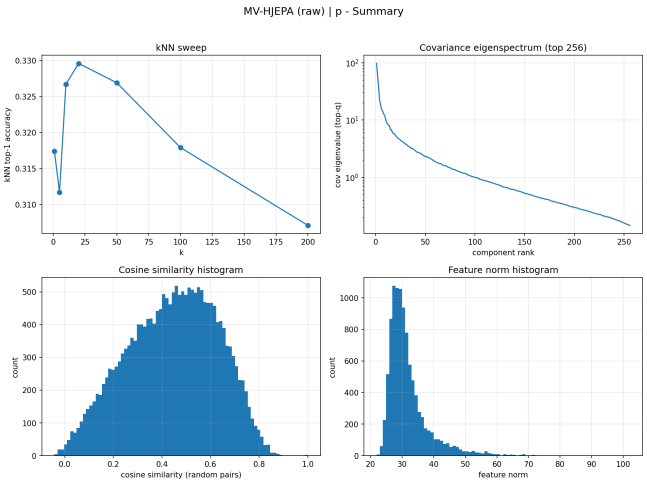
Table 3. **CIFAR-100, 80-epoch predictor-family ablation.** MLP-HJEPA keeps the phase-space encoder, channel-wise q/p split, identity projector, prediction loss, anti-collapse regularizers, optimizer, batch size, and 80-epoch schedule fixed, and replaces only the symplectic leapfrog predictor by a non-symplectic residual MLP predictor.



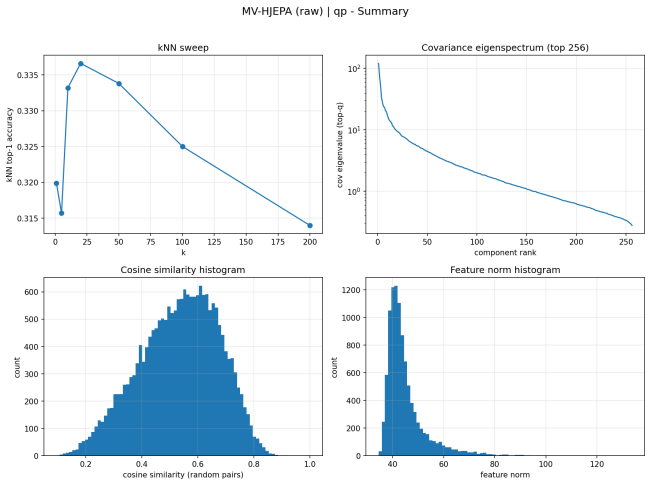
(a) SIGReg (raw).



(b) HamJEPA q (raw).



(c) HamJEPA p (raw).



(d) HamJEPA (q, p) (raw).

Figure 3. **CIFAR-100 frozen-feature diagnostic summaries (raw) 80 epochs.** Each panel contains: (top-left) kNN sweep; (top-right) mean-centered covariance eigenspectrum (top 256, log scale); (bottom-left) random-pair cosine similarity histogram; (bottom-right) feature norm histogram.

Predictor-family ablation: what changes and what stays fixed. In addition to SIGReg and symplectic HamJEPA, we evaluate a non-symplectic predictor ablation, *MLP-HJEPA*, that keeps the phase-space encoder, channel-wise q/p split, identity projector, prediction loss, anti-collapse regularizers, optimizer, batch size, and training schedule fixed, and replaces only the leapfrog Hamiltonian rollout by a generic residual MLP rollout predictor. Because Fig. 3 visualizes only SIGReg and symplectic HamJEPA, we summarize the MLP ablation numerically in Table 3. This is therefore a clean apples-to-apples test of whether the gains come mainly from the phase-space representation and encoder-side regularization, or from the symplectic predictor itself.

(1) Neighborhood quality: kNN sweeps and local semantic structure. At 80 epochs, SIGReg reaches 27.98% at kNN@20 and peaks at 28.71% over the full sweep (at k=1). It peaks at small k and degrades as k grows, consistent with semantics being locally present but becoming less class-pure as neighborhoods expand. The non-symplectic MLP-HJEPA ablation barely changes this picture: its best kNN@20 is only 28.30% (from the p readout), a gain of just $+0.32$ points over SIGReg, and its best-over-k score is in fact slightly *below* SIGReg (28.30% vs. 28.71%). By contrast, symplectic HamJEPA is qualitatively different: the q readout reaches 34.43% at kNN@20 and also attains the best-over-k value of 34.43% (at k=20), namely gains of $+6.45$ points at kNN@20 and $+5.72$ points best-over-k relative to SIGReg. Notably, the *single* q state (1024-D) outperforms the SIGReg baseline (2048-D) across the plotted k values. Best-vs.-best, the symplectic predictor outperforms the MLP predictor by $+6.13$ points at kNN@20. This is the sharpest evidence that the symplectic rollout is doing something specifically geometric: the phase-space encoder alone does *not* automatically produce high-purity local neighborhoods.

(2) Linear probe: phase-space structure already helps, but symplecticity still matters. The linear-probe comparison is more nuanced. MLP-HJEPA already substantially outperforms SIGReg, reaching 42.77% top-1 in its best variant ((q, p)) versus 33.95% for SIGReg, a gain of $+8.82$ points. Symplectic HamJEPA pushes this further to 44.59% (best q readout), i.e. another $+1.82$ points over MLP-HJEPA and $+10.64$ over SIGReg. This suggests that a large part of the linear-separability gain comes from the *phase-space parameterization plus encoder-side anti-collapse bridge*, not only from symplecticity. However, once nonparametric neighborhood quality is examined, the predictor family becomes decisive: the MLP predictor narrows much of the probe gap, but essentially none of the kNN gap.

(3) q/p decomposition: structured phase space versus generic feature blocks. Within MLP-HJEPA, the three readouts are almost indistinguishable: linear probe is 42.26%/41.90%/42.77% for q/p/(q, p), and kNN@20 is 28.10%/28.30%/28.15%. In other words, once the symplectic predictor is removed, the q/p split behaves much more like a generic two-block partition of the feature vector than like a structured position-momentum state space. This contrasts with symplectic HamJEPA, where q is the cleanest neighborhood readout, p is slightly weaker, and (q, p) remains close but does not dominate. The natural interpretation is that the Hamiltonian rollout induces a genuine functional differentiation between the two coordinates, whereas the MLP rollout does not.

(4) Mean-centered covariance geometry: high-rank symplectic q versus concentrated MLP states. The scalar covariance diagnostics sharpen this distinction. SIGReg has effective rank ≈ 99.80 and participation ratio ≈ 58.32 on the top-256 eigenspectrum. Symplectic HamJEPA-q is substantially broader still, with effective rank ≈ 128.98 and participation ratio ≈ 70.34, confirming that its downstream gains are *not* coming from collapse onto a narrow subspace. By contrast, MLP-HJEPA is much more spectrally concentrated: its effective rank is only ≈ 42.06 for q, ≈ 43.95 for p, and ≈ 42.85 for (q, p), with participation ratios only ≈ 23.65–24.99. So although the MLP ablation yields good linear-probe accuracy, it does so with a much more concentrated mean-centered covariance geometry than symplectic HamJEPA. Put differently: the MLP predictor can produce a linearly usable representation, but it does not organize a broad, high-rank content geometry in the way the symplectic predictor does.

(5) Cosine similarities and feature norms: the predictor changes geometry, not just scale. The random-pair cosine statistics again place MLP-HJEPA much closer to SIGReg than to symplectic HamJEPA. SIGReg has cosine mean ≈ 0.006 with standard deviation ≈ 0.121, while MLP-HJEPA remains near zero as well (cosine means ≈ 0.042 for q, ≈ 0.053 for p, and ≈ 0.047 for (q, p), all with standard deviation around 0.19). Symplectic HamJEPA, in contrast, exhibits strongly positive cosine means (≈ 0.625 for q, ≈ 0.443 for p, and ≈ 0.534 for (q, p)), reflecting the markedly more aligned raw geometry visible in Fig. 3. Importantly, this difference is *not* explained by trivial scale effects: the feature norms of MLP-HJEPA are nearly the same as those of symplectic HamJEPA (means ≈ 32.21, 31.89, and 45.32 for MLP-HJEPA versus ≈ 31.81,

31.88, and 45.06 for symplectic HamJEPA). Thus the predictor family is primarily changing *how* variance is organized and aligned, not simply how large the features are.

Takeaway (CIFAR-100, 80 epochs). The MLP ablation is informative in exactly the right way. It shows that the phase-space encoder, q/p split, and non-isotropic anti-collapse regularization already explain a large fraction of the linear-probe improvement over SIGReg. However, they do *not* explain the major improvement in nonparametric neighborhood quality. That gain appears to be specifically tied to the symplectic predictor: relative to the non-symplectic MLP rollout, symplectic HamJEPA produces much better kNN structure, a broader and less concentrated mean-centered covariance for the content coordinate, and a more distinctive raw embedding geometry. In this sense, the symplectic component is not merely a mild architectural preference; it is the ingredient that turns a linearly useful phase-space representation into one with substantially cleaner local semantic geometry.

C.3. Discussion ImageNet-100

(1) Frozen-feature quality: kNN and probe. On ImageNet-100, HamJEPA again improves non-parametric and linear evaluation. In Fig. 4, the dimension-matched comparison (2048-D) is SIGReg (q, p) versus HamJEPA (q, p). At the final checkpoint, SIGReg (q, p) reaches a best kNN top-1 of 21.7% (peak near k=50), whereas HamJEPA (q, p) reaches 25.9% (a $+4.2$ point gain). For linear probing, SIGReg (q, p) reaches 25.5% top-1, whereas HamJEPA (q, p) reaches 32.0% (a $+6.5$ point gain).

Figure 6 additionally provides a strict 1024-D comparison on q: at epoch 45, HamJEPA-q improves linear probe by $+7.52$ points and best kNN by $+4.90$ points over SIGReg-q (as annotated).

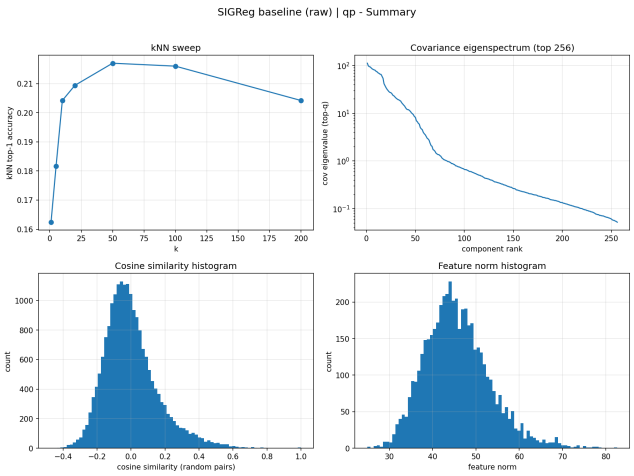
(2) Covariance rank: HamJEPA spreads variance across many directions. The covariance spectra (top-right) and the effective-rank proxy show a stark contrast: SIGReg-q and SIGReg-p have low effective rank ($\approx$ 38–39 on the top-256 eigenvalues), whereas HamJEPA-q and HamJEPA-p are much higher ($\approx$ 95–98). Thus, on ImageNet-100, HamJEPA's gains are associated with *more* variance being spread across principal directions (higher rank), not less.

(3) Cosine similarities: mild global alignment without extreme cone structure. Unlike CIFAR-100, ImageNet-100 does *not* show an extreme cone effect for HamJEPA. SIGReg's random-pair cosines remain centered near zero (mean ≈ 0), while HamJEPA shifts to moderately positive cosines (mean $\approx$ 0.27–0.28 depending on $q/p/(q, p)$). Given the simultaneously high effective rank of the mean-centered covariance, this again suggests a shared direction and/or non-zero mean, but the effect is substantially weaker than on CIFAR-100.

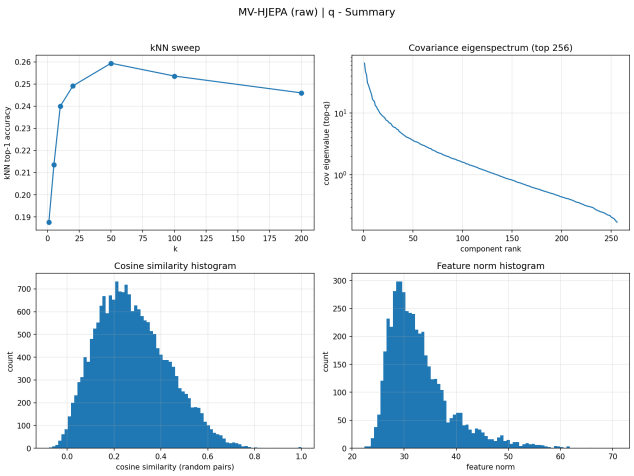
(4) q/p decomposition: complementarity for SIGReg, redundancy for HamJEPA. Figure 5 makes the structural difference explicit. For SIGReg+tokens, (q, p) is consistently best throughout training, indicating that q and p halves are not individually sufficient but become competitive when concatenated. For HamJEPA, q and p are individually as strong as (q, p) (both for probe and kNN), suggesting that both coordinates carry similar semantic content and that concatenation adds little.

(5) Dynamics: accuracy tracks geometry for HamJEPA, not for SIGReg. In Fig. 6, HamJEPA's effective rank increases over training while probe accuracy improves, yielding a strong positive association in the probe-versus-rank plot (annotated ρ). SIGReg shows the opposite: effective rank decreases while accuracy rises slightly, yielding a negative association. This supports the interpretation that HamJEPA's ImageNet gains co-occur with a progressively less concentrated (mean-centered) covariance structure.

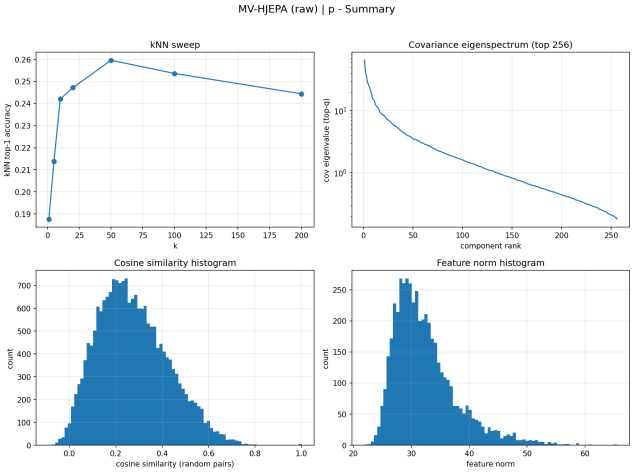
Takeaway (ImageNet-100). On ImageNet-100, HamJEPA improves kNN and linear probing while producing substantially higher-rank mean-centered covariance than SIGReg and only moderately more positive cosine alignment. In contrast to SIGReg+tokens, HamJEPA makes both q and p individually strong, with little benefit from concatenation.



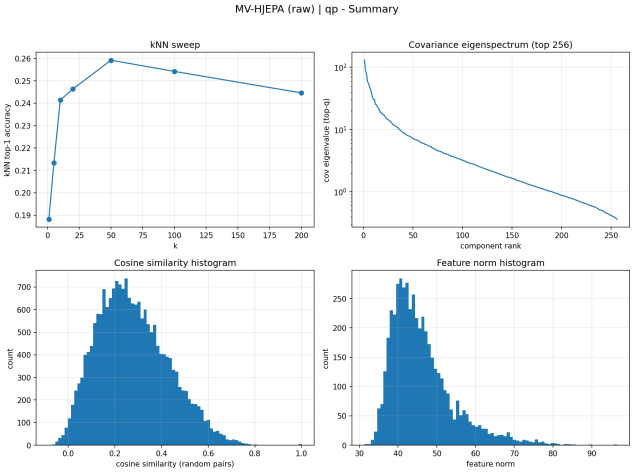
(a) SIGReg (raw).



(b) HamJEPA q (raw).

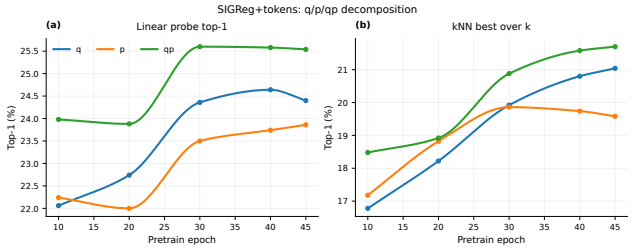


(c) HamJEPA p (raw).

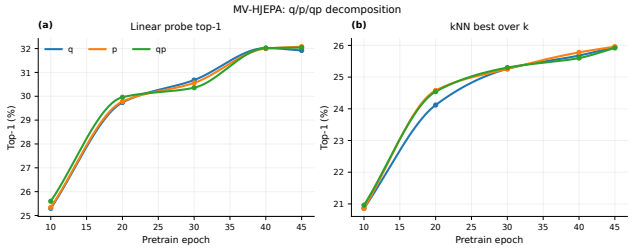


(d) HamJEPA (q, p) (raw).

Figure 4. **ImageNet-100 frozen-feature diagnostic summaries (raw).** Each panel contains: (top-left) kNN sweep; (top-right) covariance eigenspectrum (top 256, log scale); (bottom-left) cosine similarity histogram (random pairs); (bottom-right) feature norm histogram.



(a) SIGReg (tokens).



(b) HamJEPA.

Figure 5. q/p **decomposition across pretraining (ImageNet-100).** We evaluate frozen features extracted at sparse pretraining checkpoints using (left) a linear probe top-1 and (right) the best kNN top-1 across the evaluated k values. For SIGReg+tokens, concatenating (q, p) is consistently better than either half alone, indicating that the split halves are complementary (and/or that doubling feature dimension helps). For HamJEPA, q, p, and (q, p) track each other closely, suggesting that both phase-space coordinates are individually informative and largely redundant under the symplectic objective. (Left) SIGReg tokens. (Right) HamJEPA.

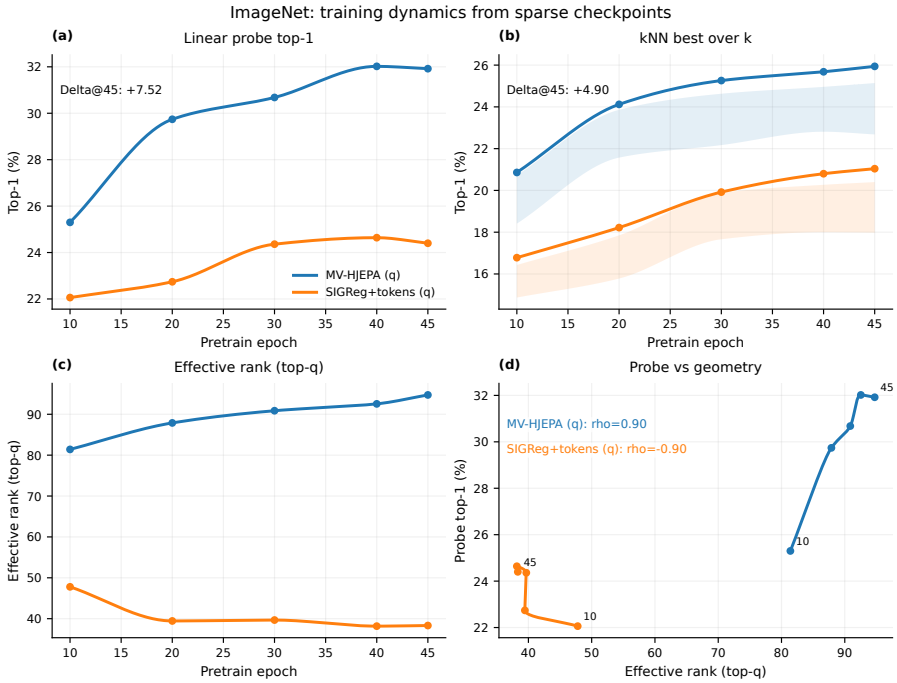


Figure 6. **ImageNet-100 training dynamics from sparse checkpoints (geometry vs. accuracy).** Using the q representation at a set of pretraining epochs, we plot: (a) linear probe top-1; (b) best kNN top-1 over k (shaded region indicates the spread over the k sweep); (c) effective-rank proxy computed from the top-256 eigenvalues of the mean-centered q covariance; (d) probe accuracy versus effective rank (points annotated by epoch), with the plotted correlation coefficient ρ. HamJEPA improves steadily and shows increasing effective rank, while SIGReg+tokens improves modestly while its effective rank decreases; the deltas at epoch 45 are annotated in panels (a,b).

C.4. Further figures

Figures 7–9 provide an intuition for a LeJEPA-inspired *sliced-projection* diagnostic: rather than comparing rollout distributions only in aggregate, we compare many 1D projections (a Cramér–Wold viewpoint) to expose direction-dependent geometric distortion.

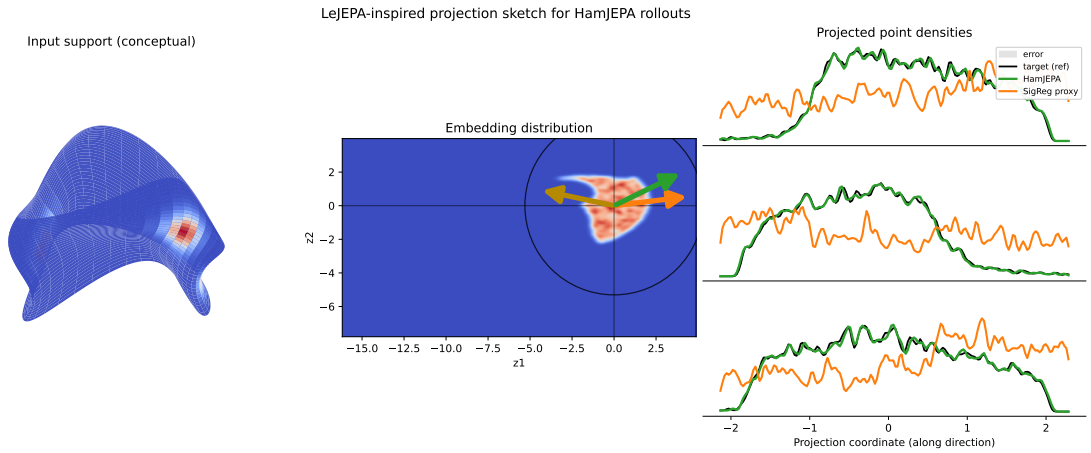


Figure 7. **LeJEPA-inspired sliced-projection diagnostic for HamJEPA rollouts (cartoon).** Left: conceptual support of the input distribution (manifold-structured data). Middle: an example embedding distribution in a 2D slice with several projection directions $u(\theta)$. Right: estimated 1D projected densities along representative directions; black denotes the reference/target, green denotes HamJEPA rollouts, orange denotes a SIGReg proxy baseline, and the shaded band visualizes the per-direction projection error. Matching many 1D projections (Cramér–Wold/sliced view) makes geometric distortion from non-structure-preserving dynamics visible, while symplectic rollouts remain close to the reference across directions.

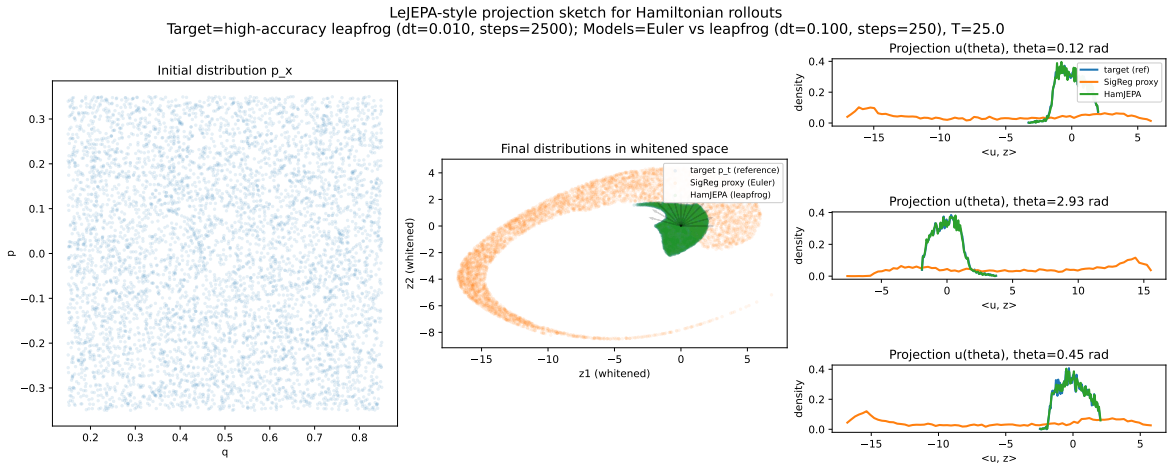


Figure 8. **LeJEPA-style projection sketch on a toy Hamiltonian transport problem.** Left: initial phase-space distribution p_x over (q, p). Middle: final distributions in whitened coordinates after rolling out to a fixed horizon T; the reference target is generated by a high-accuracy leapfrog integrator, while the two model curves compare a coarse Euler rollout (SIGReg proxy) against a coarse leapfrog rollout (HamJEPA). Right: 1D densities of projections $\langle u(\theta), z\rangle$ for representative angles θ, showing that leapfrog-based rollouts track the reference distribution across directions, whereas Euler produces large direction-dependent mismatch.

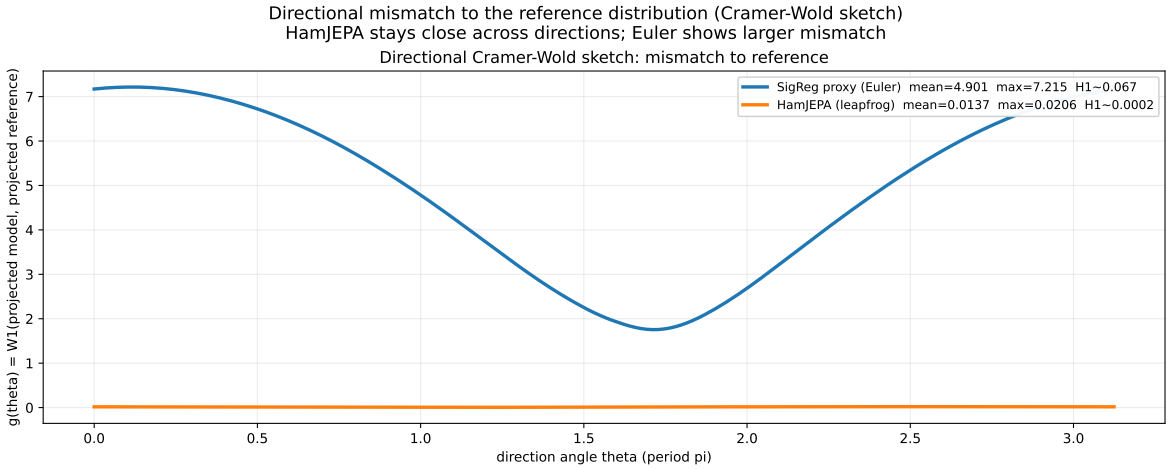


Figure 9. **Directional Cramér–Wold discrepancy (sliced mismatch) versus projection angle.** For each direction angle θ (period π), we compute a 1D mismatch $g(\theta) = W_1(\langle u(\theta), Z_{\text{model}}\rangle, \langle u(\theta), Z_{\text{ref}}\rangle)$ between the projected model and reference distributions. HamJEPA (leapfrog) stays uniformly close across directions, while the Euler/SIGReg proxy exhibits large, anisotropic deviation; the legend reports summary statistics over θ (mean/max and the aggregate score used in the plot).

D. A visual guide to HamJEPA

This appendix gives a diagrammatic explanation of HamJEPA for readers who want a more visual account than the equation-level presentation in the main text. The figures here are explanatory schematics rather than empirical results. Their purpose is to separate the method into four conceptual layers: (i) the overall two-view prediction pipeline, (ii) how the encoder constructs the phase-space state $s = [q; p]$, (iii) how the predictor evolves that state by a symplectic Hamiltonian rollout, and (iv) why encoder-side anti-collapse regularization is still required even when the predictor itself is symplectic.

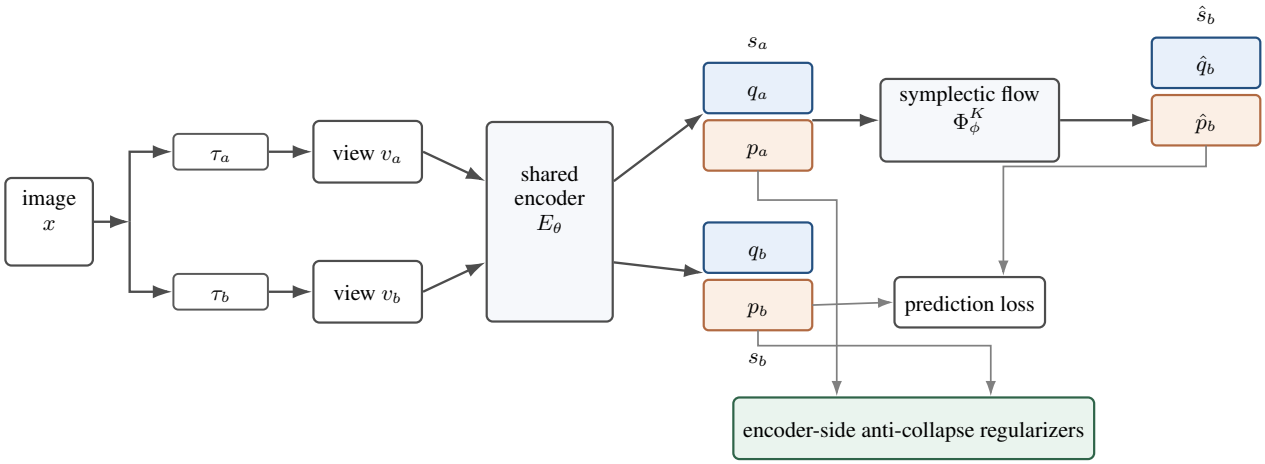


Figure 10. **HamJEPA at the highest level.** Two augmented views of the same image are encoded by a shared encoder into phase-space states $s_a = [q_a; p_a]$ and $s_b = [q_b; p_b]$. The predictor is not an unconstrained regression head: it is a structured Hamiltonian rollout Φ_ϕ^K that evolves s_a into a predicted target state $\hat{s}_b$. Training combines a prediction loss between $\hat{s}_b$ and s_b with separate encoder-side anti-collapse regularizers applied across the batch of states.

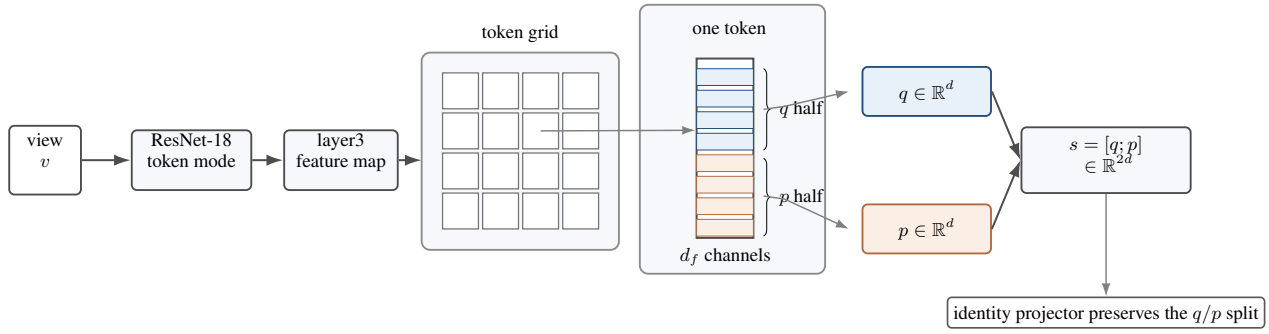


Figure 11. **How the encoder constructs the phase-space state.** HamJEPA is not just taking an arbitrary vector and renaming its halves as q and p. In the implementation, the encoder runs in token mode: an intermediate ResNet feature map is converted into a token grid, the *channel stack of each token* is split into a q half and a p half, and those halves are flattened across the spatial grid to form $q \in \mathbb{R}^d$ and $p \in \mathbb{R}^d$. These are then concatenated into $s = [q; p] \in \mathbb{R}^{2d}$. This is why HamJEPA uses an identity projector: a learned MLP after the encoder would remix the coordinates and destroy the intended q/p semantics.

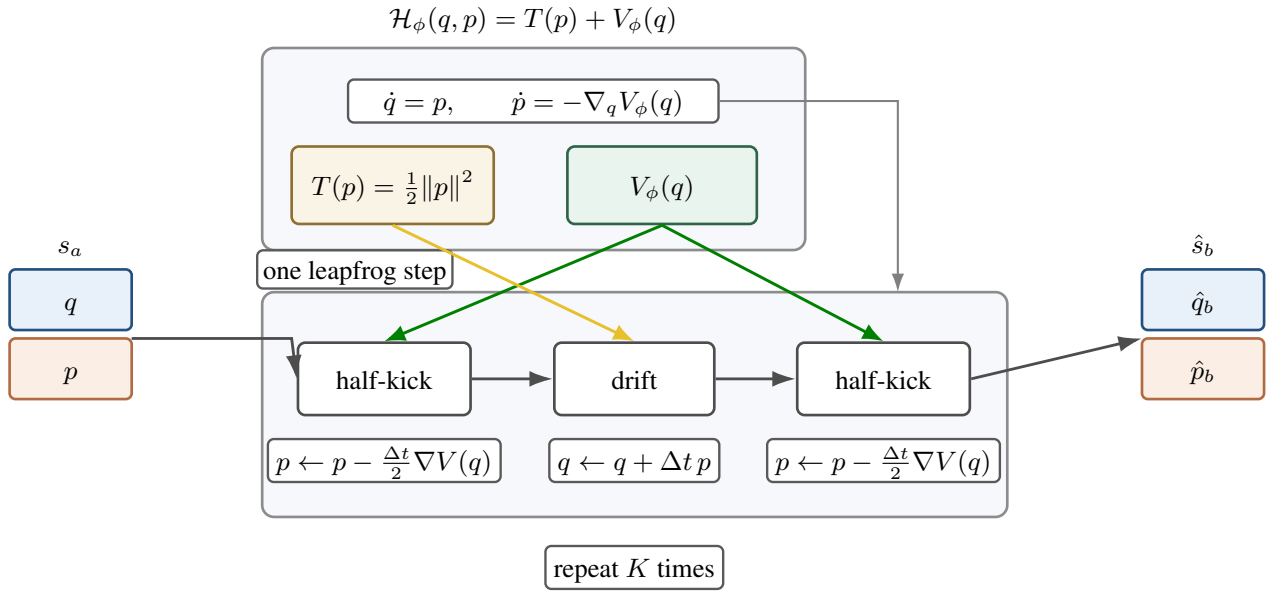


Figure 12. **What the predictor is doing.** HamJEPA does not use an arbitrary predictor $z_a \mapsto \hat{z}_b$. Instead, it rolls the state forward with a learned separable Hamiltonian $\mathcal{H}_\phi(q,p) = T(p) + V_\phi(q)$ and a leapfrog integrator. The potential $V_\phi(q)$ drives the kick updates through $\nabla_q V_\phi(q)$, while the kinetic term $T(p) = \frac{1}{2}\|p\|^2$ yields the drift update $q \leftarrow q + \Delta t\, p$. Repeating this structured step K times produces the predicted target state $\hat{s}_b = \Phi_\phi^K(s_a)$.

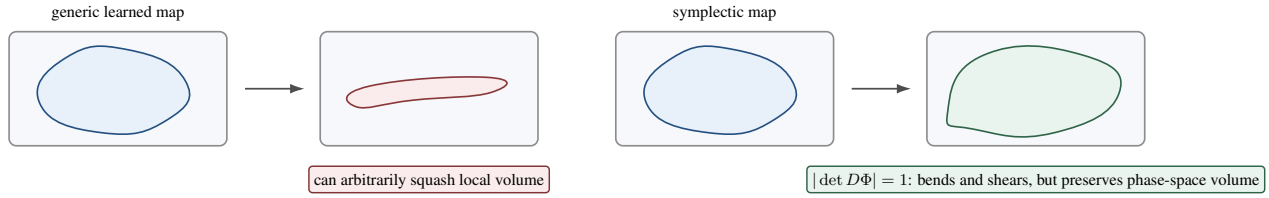


Figure 13. **What "symplectic" means geometrically.** A generic learned map can bend, stretch, and arbitrarily collapse local geometry. A symplectic map can still deform the state space, but it preserves total phase-space volume locally, i.e. $|\det D\Phi| = 1$. This gives the predictor a structured notion of reversibility and ensures that the predictor itself is not the source of volume collapse.

Symplecticity constrains the predictor, but it does *not* by itself prevent encoder collapse. A collapsed encoder could still map many different views to almost the same latent state, and a perfectly symplectic predictor would then only move around an already-degenerate state cloud. The next two figures therefore separate these two roles: first, why predictor structure alone is insufficient; second, how the encoder-side regularizers keep the latent space alive.

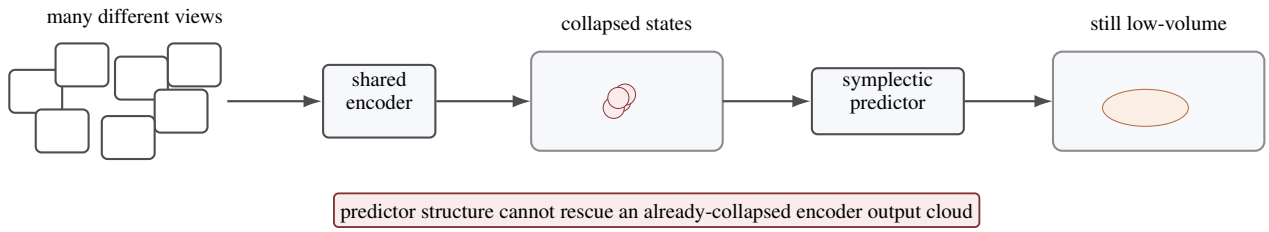


Figure 14. **Why the predictor alone is not enough.** Even if the predictor is perfectly symplectic, the encoder could still map many different views to nearly the same latent state. In that situation the rollout acts on an already-collapsed representation. This is why HamJEPA needs a second component beyond the predictor: encoder-side anti-collapse regularization.

Why the predictor alone is not enough. The symplectic constraint belongs to the predictor map, not to the encoder. This means it prevents the rollout from crushing phase-space volume, but it does *not* stop the encoder itself from mapping many different views to nearly the same latent state. A collapsed encoder would therefore still give a degenerate state cloud, even with a perfectly symplectic predictor. The next figures isolate the role of each encoder-side anti-collapse regularizer.

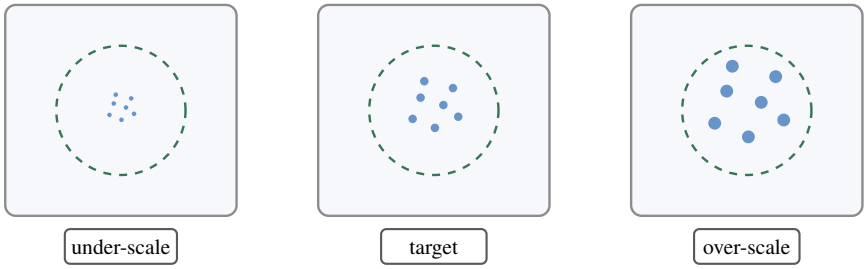


Figure 15. **Energy budget.** The energy budget keeps the average per-dimension scale of q and p near a target range. It penalizes both under-scaled states (which risk collapse toward zero) and over-scaled states (which can destabilize dynamics), so the latent cloud stays at a controlled overall radius.

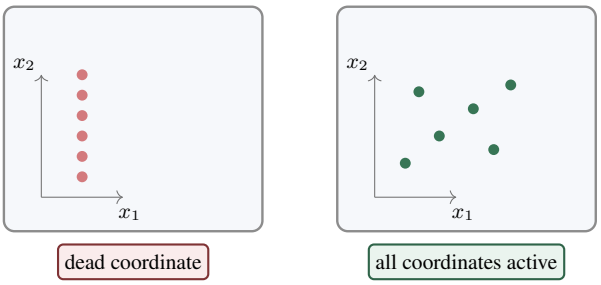


Figure 16. **Variance floor.** The variance floor prevents individual coordinates from becoming nearly constant across the batch. Visually, it revives dead axes: instead of allowing the latent cloud to collapse onto a lower-dimensional direction, it forces each coordinate to retain a minimum amount of variation.

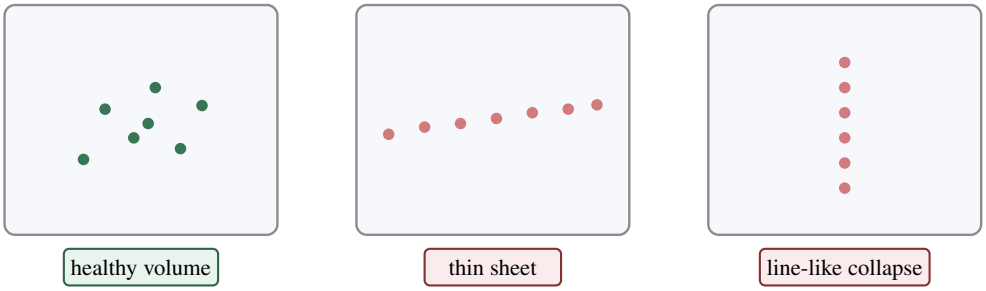


Figure 17. **Projected log-det floor.** The projected log-det floor prevents the batch representation from flattening into a very low-volume set. It does not require isotropy, but it does force the projected covariance to retain nontrivial volume, ruling out extreme degeneracies such as line-like or sheet-like collapse.

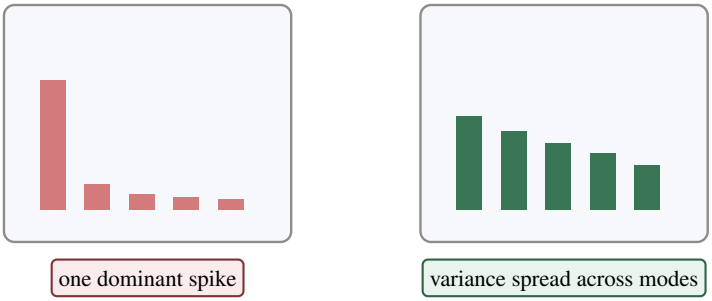


Figure 18. **Participation-ratio floor and top-eigenvalue control.** These penalties prevent the representation from surviving only through one huge dominant direction. The participation-ratio floor encourages variance to remain spread across multiple modes, while the top-eigenvalue ceiling prevents a single spike from taking over the spectrum.

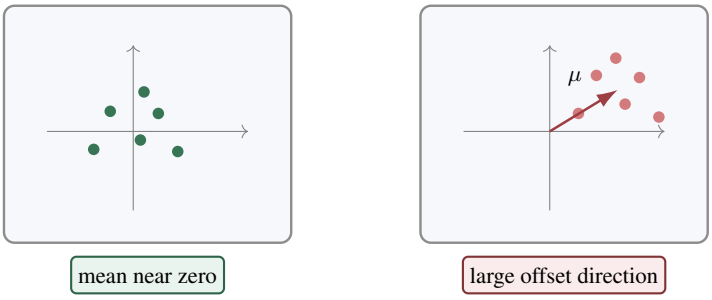


Figure 19. **Mean penalty.** A large global mean can create a dominant offset direction that makes random-pair cosine similarities look artificially cone-like. The mean penalty discourages this by keeping the batch mean closer to zero, so the geometry is not dominated by a single global shift.

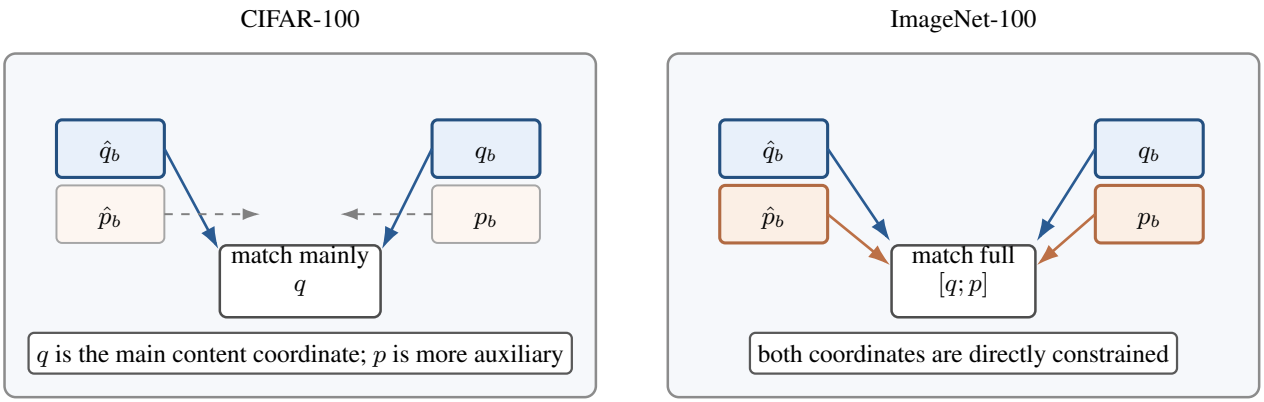


Figure 20. **Why CIFAR-100 and ImageNet-100 use different matching choices.** On CIFAR-100 the training objective emphasizes the content coordinate q, so q is the cleanest downstream readout and p mainly supports the dynamics. On ImageNet-100 the stable regime was to match the full state $[q; p]$, which directly constrains both coordinates and makes p much more informative.

How to read the sequence. Taken together, Figs. 10–20 decompose the full method into three conceptual layers: the encoder constructs a structured phase-space state, the predictor evolves that state with a symplectic Hamiltonian rollout, and the regularizers ensure that the encoder supplies a non-degenerate cloud of states for those dynamics to act on. The distinctive point is therefore twofold: HamJEPA changes both the *representation parameterization* (from a single embedding to a phase-space state) and the *predictor family* (from arbitrary regression to Hamiltonian dynamics), while avoiding the need to force the encoder marginal toward Euclidean isotropy.

E. Implementation Details and Algorithms

In this section (as in the code) we refer to *HamJEPA* as *MV-HJEPA*, denoting the minimal multi-view implementation used in our experiments.

E.1. Two training modes: MV-HJEPA vs. baseline (LeJEPA + regularizer)

Our training scripts are *mode-switched* by the presence of a `hjepa:` block in the YAML:

- **MV-HJEPA mode** (`hjepa` present): we treat the representation as a phase-space state $z = [q; p] \in \mathbb{R}^{2d}$ and train a Hamiltonian flow predictor that maps between two global views. We additionally apply scale/diversity regularizers on *all* views.
- **Baseline mode** (`hjepa` absent): we use the *LeJEPA* prediction objective where all views predict the mean of the global views, and we optionally add a distributional regularizer such as SIGReg.

E.2. Data pipeline and multi-view batching (CIFAR100 vs. ImageNet)

Multi-crop dataset contract. Each dataset example yields a list of V augmented views (multi-crop), plus labels:

$$(\{x^{(v)}\}_{v=1}^{V}, y, y_{\text{coarse}}).$$

In our configurations, we set `num_global_views=2` and `num_local_views=0`, so $V = 2$.

CIFAR100MultiCrop. `eval/datasets/cifar100_multicrop.py` applies the full augmentation *in the worker* and returns *float tensors already normalized* (using `ToTensor()` + `Normalize`). Notable defaults:

- `out_size=32` (CIFAR resolution)
- blur uses a small kernel (`kernel_size=3`)

ImageNetMultiCrop. `eval/datasets/imagenet_multicrop.py` applies the same high-level augmentation pattern, but returns *uint8 tensors* using `PILToTensor()` to reduce dataloader IPC bandwidth. We then convert to float and normalize *on device* inside the ImageNet training loop. Notable defaults:

- `out_size=224`
- blur uses a larger kernel (`kernel_size=23`) appropriate for higher resolution
- global crop scale defaults match the YAML (`global_scale=[0.4,1.0]`)

Multi-view batching optimization. For computational efficiency, we concatenate views along the batch dimension and run a *single* encoder+projector forward:

$$x_{\text{cat}} = \text{concat}\left(x^{(1)}, \ldots, x^{(V)}\right) \in \mathbb{R}^{(VB)\times C\times H\times W}, \qquad z_{\text{cat}} = P(E(x_{\text{cat}})) \in \mathbb{R}^{(VB)\times K}.$$

We then reshape:

$$z_{\text{views}} \in \mathbb{R}^{V\times B\times K}, \qquad z_{\text{views}} = \text{reshape}(z_{\text{cat}}, (V, B, K)).$$

This is exactly how both `train_cifar_hamjepa.py` and `train_imagenet_hamjepa.py` operate.

E.3. Encoder and token-state parameterization

ResNet18 backbone with dataset-specific stem. We use `torchvision.models.resnet18(weights=None)` and alter the stem depending on the dataset:

- **CIFAR stem** (`encoder_stem=cifar`): `conv1` is replaced by a 3×3 stride-1 conv and `maxpool` is removed.
- **ImageNet stem** (`encoder_stem=imagenet`): keep the default 7×7 stride-2 conv and maxpool.

Token mode (used in all shown configs). With `encoder_mode=tokens`, we expose an intermediate feature map at a chosen ResNet layer (`token_layer` in {layer2, layer3, layer4}), project its channels with a 1×1 conv to `token_d_f`, optionally pool to a fixed spatial grid (`token_hw`), then flatten.

Concretely, if the token feature map has shape `[B, token_d_f, h, w]`, the flattened representation is

$$z \in \mathbb{R}^{B\times(hw\,\texttt{token_d_f})}.$$

This is why `embed_dim` must match hw `token_d_f`:

- CIFAR100: `layer3` naturally yields an 8×8 map, and with `token_d_f=32` we get $8 \cdot 8 \cdot 32 = 2048$.
- ImageNet: `layer3` yields 14×14 pre-pool; we set `token_hw=8` so adaptive pooling yields 8×8, again giving 2048.

Projector. We support either an MLP projector or an identity projector. In MV-HJEPA we *require* `projector_type=identity` and enforce it in code, because we interpret the representation channels as structured phase-space coordinates and do not want a learned MLP to arbitrarily mix them.

Phase-space split: $z = [q; p]$. In MV-HJEPA we require an even representation dimension $K = 2d$, and interpret:

$$z = \begin{bmatrix} q \\ p \end{bmatrix} \in \mathbb{R}^{2d}, \qquad q \in \mathbb{R}^d,\ p \in \mathbb{R}^d.$$

This split is implemented *channel-wise per token* in the encoder when `split_qp=true`: the token channel dimension `token_d_f` is split into equal halves (first half $\to q$, second half $\to p$) before flattening and concatenation.

E.4. Hamiltonian flow predictor (MV-HJEPA)

Hamiltonian dynamics. We predict one view's state from another by integrating Hamiltonian dynamics in latent phase space. Given a Hamiltonian $H(q, p)$, the vector field is:

$$\frac{dq}{dt} = \frac{\partial H}{\partial p}, \qquad \frac{dp}{dt} = -\frac{\partial H}{\partial q}.$$

In code, $\partial H/\partial q$ and $\partial H/\partial p$ are obtained via automatic differentiation (`torch.autograd.grad(create_graph=True)`), so gradients propagate through the integrator.

Hamiltonian parameterizations. `hamjepa/hamiltonian.py` supports:

1. **Quadratic**: $H(q, p) = \frac{1}{2}(\|q\|^2 + \|p\|^2)$ (no learnable parameters).
2. **Learnable (residual MLP)**:
$$H_\theta(q, p) = \tfrac{1}{2}(\|q\|^2 + \|p\|^2) + s\, f_\theta([q, p]),$$
where s (`residual_scale`) is a stabilizing residual multiplier.
3. **Separable (used for MV-HJEPA)**:
$$H_\phi(q, p) = T(p) + V_\phi(q), \qquad T(p) = \tfrac{1}{2}\|p\|^2, \qquad V_\phi(q) = \tfrac{1}{2}\alpha\|q\|^2 + s\, f_\phi(q),$$
where α (`base_coeff`) is a fixed quadratic base coefficient and s (`residual_scale`) is a residual scaling factor.

The separable form enables an explicit velocity-Verlet/leapfrog update that is symplectic.

Residual-scale warmup. In MV-HJEPA, we linearly warm up `residual_scale` over `residual_scale_warmup_epochs`. Early in training the potential is close to purely quadratic (stable), and the learned residual is introduced gradually.

Step size and fp32 integration. We integrate for `steps=S` steps with step size `dt` (fixed in MV-HJEPA; `learn_dt=false`). For mixed precision runs (bf16), we optionally do the dynamics computations in fp32 and cast back (`integrate_fp32=true`) to stabilize higher-order autodiff through the integrator.

E.5. Symplectic integrators

We implement:

- **Symplectic Euler** for general $H(q, p)$.
- **Leapfrog / velocity-Verlet**:
  - a general variant using autodiff for $\nabla_q H$ and $\nabla_p H$
  - a specialized separable variant (used by MV-HJEPA when $H = T + V$) using only $\nabla V(q)$

For separable $H(q, p) = \frac{1}{2}\|p\|^2 + V(q)$, the specialized update is:

$$p_{1/2} = p - \tfrac{\Delta t}{2}\nabla V(q), \quad q \leftarrow q + \Delta t\, p_{1/2}, \quad p \leftarrow p_{1/2} - \tfrac{\Delta t}{2}\nabla V(q).$$

E.6. MV-HJEPA prediction loss

Given two global views' states $z_a = [q_a; p_a]$ and $z_b = [q_b; p_b]$, we predict:

$$\hat{z}_b = \mathrm{Flow}_H(z_a; \Delta t, S),$$

and compute a MSE matching term either on q only or the full (q, p) state:

$$\mathcal{L}_{\text{match}} = \begin{cases} \text{MSE}(\hat{q}_b, q_b) + w_p \, \text{MSE}(\hat{p}_b, p_b) & (\text{match} = q) \\ \text{MSE}(\hat{z}_b, z_b) & (\text{match} = qp). \end{cases}$$

We typically stop-gradient the target branch (`detach_target=true`) to prevent trivial collapse.

Optionally, we add an energy-consistency term that encourages the flow to preserve the input energy:

$$\mathcal{L}_{\text{energy}} = \text{MSE}\big(H(q_a, p_a),\ H(\hat{q}_b, \hat{p}_b)\big),$$

and the full prediction loss is:

$$\mathcal{L}_{\text{pred}} = \mathcal{L}_{\text{match}} + \lambda_E \mathcal{L}_{\text{energy}}.$$

Bidirectional training. If enabled, we symmetrize by predicting in both directions.

$$\mathcal{L}_{\text{pred}} \leftarrow \tfrac{1}{2} \left(\mathcal{L}_{\text{pred}}(z_0, z_1) + \mathcal{L}_{\text{pred}}(z_1, z_0) \right).$$

In the most physically consistent variant, the reverse direction uses reverse-time integration (negated Δt), which is supported by the predictor's `direction` flag.

E.7. Regularizers (applied across all views)

We compute regularizers on the concatenation of all views' states:

$$z_{\text{all}} \in \mathbb{R}^{(VB) \times 2d}.$$

Phase-space energy budget (scale control). We stabilize scale in fixed units by enforcing per-coordinate second moments:

$$\mathcal{L}_{\text{budget}} = \big(\mathbb{E}[q^2] - q_{\text{tgt}}\big)^2 + \big(\mathbb{E}[p^2] - p_{\text{tgt}}\big)^2,$$

where expectations are taken over batch and coordinates in fp32 and can be Distributed Data Parallel (DDP)-synchronized.

Variance floor (anti-collapse). Let σ_i be the batch standard deviation of dimension i (fp32 stats). We apply:

$$\mathcal{L}_{\text{var}} = \frac{1}{d} \sum_{i=1}^{d} \max(0, \sigma_{\min} - \sigma_i)^2,$$

typically on q and optionally on p depending on the experiment.

Projected log-determinant floor (spectral diversity). We enforce a lower bound on the log-determinant of a randomly projected covariance. Given $x \in \mathbb{R}^{B \times d}$, sample an orthonormal projection $R \in \mathbb{R}^{d \times k}$, compute $y = (x - \bar{x})R$, then:

$$C = \text{Cov}(y) + \varepsilon I, \qquad \mathcal{L}_{\log\det} = \max\Big(0, \tau - \tfrac{1}{k} \log\det(C)\Big)^2.$$

We optionally add constraints on the participation ratio $\text{PR} = \text{tr}(C)^2 / \text{tr}(C^2)$ and the top-eigenvalue fraction $\lambda_{\max}/\text{tr}(C)$.

Mean penalty. We penalize non-zero feature means using the squared mean (fp32):

$$\mathcal{L}_{\text{mean}} \propto \|\mathbb{E}[z]\|_2^2.$$

E.8. Training step summary

MV-HJEPA. At each step:

1. Build V views per image, concatenate them, and compute $z_{\text{views}} \in \mathbb{R}^{V \times B \times 2d}$.
2. Select the two global views (z_0, z_1) and compute $\mathcal{L}_{\text{pred}}$ via Hamiltonian flow prediction.
3. Compute $\mathcal{L}_{\text{budget}}$, $\mathcal{L}_{\text{var}}$, $\mathcal{L}_{\log\det}$, and $\mathcal{L}_{\text{mean}}$ on all views.
4. Optimize encoder + (identity) projector + predictor parameters with AdamW, optional gradient clipping, and cosine LR schedule with warmup.

Baseline (LeJEPA + SIGReg/HamSIGReg). If `hjepa` is absent:

1. Compute z_{views} as above.
2. Apply LeJEPA prediction loss: all views match the per-sample mean of global views.
3. Add a distributional regularizer (e.g. SIGReg) with weight `lambda_reg`.
4. If the regularizer includes a learnable metric H, optionally update H on a slower/periodic schedule.

E.9. Algorithms

Notation. Let V be the number of augmented views per image, B the batch size, and K the representation dimension. In MV-HJEPA we enforce $K = 2d$ and interpret each representation as a phase-space state $z = [q; p] \in \mathbb{R}^{2d}$ with $q, p \in \mathbb{R}^d$ (channel-wise split, `split_qp=true`). We write $\mathrm{concat}_v x^{(v)}$ for concatenation of views along the batch dimension, and $\mathrm{reshape}$ for a view/batch reshape with the same underlying memory order as the implementation.

Algorithm 2 Multi-crop view generation (used by `CIFAR100MultiCrop` and `ImageNetMultiCrop`)

Require: Image I, multi-crop config: (V_g, V_ℓ), output size s, scales (α_g, β_g) and $(\alpha_\ell, \beta_\ell)$
Ensure: List of views $\{x^{(v)}\}_{v=1}^{V}$ where $V = V_g + V_\ell$

```
Initialize empty list views ← [ ]
for i = 1 to V_g do
    x ← Augment(I; RandomResizedCrop(s, scale = [α_g, β_g]), HFlip, CJ, Gray, Blur, Solarize)
    Append x to views
end for
for i = 1 to V_ℓ do
    x ← Augment(I; RandomResizedCrop(s, scale = [α_ℓ, β_ℓ]), HFlip, CJ, Gray, Blur, Solarize)
    Append x to views
end for
return views
```

Implementation note (dataset-specific): CIFAR views are produced as float tensors with normalization applied in the dataset pipeline; ImageNet views are emitted as compact `uint8` tensors (via `PILToTensor`) and normalized on-device in the training loop.

Algorithm 3 ResNet token encoder forward pass (token mode, optional q/p split)

Require: Image batch $x \in \mathbb{R}^{B\times 3\times H\times W}$, ResNet stem+layers, token layer $\ell \in \{\texttt{layer2}, \texttt{layer3}, \texttt{layer4}\}$, channel projection to d_f, optional pooling to $\texttt{token_hw} = h = w$, `split_qp` flag
Ensure: Representation $z \in \mathbb{R}^{B\times K}$ where $K = h \cdot w \cdot d_f$

```
u ← stem(x)
u ← layer1(u)
u ← layer2(u); set fmap ← u if ℓ = layer2
u ← layer3(u); set fmap ← u if ℓ = layer3
u ← layer4(u); set fmap ← u if ℓ = layer4
if token_hw is set then
    fmap ← AdaptiveAvgPool2d(fmap, (h, w))
end if
t ← Conv1 × 1 + BN(fmap) ∈ R^{B×d_f×h×w}
t ← permute(t) ∈ R^{B×h×w×d_f}
if split_qp = false then
    z ← flatten(t) ∈ R^{B×(hwd_f)}
else
    Split channels: t ↦ (t_q, t_p) with t_q, t_p ∈ R^{B×h×w×(d_f/2)}
    q ← flatten(t_q) ∈ R^{B×(hw(d_f/2))};  p ← flatten(t_p) ∈ R^{B×(hw(d_f/2))}
    z ← [q; p] ∈ R^{B×(hwd_f)}
```

end if
return z

Algorithm 4 MV-HJEPA training step (matches `train_*_hamjepa.py` when `hjepa:` is present)

Require: Views $\{x^{(v)}\}_{v=1}^{V}$, encoder E, projector P (identity for MV-HJEPA), predictor Flow_H, loss/regularizer weights $\lambda_\bullet$
Require: MV-HJEPA requires `split_qp=true` and $K = 2d$
$x_{\text{cat}} \leftarrow \mathrm{concat}_v\, x^{(v)} \in \mathbb{R}^{(VB)\times C\times H\times W}$
$z_{\text{cat}} \leftarrow P(E(x_{\text{cat}})) \in \mathbb{R}^{(VB)\times 2d}$
$z_{\text{views}} \leftarrow \mathrm{reshape}(z_{\text{cat}}) \in \mathbb{R}^{V\times B\times 2d}$
$z_0 \leftarrow z_{\text{views}}[0],\ \ z_1 \leftarrow z_{\text{views}}[1]$
$\mathcal{L}_{\text{pred}} \leftarrow \mathrm{HamConsistency}(z_0, z_1)$
if `bidirectional` **then**
    $\mathcal{L}_{\text{pred}} \leftarrow \frac{1}{2}\left(\mathcal{L}_{\text{pred}} + \mathrm{HamConsistency}(z_1, z_0)\right)$
end if
$z_{\text{all}} \leftarrow z_{\text{views}}.\mathrm{reshape}(VB, 2d)$
Split $z_{\text{all}} \mapsto (q_{\text{all}}, p_{\text{all}})$ along the last dimension
$\mathcal{L}_{\text{budget}} \leftarrow \mathrm{EnergyBudget}(z_{\text{all}})$
$\mathcal{L}_{\text{var}} \leftarrow \mathrm{VarFloor}(q_{\text{all}})$ $(+\ \mathrm{VarFloor}(p_{\text{all}})$ if enabled in that script$)$
$\mathcal{L}_{\log\det} \leftarrow \mathrm{LogDetFloor}(q_{\text{all}}) + \mathrm{LogDetFloor}(p_{\text{all}})$
$\mathcal{L}_{\text{mean}} \leftarrow \mathrm{mean}(z_{\text{all}})^{\odot 2}$ averaged over dimensions {matches `mean().square().mean()`}
$\mathcal{L} \leftarrow \mathcal{L}_{\text{pred}} + \lambda_{\text{budget}}\mathcal{L}_{\text{budget}} + \lambda_{\text{var}}\mathcal{L}_{\text{var}} + \lambda_{\log\det}\mathcal{L}_{\log\det} + \lambda_{\text{mean}}\mathcal{L}_{\text{mean}}$
Update parameters of $\{E, P, \mathrm{Flow}_H\}$ with AdamW; optionally clip gradients and step LR schedule

Algorithm 5 Hamiltonian Consistency loss (`HamiltonianConsistencyLoss`)

Require: States $z_a, z_b \in \mathbb{R}^{B\times 2d}$, predictor Flow_H, options: `detach_target`, `match` $\in$ {`q`, `qp`}, `p_weight`, `energy_weight`, `bidirectional`
Ensure: Scalar loss $\mathcal{L}$
$\widehat{z}_{a\to b} \leftarrow \mathrm{Flow}_H(z_a; \mathrm{direction} = +1)$
$z_b^\star \leftarrow z_b$ if not `detach_target` else $\mathrm{stopgrad}(z_b)$
if `match=qp` **then**
    $\ell_{a\to b} \leftarrow \mathrm{MSE}(\widehat{z}_{a\to b}, z_b^\star)$
else{`match=q`}
    $\ell_{a\to b} \leftarrow \mathrm{MSE}(\widehat{z}_{a\to b}[:, :d], z_b^\star[:, :d])$
    if `p_weight` > 0 **then**
        $\ell_{a\to b} \leftarrow \ell_{a\to b} + \texttt{p_weight}\cdot \mathrm{MSE}(\widehat{z}_{a\to b}[:, d:], z_b^\star[:, d:])$
    end if
end if
if `energy_weight` > 0 **then**
    Split $z_a \mapsto (q_a, p_a)$ and $\widehat{z}_{a\to b} \mapsto (\hat{q}, \hat{p})$
    $H_a \leftarrow \mathrm{stopgrad}\big(H(q_a, p_a)\big);\ \ H_{\hat{z}} \leftarrow H(\hat{q}, \hat{p})$
    $\ell_{a\to b} \leftarrow \ell_{a\to b} + \texttt{energy_weight}\cdot \mathrm{MSE}(H_{\hat{z}}, H_a)$
end if
$\mathcal{L} \leftarrow \ell_{a\to b}$
if `bidirectional` **then**
    Compute $\ell_{b\to a}$ identically using $\mathrm{direction} = -1$ and target z_a (optionally detached)
    $\mathcal{L} \leftarrow \frac{1}{2}(\ell_{a\to b} + \ell_{b\to a})$
end if
return $\mathcal{L}$

Algorithm 6 Hamiltonian Flow Predictor (forward pass, `HamiltonianFlowPredictor`)

Require: State $z_0 \in \mathbb{R}^{\cdots\times 2d}$, Hamiltonian module H, method, steps S, base step size $\Delta t > 0$, direction $s \in \{+1, -1\}$,

`integrate_fp32`, `learn_dt`

Ensure: State $z_T \in \mathbb{R}^{\cdots \times 2d}$

Split $z_0 \mapsto (q_0, p_0)$
$\Delta t \leftarrow s \cdot \Delta t$; if `learn_dt`, set $\Delta t \leftarrow s \cdot \mathrm{softplus}(\texttt{raw_dt})$
if `integrate_fp32` enabled **and** $\mathrm{dtype}(z_0) \in \{\mathrm{fp16}, \mathrm{bf16}\}$ **then**
    Cast (q_0, p_0) to fp32
end if
if H is separable **and** method is leapfrog **then**
    $(q_T, p_T) \leftarrow \mathrm{IntegrateSeparableLeapfrog}(V_\phi, q_0, p_0, \Delta t, S)$
else
    $(q_T, p_T) \leftarrow \mathrm{IntegrateHamiltonian}(H, q_0, p_0, \Delta t, S, \mathrm{method})$
end if
$z_T \leftarrow [q_T; p_T]$
if `integrate_fp32` enabled **and** original dtype was bf16/fp16 **then**
    Cast z_T back to the original dtype
end if
return z_T

Algorithm 7 Symplectic integration routines (as in `hamjepa/integrators.py`)

Require: General Hamiltonian $H(q, p)$ or separable potential $V(q)$, initial (q, p), step size Δt, steps S, method $\in \{\texttt{leapfrog}, \texttt{symplectic_euler}\}$

Ensure: Final (q, p) after S steps

function INTEGRATEHAMILTONIAN($H, q, p, \Delta t, S$, method)
    for $n = 1$ to S **do**
        if method = symplectic_euler **then**
            $p \leftarrow p - \Delta t\, \nabla_q H(q, p)$
            $q \leftarrow q + \Delta t\, \nabla_p H(q, p)$ $\{\nabla_p$ uses updated $p\}$
        else{method = leapfrog (velocity Verlet)}
            $p_{1/2} \leftarrow p - \frac{\Delta t}{2} \nabla_q H(q, p)$
            $q \leftarrow q + \Delta t\, \nabla_p H(q, p_{1/2})$
            $p \leftarrow p_{1/2} - \frac{\Delta t}{2} \nabla_q H(q, p_{1/2})$
        end if
    end for
    return (q, p)
end function
function INTEGRATESEPARABLELEAPFROG($V, q, p, \Delta t, S$)
    for $n = 1$ to S **do**
        $p_{1/2} \leftarrow p - \frac{\Delta t}{2} \nabla_q V(q)$
        $q \leftarrow q + \Delta t\, p_{1/2}$
        $p \leftarrow p_{1/2} - \frac{\Delta t}{2} \nabla_q V(q)$ $\{\nabla_q V$ uses updated $q\}$
    end for
    return (q, p)
end function

Algorithm 8 Phase-space energy budget regularizer (`PhaseSpaceEnergyBudget`)

Require: States $z \in \mathbb{R}^{\cdots \times 2d}$, targets $(q_{\text{tgt}}, p_{\text{tgt}})$, `ddp_sync`

Ensure: Scalar loss $\mathcal{L}_{\text{budget}}$

Flatten $z \to z_f \in \mathbb{R}^{N \times 2d}$ and cast to fp32
Split $z_f \mapsto (q, p)$ with $q, p \in \mathbb{R}^{N \times d}$
$q2_sum \leftarrow \sum q^{\odot 2}$, $p2_sum \leftarrow \sum p^{\odot 2}$, `count` $\leftarrow Nd$
if `ddp_sync` and DDP initialized **then**
    All-reduce (`SUM`) $q2_sum$, $p2_sum$, and `count`
end if

$q2_\mu \leftarrow q2_sum/\texttt{count}$; $p2_\mu \leftarrow p2_sum/\texttt{count}$
$\mathcal{L}_{\text{budget}} \leftarrow (q2_\mu - q_{\text{tgt}})^2 + (p2_\mu - p_{\text{tgt}})^2$
return $\mathcal{L}_{\text{budget}}$

Algorithm 9 Variance floor regularizer (`VarianceFloor`)

Require: Features $x \in \mathbb{R}^{N\times d}$, std floor $\sigma_{\min}$, ε, `ddp_sync`
Ensure: Scalar loss $\mathcal{L}_{\text{var}}$
Cast x to fp32; if $N < 2$ return 0
Compute sufficient stats: $s_1 \leftarrow \sum_n x_n$, $s_2 \leftarrow \sum_n x_n^{\odot 2}$, and $n \leftarrow N$
if `ddp_sync` and DDP initialized **then**
    All-reduce (`SUM`) s_1, s_2, and n
end if
$\mu \leftarrow s_1/n$; $\text{var} \leftarrow s_2/n - \mu^{\odot 2}$; $\sigma \leftarrow \sqrt{\max(\text{var}, 0) + \varepsilon}$
$\mathcal{L}_{\text{var}} \leftarrow \frac{1}{d}\sum_{i=1}^{d} \text{ReLU}(\sigma_{\min} - \sigma_i)^2$
return $\mathcal{L}_{\text{var}}$

Algorithm 10 Projected log-det floor (`ProjectedLogDetFloor`)

Require: Features $x \in \mathbb{R}^{B\times D}$, projection dim k, log-det floor $\ell_{\min}$, ε, refresh interval r, optional `pr_norm_floor` and `eigmax_frac_ceiling`, `ddp_sync`
Ensure: Scalar loss $\mathcal{L}_{\log\det}$
Cast x to fp32; if $B < 2$ return 0
$k \leftarrow \min(k, D, B-1)$
if need resample (first call / device change / shape change / step $\bmod r = 0$) **then**
    Sample $R \sim \mathcal{N}(0,1)^{D\times k}$ and orthonormalize columns via QR: $R \leftarrow \text{qr}(R)$
end if
Center and project: $x_c \leftarrow x - \text{mean}(x)$; $y \leftarrow x_c R \in \mathbb{R}^{B\times k}$
Compute sufficient stats: $\texttt{sum_y} \leftarrow \sum_b y_b$, $\texttt{sum_yy} \leftarrow y^\top y$, $n \leftarrow B$
if `ddp_sync` and DDP initialized **then**
    All-reduce (`SUM`) `sum_y`, `sum_yy`, and n
end if
$n \leftarrow \max(n, 2)$; $\mu_y \leftarrow \texttt{sum_y}/n$
$\Sigma \leftarrow \frac{\texttt{sum_yy} - n\cdot(\mu_y\mu_y^\top)}{n-1}$; $\Sigma \leftarrow \frac{1}{2}(\Sigma + \Sigma^\top) + \varepsilon I$
$\ell \leftarrow \frac{1}{k}\log\det(\Sigma)$ {implemented via `slogdet`}
$\mathcal{L}_{\log\det} \leftarrow \text{ReLU}(\ell_{\min} - \ell)^2$
Compute $\text{PR} = \frac{\text{tr}(\Sigma)^2}{\text{tr}(\Sigma^2)}$, $\text{pr_norm} = \text{PR}/k$, and $\text{eigmax_frac} = \lambda_{\max}/\text{tr}(\Sigma)$
if `pr_norm_floor` is set **then**
    $\mathcal{L}_{\log\det} \leftarrow \mathcal{L}_{\log\det} + \text{ReLU}(\texttt{pr_norm_floor} - \text{pr_norm})^2$
end if
if `eigmax_frac_ceiling` is set **then**
    $\mathcal{L}_{\log\det} \leftarrow \mathcal{L}_{\log\det} + \text{ReLU}(\text{eigmax_frac} - \texttt{eigmax_frac_ceiling})^2$
end if
return $\mathcal{L}_{\log\det}$

Algorithm 11 SIGReg forward pass (random slicing + Epps–Pulley statistic)

Require: Batch $z \in \mathbb{R}^{\cdots\times N\times D}$, slices K, knots $t_1, \ldots, t_T$ (with weights w), refresh interval r, `ddp_sync`
Ensure: Scalar statistic $S(z)$ (mean-reduced in our configs)
(Slicing) If step $\bmod r = 0$, sample $A \in \mathbb{R}^{D\times K}$ with i.i.d. $\mathcal{N}(0,1)$ columns and normalize each column to unit norm
if `ddp_sync` and DDP initialized **then**
    Ensure all ranks share the same sampling seed for A (broadcast seed from rank 0)
end if
Project: $y \leftarrow zA \in \mathbb{R}^{\cdots\times N\times K}$

(Epps–Pulley) Compute empirical CF on each slice:
$\hat{c}(t) = \frac{1}{N}\sum_n \cos(t\, y_n), \quad \hat{s}(t) = \frac{1}{N}\sum_n \sin(t\, y_n)$ (optionally globally averaged via all-reduce if `ddp_sync`)
Target CF for $\mathcal{N}(0,1)$ is $\phi(t) = e^{-t^2/2}$
$S_k \leftarrow N \sum_{j=1}^{T} w_j \Big[(\hat{c}(t_j) - \phi(t_j))^2 + \hat{s}(t_j)^2\Big]$ for each slice k
Reduce over slices (and leading dims) by mean: $S(z) \leftarrow \mathrm{mean}_k(S_k)$
return $S(z)$

Algorithm 12 LeJEPA baseline step with SIGReg regularization (non-MV branch in `train_*_hamjepa.py`)

Require: Views $\{x^{(v)}\}_{v=1}^{V}$, encoder E, projector P, number of global views V_g, regularizer $\mathcal{R}$ (SIGReg), weight λ_{reg}
$x_{\text{cat}} \leftarrow \mathrm{concat}_v\, x^{(v)}$
$z_{\text{cat}} \leftarrow P(E(x_{\text{cat}})) \in \mathbb{R}^{(VB)\times D}$
$z_{\text{views}} \leftarrow \mathrm{reshape}(z_{\text{cat}}) \in \mathbb{R}^{V\times B\times D}$
$\bar{z} \leftarrow \frac{1}{V_g}\sum_{v=1}^{V_g} z_{\text{views}}[v] \in \mathbb{R}^{B\times D}$ {mean of global views}
$\mathcal{L}_{\text{pred}} \leftarrow \frac{1}{2}\,\mathrm{mean}\big(\|z_{\text{views}} - \bar{z}\|_2^2\big)$ {matches `lejepa_prediction_loss`}
$\mathcal{L}_{\text{reg}} \leftarrow \mathcal{R}(z_{\text{views}})$ {SIGReg expects shape $(\cdots, N, D)$}
$\mathcal{L} \leftarrow \mathcal{L}_{\text{pred}} + \lambda_{\text{reg}}\mathcal{L}_{\text{reg}}$ {spectral term omitted here (zero in our SIGReg configs)}
Update $\{E, P\}$ with AdamW; optionally step LR schedule and log diagnostics

E.10. YAML Configuration Knobs

The repository is fully config-driven: the training scripts read a YAML file and route into either (i) MV-HJEPA mode when the `hjepa:` stanza is present (and `split_qp=true`), or (ii) the LeJEPA+regularizer baseline when `hjepa:` is absent (e.g. `regularizer.type=sigreg`). Below we document every key appearing in the provided configs, exactly as it is consumed by the code. We keep `seed=42` for reproducibility across runs.

E.10.1. Data (`data:`)

- `root`: Dataset root path passed to `CIFAR100` / `ImageFolder` loader.
- `batch_size`: DataLoader batch size (`batch_size=batch_size`). We use larger batches on CIFAR (512) and smaller on ImageNet (256) to balance GPU memory and throughput.
- `num_workers`: DataLoader worker count. Higher values improve input pipeline throughput when CPU-bound.
- `drop_last`: If true, drops the last incomplete batch. This avoids smaller last batches, which can destabilize batch statistics and covariance-based regularizers (and helps DDP batch-size consistency).
- `pin_memory` (ImageNet configs): Enables pinned host memory to speed CPU→GPU transfers.
- `persistent_workers` (ImageNet configs): Keeps workers alive across epochs to reduce worker spin-up overhead.
- `prefetch_factor` (ImageNet configs): Worker prefetch depth; increases overlap between CPU augmentation and GPU training.
- `num_global_views`, `num_local_views`: Multi-crop configuration. In all provided runs we set two global views and no local views (`2/0`), matching the two-view training objective in both MV-HJEPA and the LeJEPA baseline.
- `out_size` (ImageNet configs): Output crop resolution for `RandomResizedCrop`. We use 224 (standard ImageNet scale).
- `global_scale`, `local_scale` (ImageNet configs): Scale ranges for global/local crops. Only global crops are used when `num_local_views=0`.
- *CIFAR note:* the CIFAR YAMLs omit `out_size/global_scale/local_scale`, so the dataset defaults are used (`out_size=32`, `global_scale=(0.3,1.0)`, `local_scale=(0.05,0.3)`).

E.10.2. MODEL (`MODEL:`)

- `encoder_mode`: `global` uses GAP (global average pooling)+FC; `tokens` exposes an intermediate feature map and flattens it into tokens. All provided configs use `tokens`.
- `encoder_stem`: `cifar` modifies ResNet-18 stem (3×3 stride 1, no maxpool) for 32×32 inputs; `imagenet` keeps the default 7×7 stride 2 + maxpool stem.
- `token_layer`: Which ResNet stage to expose as the token grid (`layer2/layer3/layer4`). We use `layer3`, giving an 8×8 grid on CIFAR and a 14×14 grid on ImageNet before any optional pooling.
- `token_d_f`: Number of channels after the 1×1 token projection (`Conv1x1+BN`). In token mode, the representation dimension is $K = h \cdot w \cdot$ `token_d_f`.
- `token_hw` (ImageNet configs): Optional adaptive pooling size for the token grid. We set `token_hw=8` on ImageNet so that with `token_d_f=32` we obtain $K = 8 \cdot 8 \cdot 32 = 2048$ (matching `embed_dim`).
- `embed_dim`: Output dimension K of the encoder in token mode. For our settings this is tied to the token grid geometry (e.g. $8 \cdot 8 \cdot 32 = 2048$).
- `proj_hidden`, `proj_out`: Hidden/output sizes for the optional MLP projector. These are read from the YAML but are only used when `projector_type=mlp`.
- `projector_type`: `identity` returns z unchanged; `mlp` applies `MLPProjector`. MV-HJEPA requires `identity` so the channel-wise q/p semantics are preserved.
- `split_qp`: If true, the token channels are split into (q, p) halves per token and concatenated, enforcing $K = 2d$ and enabling the phase-space interpretation required by MV-HJEPA.

E.10.3. MV-HJEPA PREDICTOR (`HJEPA:`)

(Only present in `*_hjepa_mv.yaml` configs.)

- `hamiltonian`: Choice of Hamiltonian parameterization. We use `separable`, i.e. $H(q, p) = \frac{1}{2}\|p\|^2 + V_\phi(q)$, to enable explicit symplectic leapfrog integration.
- `method`: Integrator choice. We use `leapfrog` (velocity Verlet), implemented in `integrate_separable_leapfrog` for separable H.
- `steps`: Number of integrator steps S per forward pass. We use `2` to keep compute low while still giving a nontrivial flow.
- `dt`: Fixed step size Δt. Used as a constant buffer when `learn_dt=false`.
- `learn_dt`: If true, Δt becomes learnable via a softplus parameterization. MV-HJEPA disables this (`false`) and the training script explicitly rejects learnable-dt to avoid degenerate solutions ("dt-collapse").
- `hidden_dim`, `depth`: MLP size/depth used inside the Hamiltonian residual network (for V_ϕ in the separable case).
- `residual_scale`: Multiplier on the Hamiltonian residual output. This is a stability knob: the code is designed so the Hamiltonian has a quadratic base plus a small learned residual.
- `residual_scale_warmup_epochs`: Linearly ramps `residual_scale` from 0 to its target value over the first epochs (implemented by overwriting `predictor.H.residual_scale` each epoch).
- `base_coeff`: Coefficient on the quadratic base potential term in $V(q)$ (i.e. $0.5\,$`base_coeff`$\,\|q\|^2$). Keeping this nonzero prevents the "trivial potential" loophole where dynamics can collapse.
- `integrate_fp32`: If true, the predictor casts z to fp32 during integration when training in bf16/fp16, improving numerical stability of gradient-based integration.

E.10.4. MV-HJEPA LOSS (`LOSS:`)

- `match`: Which part of the state to match in the consistency loss: `q` matches positions only (optionally with a small p penalty), while `qp` matches the full state.
- `p_weight`: Only used when `match=q`. Adds an auxiliary MSE on p with weight `p_weight`.
- `detach_target`: If true, the target branch is detached in the prediction loss (safer default to prevent the target moving to satisfy the predictor).
- `energy_weight`: Optional extra term matching $H(z)$ before/after the flow. Set to `0.0` in all provided configs (disabled).
- `bidirectional`: If true, averages forward and backward consistency ($z_0 \to z_1$ and $z_1 \to z_0$). In the ImageNet training script this averaging is done explicitly in the loop; in the CIFAR script it can be handled inside the loss module.

E.10.5. MV-HJEPA ANTI-COLLAPSE REGULARIZERS (`REGULARIZER:`)

(These are the *phase-space* regularizers used in the MV-HJEPA branch; they are not the SIGReg baseline.)

- `q_per_dim_target`, `p_per_dim_target`: Targets for the phase-space energy budget: they control the desired per-dimension mean-square magnitude of q and p.
- `q_std_floor`: Per-dimension standard deviation floor used by `VarianceFloor` on q (and on p in the CIFAR MV script; ImageNet MV applies the variance floor to q only).
- `ddp_sync`: If true and DDP is initialized, the sufficient statistics for the budget/variance/logdet terms are aggregated across ranks using all-reduce. This makes the regularizers behave as if computed on the global batch.
- `q_logdet_proj_dim`, `p_logdet_proj_dim`: Projection dimension k for the log-det regularizer on q and p.
- `q_logdet_floor`, `p_logdet_floor`: Minimum allowed log-determinant per projected dimension (a volume floor) used by `ProjectedLogDetFloor`.
- `q_logdet_eps`, `p_logdet_eps`: Diagonal jitter εI added to the covariance before computing `slogdet`.
- `q_logdet_refresh_interval`, `p_logdet_refresh_interval`: How often the random projection matrix is resampled (larger values reduce overhead and noise by reusing projections).
- `q_pr_norm_floor`, `p_pr_norm_floor`: Optional lower bounds on normalized participation ratio PR/k (encourages higher effective rank).
- `q_eigmax_frac_ceiling`, `p_eigmax_frac_ceiling`: Optional upper bounds on $\lambda_{\max}/\mathrm{tr}(\Sigma)$ to prevent a single dominant direction.

E.10.6. TRAINING (`TRAIN:`)

- `epochs`: Total training epochs.
- `lr`: Base AdamW learning rate for encoder/projector (and predictor in CIFAR MV).
- `h_lr` (ImageNet MV config): Separate learning rate used specifically for the Hamiltonian parameters inside the predictor (`predictor.H.parameters()`); the ImageNet training script builds a two-group optimizer when these parameters exist.
- `weight_decay`: AdamW weight decay for the main optimizer.
- `warmup_epochs`: Warmup duration used by the linear warmup + cosine decay scheduler (when enabled by the script).
- `min_lr_ratio`: Sets $\eta_{\min} = \texttt{lr} \cdot \texttt{min_lr_ratio}$ for cosine annealing.

- `precision`: Mixed precision mode. `bf16` enables CUDA autocast bf16 in training.
- `ckpt_dir`: Output directory for checkpoints.
- `checkpoint_every` (ImageNet configs): Save checkpoints every k epochs (ImageNet script only).
- `grad_clip`: If > 0, applies global-norm gradient clipping (`clip_grad_norm_`).
- `log_every`: Print training diagnostics every `log_every` steps.
- `profile_timing`, `profile_every` (ImageNet MV config): Optional lightweight profiling of data time vs step time (only used in the ImageNet script).
- **MV-HJEPA loss weights:** `lambda_budget`, `lambda_var`, `lambda_logdet`, `lambda_mean` weight the corresponding regularizers in Algorithm 4.
- **SIGReg baseline weights:** `lambda_reg` is the coefficient on the SIGReg term in Algorithm 12.
- `h_update_interval`, `h_grad_clip` (SIGReg YAMLs): These knobs are parsed by the training scripts for the *learnable Hamiltonian regularizer* path (`regularizer.type=ham_sigreg` with learnable H). In the provided SIGReg-only configs (`type=sigreg`), there are no H parameters, so these settings are effectively inert.